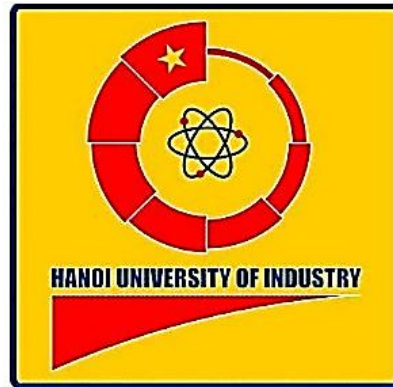


ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CƠ KHÍ – Ô TÔ
KHOA CƠ ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP
ROBOT VÀ TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI

**NGHIÊN CỨU, THIẾT KẾ VÀ CHẾ TẠO ROBOT DI ĐỘNG
TỰ HÀNH ỨNG DỤNG TRÍ TUỆ NHÂN TẠO VÀ THỊ GIÁC
MÁY TÍNH TRONG NHẬN DIỆN, XÁC ĐỊNH VÀ VẬN
CHUYỂN VẬT THỂ THEO ĐƯỜNG LINE**

Giảng viên hướng dẫn: TS. Phạm Tiến Hùng

Nhóm sinh viên thực hiện: Lê Văn Thắng 2022607503

Nguyễn Bùi Tấn Dũng 2022600778

Nguyễn Minh Vương 2022604515

Hà Nội – 2026

MỤC LỤC

MỤC LỤC	i
DANH MỤC HÌNH ẢNH	iv
DANH MỤC BẢNG BIỂU	vii
DANH MỤC CÁC TỪ VIẾT TẮT	viii
MỞ ĐẦU	ix
CHƯƠNG 1: GIỚI THIỆU CHUNG.....	1
1.1. LỊCH SỬ NGHIÊN CỨU	1
1.2. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU.....	3
1.3. MỤC TIÊU NGHIÊN CỨU ĐỀ TÀI	4
1.4. PHƯƠNG PHÁP THỰC HIỆN	4
1.5. DỰ KIẾN KẾT QUẢ ĐẠT ĐƯỢC	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	7
2.1. TỔNG QUAN VỀ ROBOT DI ĐỘNG TỰ HÀNH	7
2.2. PHƯƠNG TRÌNH ĐỘNG LỰC HỌC CỦA ROBOT VI SAI	8
2.3. PHƯƠNG PHÁP DÒ LINE CHO ROBOT.....	10
2.3.1. Nguyên lý hoạt động của phương pháp dò line	10
2.3.2. Dò line sử dụng bộ điều khiển PID.....	11
2.4. TỔNG QUAN VỀ TRÍ TUỆ NHÂN TẠO VÀ THỊ GIÁC MÁY TÍNH	12
2.4.1. Khái niệm	12
2.4.2. Học sâu và Mô hình mạng tích chập (CNN)	13
2.4.3. Mô hình YOLOv8 và cấu trúc Backbone, Neck, Head	14

2.5. PHƯƠNG PHÁP NHẬN DIỆN VẬT THỂ BẰNG CAMERA.....	18
2.6. GIỚI THIỆU CÁC PHẦN CỨNG SỬ DỤNG TRONG HỆ THỐNG	20
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	22
3.1. PHÂN TÍCH YÊU CẦU HỆ THỐNG	22
3.2. THIẾT KẾ TỔNG THỂ HỆ THỐNG ROBOT	24
3.3. THIẾT KẾ HỆ THỐNG CƠ KHÍ CỦA ROBOT.....	24
3.3.1. Khung robot	24
3.3.2. Cơ cấu tay gấp.....	25
3.3.3. Hệ thống truyền động.....	26
3.4. THIẾT KẾ HỆ THỐNG ĐIỆN VÀ ĐIỀU KHIỂN	27
3.4.1. Sơ đồ kết nối Arduino	27
3.4.2. Sơ đồ kết nối Raspberry Pi.....	28
3.4.3. Kết nối các cảm biến.....	29
3.5. THIẾT KẾ SƠ ĐỒ KHỐI CỦA HỆ THỐNG ROBOT	30
CHƯƠNG 4: THUẬT TOÁN ĐIỀU KHIỂN VÀ XÂY DỰNG CHƯƠNG TRÌNH.....	32
4.1. THUẬT TOÁN ĐIỀU KHIỂN ROBOT BẮM LINE	32
4.1.1. Cơ chế đọc cảm biến và tính toán sai số	32
4.1.2. Hệ thống Máy trạng thái	32
4.1.3. Thuật toán điều khiển tỷ lệ - tích phân - vi phân (PID)	33
4.1.4. Phương pháp xử lý sai số	34
4.2. THUẬT TOÁN NHẬN DIỆN VẬT THỂ BẰNG AI	35
4.2.1. So sánh các mô hình học sâu	35
4.2.2. Huấn luyện mô hình nhận diện đối tượng bằng YOLOv8.....	36

4.2.3. Phân tích kết quả train.....	40
4.2.4. Hậu xử lý và lọc bỏ vật thể trùng lặp.....	46
4.3. THUẬT TOÁN XÁC ĐỊNH VỊ TRÍ VẬT THỂ.....	48
4.3.1. Phân tích và trích xuất tham số hình học của đối tượng.....	48
4.3.2. Xác định vị trí tiếp cận chính xác	48
4.3.3. Chuỗi logic "Xác nhận" và “Bắt giữ”	49
4.4. THUẬT TOÁN ĐIỀU KHIỂN TAY GẤP.....	49
4.5. LƯU ĐỒ THUẬT TOÁN HOẠT ĐỘNG CỦA ROBOT	52
4.6. XÂY DỰNG CHƯƠNG TRÌNH ĐIỀU KHIỂN CHO ARDUINO ...	55
CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG	58
5.1. XÂY DỰNG MÔ HÌNH ROBOT THỰC TẾ	58
5.1.1. Lắp ráp hệ thống cơ khí	58
5.1.2. Tích hợp hệ thống điện, cảm biến và điều khiển	59
5.1.3. Tổng thể mô hình sau khi hoàn thiện.....	61
5.2. THỬ NGHIỆM KHẢ NĂNG BẮM LINE CỦA ROBOT.....	63
5.3. THỬ NGHIỆM NHẬN DIỆN VẬT THỂ BẰNG CAMERA	65
5.4. THỬ NGHIỆM GẤP VÀ VẬN CHUYỂN VẬT THỂ.....	67
5.5. ĐÁNH GIÁ KẾT QUẢ THỰC NGHIỆM	68
5.6. HẠN CHẾ CỦA HỆ THỐNG VÀ HƯỚNG PHÁT TRIỂN	69
5.6.1. Hạn chế của hệ thống.....	69
5.6.2. Phương hướng phát triển.....	70
TÀI LIỆU THAM KHẢO.....	71
PHỤ LỤC	73

DANH MỤC HÌNH ẢNH

Hình 1.1: Robot tự hành Shakey	1
Hình 1.2: Robot Khepera	2
Hình 1.3: Robot AGV trong công nghiệp	3
Hình 2.1: Sơ đồ biểu diễn Vector động học của Robot vi sai.....	8
Hình 2.2: Trí tuệ nhân tạo	13
Hình 2.3: Mạng nơ ron học sâu phân tích xe ô tô.....	13
Hình 2.4: Mô hình CNN.....	14
Hình 2.5: Kiến trúc của YOLOv8.....	14
Hình 2.6: Mô tả thực tế của bounding box	16
Hình 2.7: Tỷ lệ IoU	18
Hình 2.8: Tập dữ liệu thực tế đã được gán nhãn cho các lớp vật thể mục tiêu trên nền tảng Roboflow	19
Hình 2.9: Logo của Google Colab	20
Hình 3.1: Vỏ và đế của robot	25
Hình 3.2: Cơ cấu tay gấp.....	26
Hình 3.3: Bố trí các linh kiện phần đế robot (Kích thước 330mm x 250mm)	27
Hình 3.4: Sơ đồ kết nối arduino	28
Hình 3.5: Sơ đồ kết nối Raspberry pi.....	29
Hình 3.6: Sơ đồ kết nối các cảm biến	30
Hình 3.7: Sơ đồ khối hệ thống	31
Hình 4.1: Lưu đồ thuật toán bám line sử dụng PID.....	35
Hình 4.2: Tập dữ liệu ảnh chứa các đối tượng được nhận diện.....	36

Hình 4.3: Gán nhãn các khối màu.....	37
Hình 4.4: Gán nhãn nước ngọt.....	37
Hình 4.5: Gán nhãn hộp thuốc	37
Hình 4.6: Gán nhãn đa vật thể.....	37
Hình 4.7: Tập dữ liệu được chia thành các tập huấn luyện, đánh giá, kiểm thử	38
Hình 4.8: Cài đặt thư viện Ultralytics	38
Hình 4.9: Tiến hành nhập dữ liệu từ Roboflow	38
Hình 4.10: Tiến hành thiết lập và huấn luyện mô hình qua 100 epochs.....	39
Hình 4.11: Hệ thống chia files	39
Hình 4.12: Chuyển đổi định dạng sang file TFLite chạy qua Raspberry Pi.....	40
Hình 4.13: Đồ thị F1 – Confidence.....	41
Hình 4.14: Đồ thị Precision-Recall	42
Hình 4.15: Đồ thị mất mát (Loss Graphs) và mAP.....	44
Hình 4.16: Đồ thị nhầm lẫn confusion matrix	45
Hình 4.17: Lưu đồ thuật toán điều khiển tay gấp.....	52
Hình 4.18: Lưu đồ thuật toán hoạt động của robot	54
Hình 5.1: Khung đế và vỏ của robot sau khi gia công.....	58
Hình 5.2: Cơ cấu chấp hành của robot sau khi gia công.....	59
Hình 5.3: Hai bộ điều khiển chính được đi dây và lắp lên robot.....	60
Hình 5.4: Cảm biến dò line và cảm biến tiệm cận từ sau khi lắp lên robot....	60
Hình 5.5: Mô hình robot sau khi hoàn thiện	62
Hình 5.6: Đồ thị so sánh sai số góc lắc lư (MPU6050) giữa hai hệ thống có PID và không có PID.....	63

Hình 5.7: Đồ thị so sánh độ lệch quỹ đạo (Cảm biến Line) giữa hai hệ thống có PID và không có PID	64
Hình 5.8: Thử nghiệm nhận diện đa vật thể trong một khung hình.....	66
Hình 5.9: Nhận diện vật thể robot chạy theo đường line và trả về tọa độ tâm vật thể	67
Hình 5.10: Thử nghiệm gấp và vận chuyển vật thể	68

DANH MỤC BẢNG BIỂU

Bảng 2.1: Tổng hợp thông số kỹ thuật các linh kiện phân cứng	21
Bảng 3.1: Danh sách yêu cầu robot.....	22
Bảng 4.1: Kết quả huấn luyện của các mô hình Yolov5 , Yolov8 và Yolov11	35
Bảng 4.2: So sánh thời gian huấn luyện và độ chính xác của các mô hình YOLO (Yolov5 , Yolov8 và Yolov11)	36
Bảng 5.1: Thông số thực tế của robot	62

DANH MỤC CÁC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh (Nguyên gốc)	Tiếng Việt (Nghĩa)
AGV	Automated Guided Vehicle	Xe tự hành
AI	Artificial Intelligence	Trí tuệ nhân tạo
AMR	Autonomous Mobile Robot	Robot di động tự hành
BLDC	Brushless Direct Current	Động cơ điện một chiều không chổi than
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
D	Demand / Design Requirement	Yêu cầu thiết kế
DC	Direct Current	Dòng điện một chiều
FN	False Negative	Dương tính giả (Bỏ sót đối tượng)
FP	False Positive	Âm tính giả (Nhận diện nhầm đối tượng)
IoU	Intersection over Union	Tỉ lệ vùng giao trên vùng hợp
mAP	Mean Average Precision	Độ chính xác trung bình (trong nhận diện)
NMS	Non-Maximum Suppression	Thuật toán ức chế phi cực đại
PID	Proportional Integral Derivative	Bộ điều khiển Tỉ lệ - Tích phân - Vi phân
PWM	Pulse Width Modulation	Điều chế độ rộng xung
SLAM	Simultaneous Localization and Mapping	Định vị và lập bản đồ đồng thời
TFLite	TensorFlow Lite	Định dạng mô hình học máy cho thiết bị nhúng
TP	True Positive	Dương tính thật (Nhận diện đúng đối tượng)
W	Wish / Want	Mong muốn (Yêu cầu mở rộng)
YOLO	You Only Look Once	Mô hình nhận diện vật thể YOLO

MỞ ĐẦU

Trong xu thế của cuộc cách mạng công nghiệp 4.0, sự kết hợp giữa Robot tự hành và Trí tuệ nhân tạo đã trở thành chìa khóa để nâng cao năng suất trong các hệ thống logistics và kho hàng thông minh. Robot ngày nay không chỉ di chuyển theo đường định sẵn mà còn cần khả năng nhận diện và tương tác linh hoạt với vật thể.

Nhằm hiện thực hóa các kiến thức về điều khiển và thị giác máy tính, nhóm sinh viên chúng em thực hiện đề tài: “Nghiên cứu, thiết kế và chế tạo robot di động tự hành ứng dụng trí tuệ nhân tạo và thị giác máy tính trong nhận diện, xác định và vận chuyển vật thể theo đường line”. Đồ án tập trung vào việc tích hợp thuật toán bám line PID ổn định trên nền tảng Arduino cùng mô hình nhận diện YOLOv8 trên Raspberry Pi 4 để thực hiện quy trình tự động hóa: di chuyển – nhận diện – gấp – phân loại vật phẩm.

Để hoàn thành đề tài này, chúng em xin bày tỏ lòng biết ơn sâu sắc tới **TS. Phạm Tiến Hùng**. Thầy đã tận tâm hướng dẫn, định hướng kỹ thuật và truyền đạt những kinh nghiệm quý báu giúp chúng em tháo gỡ những khó khăn trong suốt quá trình nghiên cứu.

Đồng thời, chúng em xin chân thành cảm ơn các thầy cô giáo tại Khoa Cơ điện tử – Trường Cơ khí - Ô tô, Đại học Công nghiệp Hà Nội đã tạo điều kiện thuận lợi và trang bị cho chúng em nền tảng kiến thức vững chắc. Dù đã nỗ lực hoàn thiện, song đề tài khó tránh khỏi những hạn chế nhất định, chúng em rất mong nhận được sự chỉ bảo và đóng góp ý kiến từ các thầy cô.

Chúng em xin chân thành cảm ơn!

Nhóm sinh viên thực hiện

Lê Văn Thắng

Nguyễn Bùi Tấn Dũng

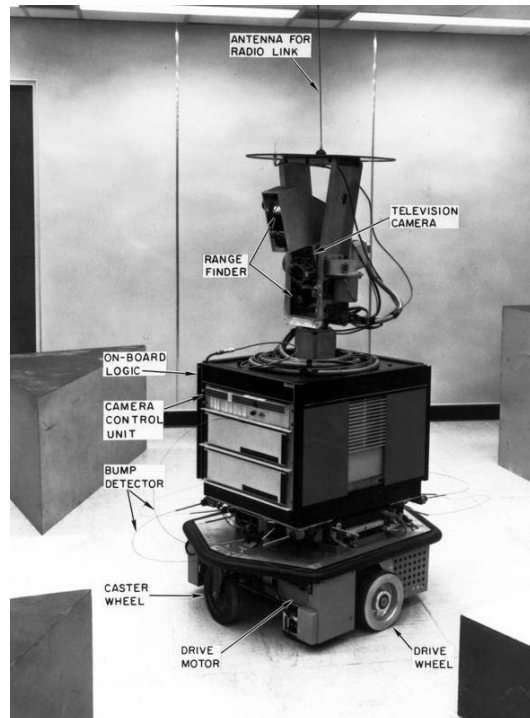
Nguyễn Minh Vương

CHƯƠNG 1: GIỚI THIỆU CHUNG

1.1. LỊCH SỬ NGHIÊN CỨU

– *Giai đoạn hình thành và phát triển ban đầu*

Những năm 1960, một cột mốc quan trọng trong lịch sử robot tự hành là sự ra đời của Shakey – robot di động đầu tiên có khả năng suy luận và tự lập kế hoạch hành động. Robot này được phát triển tại Viện nghiên cứu Stanford (SRI) trong giai đoạn 1966–1972. Shakey có thể phân tích môi trường, lập kế hoạch và thực hiện nhiệm vụ một cách tự động, đánh dấu sự kết hợp đầu tiên giữa trí tuệ nhân tạo và robot vật lý.



Hình 1.1: Robot tự hành Shakey

– *Giai đoạn phát triển hệ thống robot di động (1980–2000)*

Từ những năm 1980, robot di động bắt đầu có những bước phát triển mạnh mẽ nhờ sự tiến bộ của vi xử lý, cảm biến và thuật toán điều khiển. Một trong những hướng nghiên cứu nổi bật trong giai đoạn này là kiến trúc điều khiển phản xạ do Rodney Brooks đề xuất, cho phép robot phản ứng trực tiếp với môi trường mà không cần mô hình hóa phức tạp.

Cùng thời điểm đó, các công ty công nghiệp như Automatix đã phát triển robot tích hợp thị giác máy tính, mở ra khả năng nhận diện môi trường bằng hình ảnh.

Đến những năm 1990, các nền tảng robot nghiên cứu như Khepera được phát triển và sử dụng rộng rãi trong các phòng thí nghiệm trên toàn thế giới. Robot Khepera có kích thước nhỏ gọn, sử dụng cơ cấu vi sai và cảm biến hồng ngoại, trở thành một nền tảng tiêu chuẩn cho nghiên cứu robot tự hành.



Hình 1.2: Robot Khepera

– ***Giai đoạn phát triển robot tự hành thông minh (2000–nay)***

Từ những năm 2000 đến nay, robot di động tự hành đã có bước phát triển vượt bậc nhờ sự kết hợp của nhiều công nghệ tiên tiến như: trí tuệ nhân tạo (AI), học máy (Machine Learning), thị giác máy tính và cảm biến hiện đại.

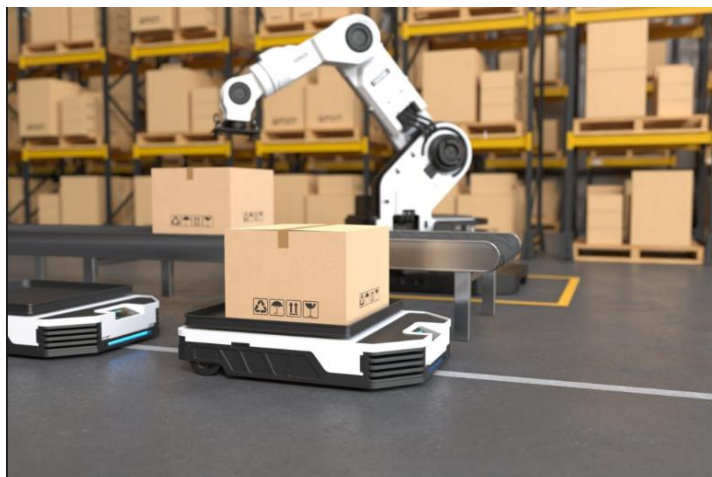
Robot không còn chỉ di chuyển theo đường định sẵn mà có thể:

- Nhận diện vật thể.
- Hiểu môi trường xung quanh.
- Tự đưa ra quyết định.

Sự chuyển dịch từ hệ thống phương tiện tự dẫn (AGV) sang các nền tảng robot di động tự hành (AMR) đánh dấu một bước tiến quan trọng, cho phép linh hoạt hóa quá trình lập kế hoạch và điều hướng phi tập trung trong các môi

trường logistics và sản xuất thông minh [1]. Theo các nghiên cứu gần đây, robot hiện đại sử dụng các công nghệ như:

- SLAM (định vị và lập bản đồ đồng thời).
- Thị giác máy tính (Computer Vision).
- Điều hướng bằng trí tuệ nhân tạo.



Hình 1.3: Robot AGV trong công nghiệp

Xu hướng nghiên cứu hiện đại đang tập trung tích hợp các thuật toán thị giác máy tính và định vị – lập bản đồ đồng thời (Visual SLAM) nhằm thay thế hoặc bổ trợ cho cảm biến LiDAR, qua đó nâng cao khả năng nhận thức không gian của robot với chi phí phần cứng được tối ưu hóa [2].

1.2. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

- Đối tượng nghiên cứu

- Robot di động bao gồm các phần cơ khí, điện-điện tử.
- Các giải pháp tích hợp trí tuệ nhân tạo, đặc biệt là thị giác máy tính để xác định, nhận diện và phân loại sản phẩm.
- Robot di động thông minh có thể tự hành trong vòng lặp khép kín, phân loại và di chuyển sản phẩm đến các vị trí cố định.

- Phạm vi nghiên cứu
 - Đề tài giới hạn trong việc thiết kế, chế tạo trong phạm vi phòng thí nghiệm chưa thể triển khai trong quy mô công nghiệp thực tế. Hệ thống bám line được thiết kế dưới dạng các đường cơ bản như đường thẳng, đường cong, các nút giao không quá phức tạp.
 - Phân xử lý ảnh tập trung vào các bài toán phân loại ở mức cơ bản đến trung bình kết hợp thêm cùng các mô hình học máy cơ bản, chưa đi sâu vào các mô hình học sâu phức tạp.

1.3. MỤC TIÊU NGHIÊN CỨU ĐỀ TÀI

Mục tiêu của đề tài là thiết kế và xây dựng một hệ thống robot di động tự hành có khả năng hoạt động tự động theo quy trình khép kín, kết hợp giữa điều khiển chuyển động và nhận diện vật thể. Cụ thể, đề tài tập trung vào các mục tiêu sau:

- Xây dựng mô hình robot di động sử dụng cơ cấu truyền động vi sai.
- Phát triển thuật toán điều khiển giúp robot di chuyển ổn định và bám line chính xác.
- Thiết kế và chế tạo mô hình robot hoàn chỉnh bao gồm phần cơ khí, điện và điều khiển.
- Xây dựng hệ thống nhận diện vật thể bằng camera kết hợp trí tuệ nhân tạo.
- Tích hợp các khối chức năng thành một hệ thống thống nhất.
- Thực nghiệm và đánh giá hiệu năng hoạt động của robot.

1.4. PHƯƠNG PHÁP THỰC HIỆN

- Phương pháp lý thuyết
 - Nghiên cứu các bài báo, tài liệu nước ngoài.

- Nghiên cứu lý thuyết về cảm biến dò line sử dụng nguyên lý phản xạ ánh sáng hồng ngoại để phân biệt giữa đường line và nền.
 - Thu nhập đa dạng hóa hình ảnh vật thể, tối ưu hóa dữ liệu qua tiền xử lý dữ liệu và nghiên cứu mạng Yolo nhận diện vật thể.
 - Ứng dụng mạng nơ-ron tích chập (CNN) trong thị giác máy tính để tối ưu hóa khả năng nhận diện vật thể.
 - Nghiên cứu các tài liệu liên quan tới lập trình nhúng, Python và OpenCV.
 - Nghiên cứu giao tiếp hệ thống thiết lập kết nối giữa vi điều khiển Arduino Nano và Raspberry Pi 4.
- Phương pháp thực nghiệm
- Thực hiện hoàn thiện từng giai đoạn của hệ thống.
 - Tính toán, thiết kế và thử nghiệm hệ thống.
 - Thử nghiệm khả năng bám line và gấp đặt, vận chuyển vật thể.
 - Tiến hành chế tạo, lắp ráp thành sản phẩm hoàn chỉnh.
 - Sau khi chế tạo mô hình thì theo dõi, đánh giá và nhận xét quá trình thử nghiệm để cải thiện mô hình hoạt động ổn định nhất.

1.5. DỰ KIẾN KẾT QUẢ ĐẠT ĐƯỢC

Sau khi hoàn thành, đề tài dự kiến đạt được một hệ thống robot di động tự hành có khả năng hoạt động ổn định theo một quy trình khép kín.

- Robot có thể di chuyển theo đường line đã thiết kế với độ chính xác tương đối cao, đảm bảo bám line ổn định trong các đoạn thẳng và đoạn cong cơ bản.
- Robot được tích hợp camera và mô hình trí tuệ nhân tạo, cho phép nhận diện và phân loại vật thể dựa trên đặc trưng màu sắc hoặc hình ảnh. Dựa

trên kết quả nhận diện, robot có thể xác định vị trí vật thể, thực hiện thao tác gấp bằng cơ cấu chấp hành và vận chuyển đến vị trí đích tương ứng.

- Hệ thống điều khiển được xây dựng đảm bảo khả năng phối hợp giữa các khối chức năng như di chuyển, nhận diện và thao tác, giúp robot vận hành liên tục và tự động.

Kết quả đạt được là một mô hình robot hoàn chỉnh, có tính ổn định và có thể mở rộng cho các ứng dụng thực tế trong lĩnh vực tự động hóa.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. TỔNG QUAN VỀ ROBOT DI ĐỘNG TỰ HÀNH

Robot di động là một trong những hướng phát triển quan trọng trong lĩnh vực Robotics, cho phép hệ thống có khả năng di chuyển linh hoạt trong môi trường thay vì cố định tại một vị trí như các robot công nghiệp truyền thống. Nhờ khả năng này, robot di động được ứng dụng rộng rãi trong nhiều lĩnh vực như vận chuyển hàng hóa, sản xuất thông minh, dịch vụ và nghiên cứu.

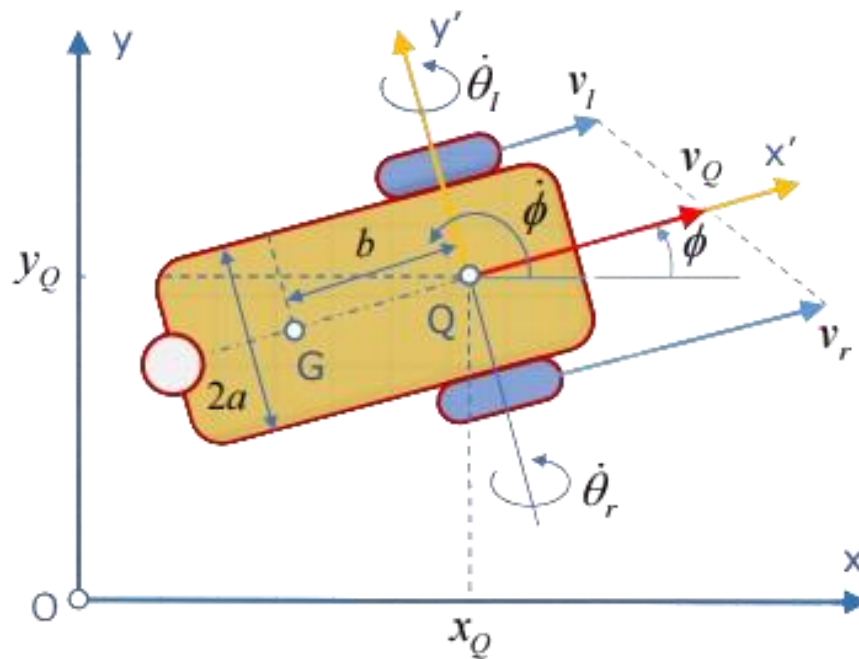
Về cấu trúc, robot di động thường bao gồm các thành phần chính như: bộ điều khiển trung tâm, hệ thống truyền động, cảm biến và nguồn năng lượng. Trong đó, bộ điều khiển đóng vai trò xử lý dữ liệu và đưa ra quyết định, hệ thống truyền động giúp robot di chuyển, còn cảm biến hỗ trợ robot nhận biết môi trường xung quanh.

Một trong những mô hình phổ biến của robot di động là robot sử dụng cơ cấu truyền động vi sai. Động lực học của robot truyền động vi sai dựa trên sự điều phối vận tốc độc lập giữa hai bánh xe chủ động đặt song song, từ đó tạo ra mô-men quay cho phép hệ thống thay đổi quỹ đạo linh hoạt mà không cần cơ cấu lái cơ học phức tạp [3]. Nhờ cấu trúc đơn giản, dễ điều khiển và phù hợp với nhiều bài toán thực tế, mô hình này được sử dụng rộng rãi trong các nghiên cứu và ứng dụng robot tự hành.

Trong đề tài này, robot di động được thiết kế theo hướng bám line, tức là di chuyển theo một quỹ đạo định sẵn được đánh dấu trên mặt sàn. Phương pháp này giúp đơn giản hóa bài toán điều hướng, đồng thời đảm bảo tính ổn định trong quá trình vận hành. Robot sử dụng các cảm biến hồng ngoại để phát hiện vị trí của đường line, từ đó xác định sai số và điều chỉnh hướng di chuyển. Bên cạnh đó, hệ thống còn được tích hợp khả năng xử lý hình ảnh sử dụng trí tuệ nhân tạo nhằm nhận diện và phân loại vật thể trong môi trường. Sự kết hợp giữa điều khiển chuyển động và nhận diện thông minh giúp nâng cao tính linh hoạt và khả năng ứng dụng của robot.

2.2. PHƯƠNG TRÌNH ĐỘNG LỰC HỌC CỦA ROBOT VI SAI

Để đảm bảo điều khiển robot chính xác và ổn định khi bám quỹ đạo, việc xây dựng mô hình robot tin cậy là vô cùng quan trọng. Mô hình này cần mô phỏng chính xác động lực học của robot, bao gồm cả các yếu tố phi tuyến và nhiễu động.



Hình 2.1: Sơ đồ biểu diễn Vector động học của Robot vi sai

Cấu trúc của robot được thiết kế có 3 bánh. Hai bánh ở hai bên thân xe là hai bánh vi sai. Có hai thiết bị truyền động gắn liền với các bánh xe này. Bánh xe ở phía trước là bánh xe đa hướng tồn tại nhằm cân bằng cho robot di động.

Phương trình động lực học

- Mô hình động lực học Newton-Euler:

$$\begin{cases} m\dot{v} = F \\ I\dot{\omega} = N \end{cases} \quad (2.1)$$

- F là tổng các lực tác dụng lên robot, N là tổng mô men đối với trọng tâm tác dụng lên robot, I tensor quán tính của robot. Giả thiết rằng trọng tâm trùng với điểm giữa của hai bánh dẫn động ($b=0$):

$$\begin{cases} F = F_r + F_l \\ \tau_r = rF_r \\ c = rF_l \\ N = (F_r - F_l)2a = \frac{2a}{r}(\tau_r - \tau_l) \end{cases} \quad (2.2)$$

- Mô hình động lực học của robot:

$$\begin{cases} \dot{v} = \frac{1}{mr}(\tau_r + \tau_l) \\ \dot{w} = \frac{2a}{Ir}(\tau_r - \tau_l) \end{cases} \quad (2.3)$$

- Giả thiết rằng trọng tâm trùng với điểm giữa của hai bánh dẫn động (b=0), ma trận ràng buộc nonholonomic:

$$M(q) = [-\sin\phi \quad \cos\phi \quad 0] \quad (2.4)$$

- Giả thiết robot di chuyển trên địa hình phẳng nằm ngang

$$(C(q, \dot{q}) = 0, g(q) = 0) \quad (2.5)$$

- Mô hình động lực học Lagrange của robot:

$$D(q)\ddot{q} + M^T(q)\lambda = E\tau \quad (2.6)$$

- Trong đó:

$$q = \begin{bmatrix} x_Q \\ y_Q \\ \phi \end{bmatrix} \quad (2.7)$$

$$\tau = \begin{bmatrix} \tau_r \\ \tau_l \end{bmatrix} \quad (2.8)$$

$$D(q) = \begin{bmatrix} m & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & I \end{bmatrix} \quad (2.9)$$

$$E = \frac{1}{r} \begin{bmatrix} \cos\phi & \cos\phi \\ \sin\phi & \sin\phi \\ 2a & -2a \end{bmatrix} \quad (2.10)$$

- Trong đó:

$$B^T(q)M^T(q) = 0 \Rightarrow B(q) = \begin{bmatrix} \cos\phi & 0 \\ \sin\phi & 0 \\ 0 & 1 \end{bmatrix} \quad (2.11)$$

$$\bar{D} = B^TDB = \begin{bmatrix} m & 0 \\ 0 & I \end{bmatrix} \quad (2.12)$$

$$\bar{E} = \frac{1}{r} \begin{bmatrix} 1 & 1 \\ 2a & -2a \end{bmatrix} \quad (2.13)$$

- Phương trình động lực học robot

$$\begin{bmatrix} m & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v}_1 \\ \dot{v}_2 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} 1 & 1 \\ 2a & -2a \end{bmatrix} \begin{bmatrix} \tau_r \\ \tau_l \end{bmatrix} \Rightarrow \sim \begin{cases} \dot{v} = \frac{1}{mr} (\tau_r + \tau_l) \\ \dot{w} = \frac{2a}{Ir} (\tau_r - \tau_l) \end{cases} \quad (2.14)$$

2.3. PHƯƠNG PHÁP DÒ LINE CHO ROBOT

2.3.1. Nguyên lý hoạt động của phương pháp dò line

Phương pháp dò line được sử dụng nhằm giúp robot di chuyển theo một quỹ đạo xác định sẵn trên mặt phẳng, thông qua việc nhận biết sự khác biệt giữa đường line và nền. Trong đề tài này, robot sử dụng cảm biến dò line (cảm biến hồng ngoại) để phát hiện sự thay đổi về độ phản xạ ánh sáng giữa bề mặt nền và đường line. Khi ánh sáng hồng ngoại được phát ra và phản xạ lại từ bề mặt, cảm biến sẽ trả về tín hiệu khác nhau tùy thuộc vào màu sắc và độ phản xạ của bề mặt đó.

Nguyên lý bám quỹ đạo quang học hoạt động dựa trên sự chênh lệch hệ số hấp thụ và phản xạ bức xạ hồng ngoại giữa bề mặt tham chiếu (vạch đen) và nền (mặt trắng), qua đó chuyển đổi quang thông thành tín hiệu logic rời rạc để ước lượng sai số vị trí [4]. Dựa vào sự khác biệt này, hệ thống có thể xác định được vị trí tương đối của robot so với đường line, từ đó đưa ra các điều chỉnh phù hợp để duy trì chuyển động theo đúng quỹ đạo.

2.3.2. Dò line sử dụng bộ điều khiển PID

Trong các hệ thống robot di động bám line, việc đảm bảo robot di chuyển ổn định, chính xác theo quỹ đạo là một yêu cầu quan trọng. Để khắc phục sự mất ổn định của các phương pháp điều khiển logic rời rạc, bộ điều khiển tỷ lệ – tích phân – vi phân (PID) được áp dụng nhằm tối ưu hóa phản ứng động lực học, giảm thiểu sai số tĩnh và triệt tiêu hiện tượng dao động (oscillation) khi robot di chuyển tốc độ cao [5]. Robot sử dụng dây cảm biến hồng ngoại để phát hiện vị trí tương đối của đường line so với tâm robot. Từ đó, hệ thống xác định sai số $e(t)$. Bộ điều khiển PID sẽ tính toán tín hiệu điều khiển dựa trên sai số này nhằm điều chỉnh tốc độ hai bánh xe, giúp robot quay về vị trí cân bằng.

Mô hình bộ điều khiển PID

- Bộ điều khiển PID được biểu diễn theo công thức:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (2.15)$$

- Trong đó:

- $e(t)$: sai số tại thời điểm t
- $u(t)$: tín hiệu điều khiển đầu ra
- K_p : hệ số tỉ lệ (Proportional)
- K_i : hệ số tích phân (Integral)
- K_d : hệ số vi phân (Derivative)

- Ý nghĩa các thành phần:

- Thành phần P: phản ứng nhanh với sai số hiện tại, giúp robot quay lại line.
- Thành phần I: tích lũy sai số theo thời gian, giúp loại bỏ sai số tĩnh.

- Thành phần D: dự đoán xu hướng sai số, giúp giảm dao động và overshoot.

Điều khiển động cơ

Tín hiệu đầu ra từ bộ điều khiển PID được sử dụng để điều chỉnh tốc độ hai bánh xe theo nguyên lý:

- Khi robot lệch trái $\rightarrow$ tăng tốc bánh trái, giảm bánh phải
- Khi robot lệch phải $\rightarrow$ ngược lại
- Công thức điều khiển:

$$\begin{cases} V_{left} = V_{base} - u(t) \\ V_{right} = V_{base} + u(t) \end{cases} \quad (2.16)$$

- Trong đó:
 - V_{base} : tốc độ cơ bản
 - $u(t)$: tín hiệu PID

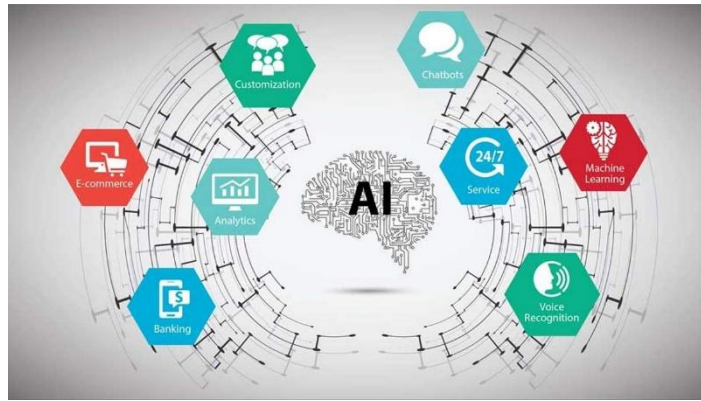
2.4. TỔNG QUAN VỀ TRÍ TUỆ NHÂN TẠO VÀ THỊ GIÁC MÁY TÍNH

2.4.1. Khái niệm

Trí tuệ nhân tạo: Là lĩnh vực khoa học máy tính mô phỏng hành vi và nhận thức của con người, cho phép hệ thống tự học từ dữ liệu, thích nghi với môi trường và đưa ra quyết định độc lập.

Học máy: Là phân nhánh của AI, giúp máy tính tự cải thiện hiệu suất thông qua dữ liệu mà không cần lập trình các quy tắc cố định.

Thị giác máy tính: Là lĩnh vực chuyên nghiên cứu cách máy tính "nhìn", xử lý và phân tích thông tin trực quan từ hình ảnh và video.

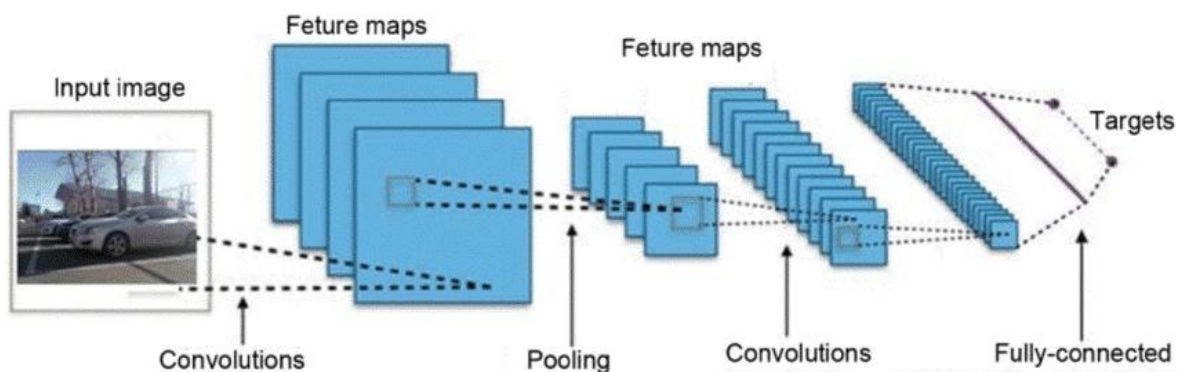


Hình 2.2: Trí tuệ nhân tạo

2.4.2. Học sâu và Mô hình mạng tích chập (CNN)

Học sâu (Deep Learning), một phân nhánh tiên tiến của học máy, thể hiện sự vượt trội so với các kỹ thuật học máy truyền thống (shallow learning) nhờ khả năng tự động trích xuất các đặc trưng phân cấp phức tạp từ dữ liệu phi cấu trúc thông qua kiến trúc mạng nơ-ron nhiều lớp [6]. Học sâu cho phép máy tính giải quyết rất nhiều vấn đề phức tạp bằng cách “học” từ một lượng lớn dữ liệu.

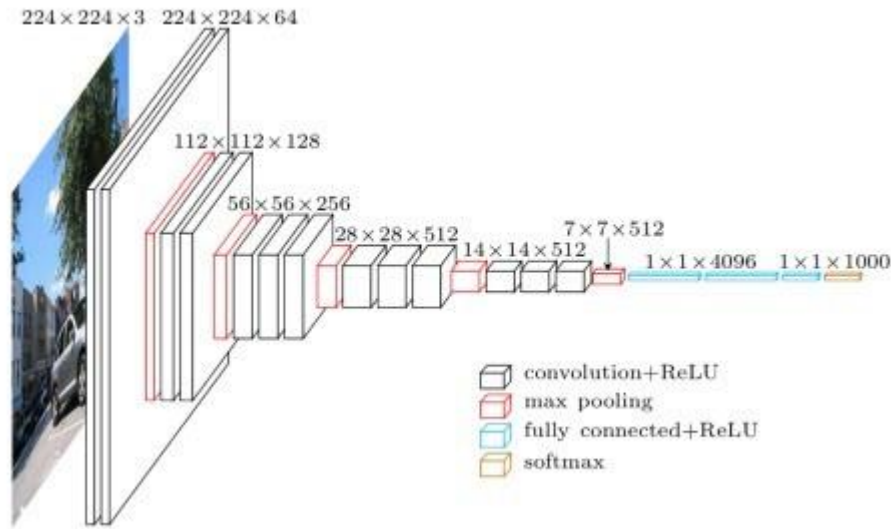
Mạng nơ-ron tích chập (CNN) hiện là kiến trúc cốt lõi trong các bài toán thị giác máy tính, sử dụng các bộ lọc tích chập (convolutional filters) nhằm bảo toàn tính bất biến không gian của hình ảnh, mang lại độ chính xác vượt trội trong các tác vụ phát hiện và phân loại đối tượng [7]. CNN được dùng trong trong nhiều bài toán như nhận dạng ảnh, phân tích video.



Hình 2.3: Mạng nơ-ron học sâu phân tích xe ô tô

Trong mô hình CNN thường có các dạng lớp cơ bản: lớp tích chập, lớp phi tuyến tính, lớp gộp, lớp kết nối đầy đủ. Các layer liên kết được với nhau

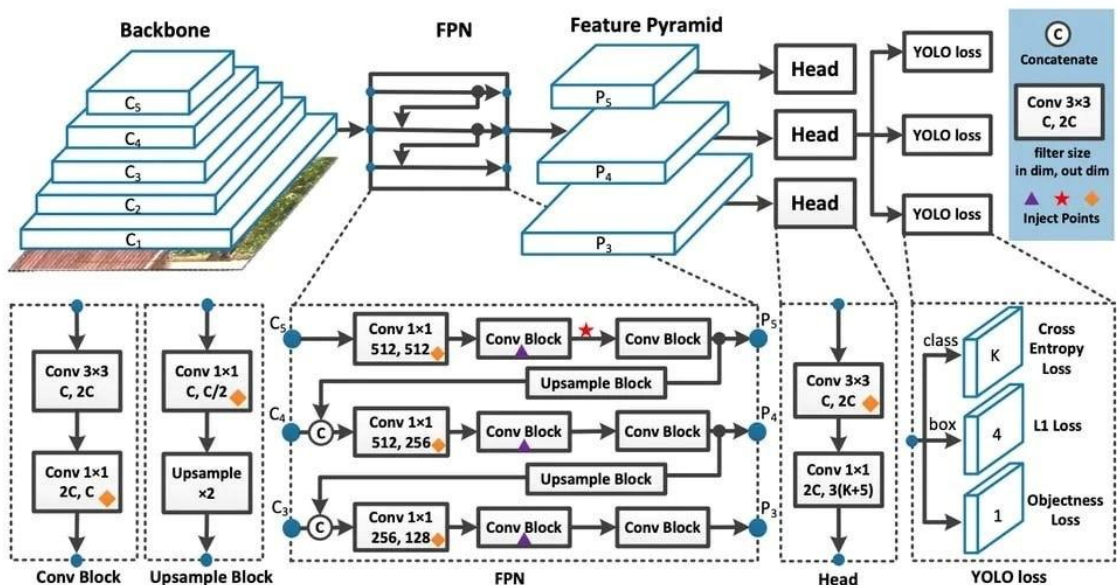
thông qua cơ chế convolution. Layer tiếp theo là kết quả convolution từ layer trước đó.



Hình 2.4: Mô hình CNN

2.4.3. Mô hình YOLOv8 và cấu trúc Backbone, Neck, Head

Kiến trúc mạng YOLOv8 được thiết kế tối ưu với ba khối xử lý lõi: mạng xương sống (Backbone) để trích xuất đặc trưng, phần cổ (Neck) hợp nhất thông tin đa kích thước, và phần đầu (Head) thực hiện dự đoán, qua đó cải thiện đáng kể độ nhạy (recall) đối với các vật thể có kích thước biến thiên [8].



Hình 2.5: Kiến trúc của YOLOv8

- **Backbone**

Backbone là thành phần mạng nơ-ron chịu trách nhiệm trích xuất đặc trưng từ ảnh đầu vào.

Giai đoạn trích xuất đặc trưng trong YOLOv8 được củng cố bởi kiến trúc mạng phân chia giai đoạn chéo (Cross Stage Partial Network - CSPNet), giúp giảm bớt rườm rà trong tính toán và tối ưu hóa luồng suy luận gradient, đảm bảo hiệu năng cao trên các thiết bị tài nguyên thấp [9]. CSPNet chia mạng nơ-ron thành hai phần chính, một phần giữ nguyên dòng thông tin, và một phần thực hiện các phép biến đổi sâu hơn để giảm thiểu độ trùng lặp thông tin. Điều này giúp giảm thiểu lượng tính toán, đồng thời duy trì độ chính xác cao.

Kiến trúc của CSPNet có thể được biểu diễn như sau:

$$F_{out} = CSP(F_{in}) = F_{in}^1 + G(F_{in}^2) \quad (2.17)$$

Trong đó:

- F_{in} là đầu vào của một khối CSPNet
- F_{in}^1 là dòng thông tin không thay đổi
- $G(F_{in}^2)$ là phần áp dụng các phép biến đổi lên dòng thông tin thứ hai
- F_{out} là đầu ra sau khi kết hợp hai dòng

- **Neck**

Neck đóng vai trò cầu nối tổng hợp đặc trưng ngữ nghĩa bằng việc kết hợp Mạng kim tự tháp đặc trưng (Feature Pyramid Network - FPN) kết hợp cùng mạng tổng hợp đường dẫn (PANet) cho phép hệ thống tổng hợp hiệu quả các đặc trưng ngữ nghĩa từ nhiều cấp độ phân giải, giải quyết triệt để bài toán biến thiên tỷ lệ (scale variation) của vật thể mục tiêu [10].

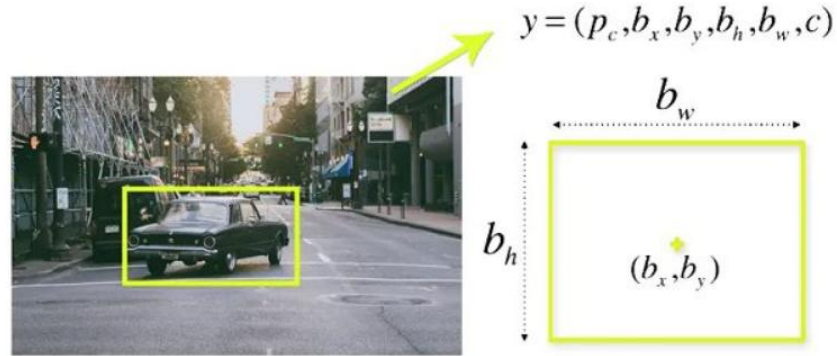
- PANet tối ưu hoá việc tổng hợp đặc trưng từ nhiều tầng khác nhau, tăng cường khả năng phát hiện vật thể nhỏ và mờ nhạt trong ảnh.

- FPN và PANet giúp YOLO v8 xử lý được nhiều kích thước và kiểu vật thể khác nhau, từ đó nâng cao độ chính xác trong các bài toán nhận diện phức tạp.

- **Head**

Head: là bộ phận ra quyết định của kiến trúc mạng YOLOv8, nhận các bản đồ đặc trưng đã được xử lý và tổng hợp từ Neck sau đó thực hiện dự đoán cuối cùng.

Một cải tiến đột phá ở khối dự đoán của YOLOv8 là việc chuyển đổi sang cơ chế phát hiện phi hộp neo (Anchor-free), giúp loại bỏ sự phụ thuộc vào các tham số neo định sẵn mang tính chấp vá, đồng thời tăng tốc độ suy luận bằng cách quy hồi trực tiếp tọa độ của đối tượng [11]. Điều này có nghĩa là thay vì cần xác định các khung tham chiếu trước, YOLO v8 trực tiếp dự đoán vị trí và kích thước của các bounding box dựa trên các đặc trưng từ Neck.



Hình 2.6: Mô tả thực tế của bounding box

YOLO v8 sử dụng một hàm hồi quy để dự đoán các bounding box từ các đặc trưng đã tổng hợp. Giả sử x_c , y_c là tọa độ trung tâm của bounding box với w, h là chiều rộng và chiều cao.

$$x_c = \sigma(t_x) + c_x \quad (2.18)$$

$$y_c = \sigma(t_y) + c_y \quad (2.19)$$

$$w = p_w e^{t_w} \quad (2.20)$$

$$h = p_h e^{t_h} \quad (2.21)$$

Trong đó:

- σ là hàm sigmoid để đảm bảo giá trị trong khoảng $[0, 1]$.
- t_x, t_y, t_w, t_h là các giá trị mà mạng dự đoán.
- c_x, c_y là các tọa độ của ô lưới hiện tại.
- p_w, p_h là các giá trị tham chiếu về kích thước.

- **Các thông số đánh giá hiệu suất của YOLOv8**

C- mAP50: viết tắt của mean Average Precision với IOU threshold 0.5. Đây là trung bình của AP được tính cho tất cả các lớp.

Precision: đo lường độ chính xác dự đoán của bạn. Nghĩa là phần trăm dự đoán của bạn là chính xác. Precision càng cao cho thấy phần lớn các dự đoán của mô hình là chính xác. Precision được tính bằng công thức:

$$Precision = \frac{TP}{TP + FP} \quad (2.22)$$

Trong đó TP là số lượng True Positive (dự đoán đúng) và FP là số lượng False Positive (dự đoán sai).

Recall: đo lường khả năng tìm thấy tất cả các kết quả tích cực. Recall cao cho thấy mô hình tìm thấy được phần lớn các kết quả tích cực. Recall được tính bằng công thức:

$$Recall = \frac{TP}{TP + FN} \quad (2.23)$$

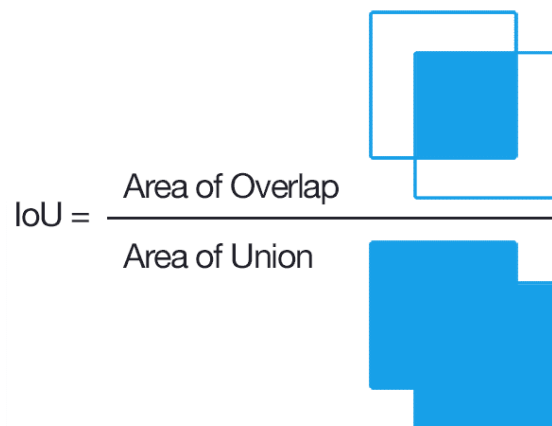
Trong đó TP là số lượng True Positive (dự đoán đúng) và FN là số lượng False Negative (không dự đoán được).

F1 score: là trung bình điều hòa của precision và Recall. F1 score cao cho thấy cả Precision và Recall đều cao.

Confidence: Thuật toán chia nhỏ ảnh input thành lưới gồm $S \times S$ cell. Tại mỗi cell sẽ có trách nhiệm xác định B bounding box có kích thước và tỉ lệ khác nhau để bắt lấy các đối tượng cần xác định trong hình. Sau khi thuật toán được thực thi, mỗi bounding box sẽ được gán những confident rate. Chỉ số này phản ánh cho biết box có đang chứa một đối tượng nào hay không, với xác suất cụ thể. Chỉ số này được tính theo công thức:

$$Confidence = Pr(Object) \times IoU_{truth\ pred} \quad (2.24)$$

Trong đó $Pr(Object)$ là xác suất có một đối tượng được chứa bên trong bounding box đó. $IoU_{truth\ pred}$ (IoU - Intersection over union) là tỉ lệ trùng khớp của bounding box được dự đoán với groundtruth của một đối tượng. Overlap là vùng mà bounding box và groundtruth của một đối tượng trùng lên nhau còn Union là tổng diện tích của cả 2 vùng. Tỷ lệ IoU càng tiến sát 1 có nghĩa là 2 vùng càng gần như nằm trùng với nhau.



Hình 2.7: Tỷ lệ IoU

2.5. PHƯƠNG PHÁP NHẬN DIỆN VẬT THỂ BẰNG CAMERA

- Thu thập bộ dữ liệu

Thu thập từ 200 đến 300 hình ảnh cho mỗi lớp đối tượng dưới nhiều điều kiện biến động về cường độ ánh sáng, góc chụp, khoảng cách và phong nền. Dữ liệu được phân bổ theo tỷ lệ 4-4-2: 40% ảnh đơn lẻ (học đặc trưng cốt lõi), 40% ảnh đa vật thể (tối ưu phân loại đồng thời) và 20% ảnh che khuất một phần (tăng khả năng chống nhiễu).

- **Vai trò của tiền xử lý và tăng cường dữ liệu**

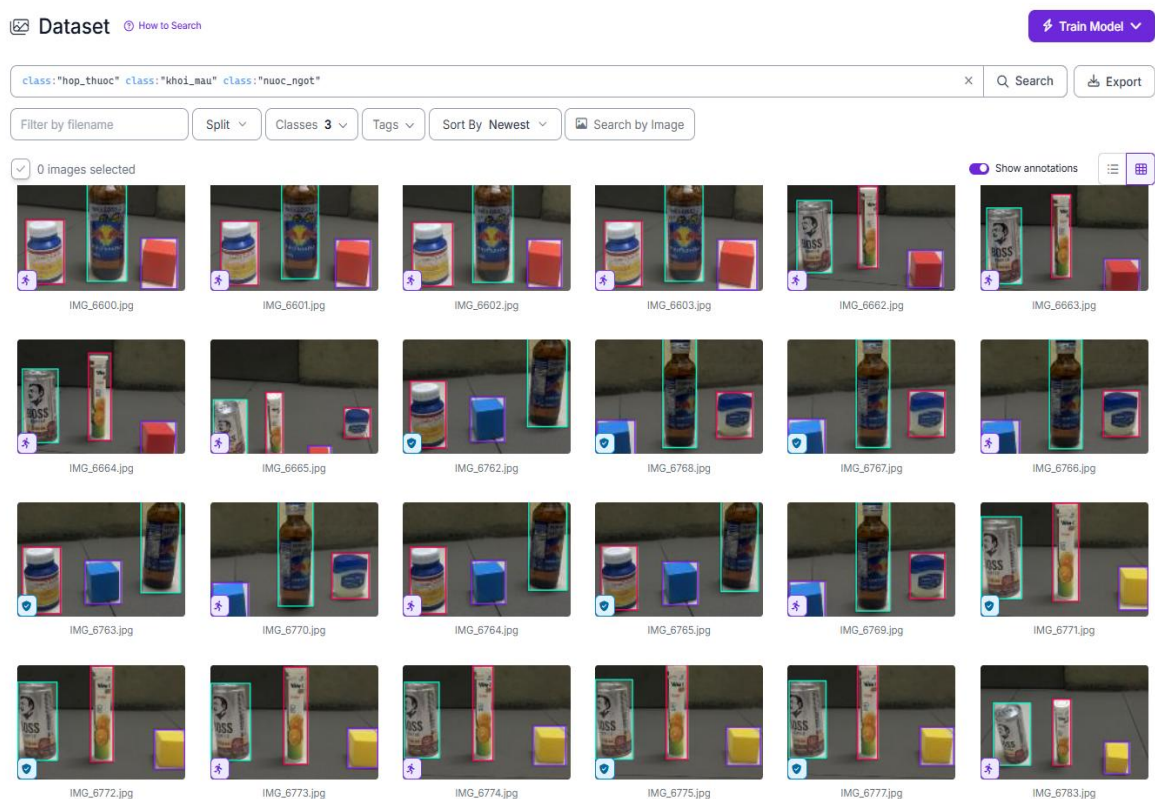
Áp dụng kỹ thuật tăng cường dữ liệu (Data Augmentation) thông qua các phép biến đổi hình học và thêm nhiễu quang học có kiểm soát.

Mục đích: Mở rộng tính đa dạng của không gian mẫu, khắc phục tình trạng khan hiếm dữ liệu thực tế và ngăn chặn hiện tượng quá khớp (overfitting) khiến độ chính xác bị suy giảm khi vận hành thực tế.

- **Gán nhãn và xử lý trên nền tảng Roboflow**

Hình ảnh được tải lên Roboflow để khoanh vùng và gán nhãn khung bao (Bounding Box) thủ công, thiết lập "Ground Truth" cho mô hình học tập.

Dữ liệu được chuẩn hóa về kích thước 640x640 (phù hợp với kiến trúc đầu vào mạng YOLOv8) và được tự động phân chia theo tỷ lệ chuẩn: 70% để huấn luyện, 20% để kiểm thử tinh chỉnh và 10% để đánh giá.



Hình 2.8: Tập dữ liệu thực tế đã được gán nhãn cho các lớp vật thể mục tiêu trên nền tảng Roboflow

- ***Huấn luyện mô hình YOLOv8 trên Google Colab***

Để thực hiện huấn luyện mô hình YOLOv8, nhóm lựa chọn Google Colab làm nền tảng chính. Google Colab được phát triển bởi nhóm Google Research nhằm hỗ trợ quá trình nghiên cứu và phát triển trong lĩnh vực học máy tính và khoa học dữ liệu.

Lý do lựa chọn:

- Miễn phí GPU
- Tích hợp sẵn thư viện AI
- Dễ chia sẻ và tái sử dụng
- Tự động kết nối với Drive



Hình 2.9: Logo của Google Colab

2.6. GIỚI THIỆU CÁC PHẦN CỨNG SỬ DỤNG TRONG HỆ THỐNG

Hệ thống phần cứng của robot được thiết kế ưu tiên tính ổn định, độ chính xác và khả năng xử lý song song. Việc sử dụng động cơ bước kết hợp module A4988 đem lại lợi thế vượt trội về độ chính xác khi điều khiển vị trí và tốc độ, đáp ứng rất tốt yêu cầu bám line khắt khe của bộ điều khiển PID.

Bên cạnh đó, để đảm bảo hệ thống vận hành trơn tru khi máy tính nhúng (Raspberry Pi 4) phải tải các mô hình AI nặng như YOLOv8, giải pháp quản lý năng lượng phân tách đã được áp dụng. Cụ thể, khối xử lý ảnh được cấp nguồn riêng biệt qua nguồn 5V-2A, trong khi hệ thống truyền động (động cơ bước, servo, cảm biến) và vi điều khiển trung tâm (Arduino Nano) sử dụng một khối 3 cell pin 18650 độc lập. Thiết kế này triệt tiêu hoàn toàn hiện tượng sụt áp và nhiễu điện từ khi các cơ cấu chấp hành hoạt động tải nặng.

Các thông số kỹ thuật cơ bản của các linh kiện trong hệ thống được tổng hợp trong Bảng 2.1 dưới đây:

Bảng 2.1: Tổng hợp thông số kỹ thuật các linh kiện phần cứng

Linh kiện	Chức năng chính trong hệ thống	Thông số kỹ thuật cơ bản
Arduino Nano	Vi điều khiển trung tâm, xử lý tín hiệu cảm biến và xuất xung điều khiển động cơ.	Chip ATmega328P, Xung nhịp 16 MHz, Nguồn 5VDC
Raspberry Pi4	Máy tính nhúng, xử lý AI, phân loại hình ảnh vật thể (YOLOv8).	Broadcom BCM2711 Cortex-A72, 4GB RAM
Động cơ Bước 42	Truyền động bánh xe di chuyển.	1.8°/bước, 42x42x40mm
Driver A4988	Điều khiển động cơ bước, hỗ trợ vi bước để chuyển động mượt mà.	Dòng định 2A, ngõ ra tối đa 35V
Camera HD Full	Thu nhận luồng hình ảnh môi trường truyền về Raspberry Pi.	1080p/2K/4K, Giao tiếp USB, AI Auto Focus
TCRT5000	Cảm biến dò đường line (gồm 5 mắt hồng ngoại).	3.3~5VDC, Khoảng cách quét 10~15mm
Cảm biến tiệm cận	Phát hiện vật thể tiếp cận để kích hoạt dừng và gấp.	10~30VDC, Khoảng cách 5mm, Ngõ ra NPN NO
Servo MG995/996	Chấp hành đóng/mở cơ cấu tay gấp khớp động.	Góc quay 0-180°, Lực kéo 13-15kg.cm

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. PHÂN TÍCH YÊU CẦU HỆ THỐNG

Trước khi bắt đầu phát triển sản phẩm, cần phải làm rõ được nhiệm vụ thiết kế một cách chi tiết. Việc phân tích nhiệm vụ thiết kế trải qua các bước cơ bản sau:

Bảng 3.1: Danh sách yêu cầu robot

Loại yêu cầu	DANH SÁCH YÊU CẦU ROBOT
D W	Yêu cầu
D D D D	Kích thước: - Chiều dài: 300 - 350 mm. - Chiều rộng: 200 - 250 mm. - Chiều cao: 100 - 140 mm. - Khối lượng: ≤ 5 kg.
D D	Động học: - Tốc độ bánh xe: ≈ 13.9 cm/s - Gia tốc: $\approx 1,2$ m/s²
D D D W	Năng lượng: - Hoạt động liên tục ≥ 30 phút. - Thời gian sạc đầy ≤ 90 phút. - Công suất trung bình $\leq 9,6$ W. - Có các mạch hạ áp tách biệt nguồn logic và nguồn công suất để chống nhiễu
D	Vật liệu: - Đảm bảo độ cứng vững và trọng lượng nhẹ

3.2. THIẾT KẾ TỔNG THỂ HỆ THỐNG ROBOT

Dựa trên các phân tích yêu cầu từ hệ thống, robot được thiết kế theo kiến trúc phân tán và mô-đun hóa, nhằm tối ưu hóa khả năng vận hành, dễ dàng gỡ lỗi và nâng cấp trong tương lai. Tổng thể robot là sự kết hợp chặt chẽ giữa ba nền tảng cốt lõi: cơ cấu cơ khí, mạch điện tử và phần mềm trí tuệ nhân tạo.

Về mặt vận hành, robot hoạt động theo một chu trình điều khiển khép kín: thu nhận tín hiệu môi trường, phân tích, ra quyết định, thực thi cơ cấu chấp hành. Hệ thống phân chia rõ ràng các cấp độ xử lý: các tác vụ xử lý ảnh nặng, đòi hỏi tài nguyên lớn để nhận diện và phân loại chính xác các mục tiêu như 'hop_thuoc', 'khoi_mau', hay 'nuoc_ngot' được giao hoàn toàn cho máy tính nhúng. Trong khi đó, các tác vụ đòi hỏi tính thời gian thực cao như thuật toán bám line tốc độ cao và điều khiển động học được xử lý riêng biệt bởi vi điều khiển trung tâm.

Về mặt bố trí vật lý, toàn bộ hệ thống được đặt trên khung gầm dẫn động vi sai ổn định với trọng tâm được tối ưu. Đặc biệt, chiến lược thiết kế tổng thể nhấn mạnh việc tách biệt hoàn toàn giữa nguồn logic và nguồn công suất. Sự phân tách này giúp ngăn chặn hiện tượng sụt áp hoặc nhiễu điện từ hệ thống truyền động lan truyền ngược lại làm gián đoạn "não bộ" xử lý của robot.

3.3. THIẾT KẾ HỆ THỐNG CƠ KHÍ CỦA ROBOT

3.3.1. Khung robot

Khung robot được thiết kế thành 2 phần:

Phần đế: Được thiết kế để phù hợp với cơ cấu truyền động 3 bánh vi sai dạng mobile robot. Vật liệu sử dụng là nhôm 6060 với độ dày 3mm được cắt lazer để đảm tính chính xác về dung sai và kích thước thiết kế, mặt khác nhôm giúp phần đế robot được chắc chắn và ổn định hơn trong quá trình hoạt động. Việc lựa chọn hợp kim nhôm thay vì thép không gỉ (inox) hoặc nhựa được quyết định dựa trên các tối ưu về cơ tính và kinh tế. Nhôm sở hữu tỷ lệ độ bền trên trọng lượng cao, dễ gia công và tản nhiệt tốt. Trong khi đó, thép không gỉ

tuy cứng nhưng làm tăng tải trọng hệ thống, còn vật liệu nhựa lại không đáp ứng được yêu cầu về độ cứng vững.

Phần vỏ: Được thiết dạng xe ô tô, ăn khớp với phần đế, bao gồm các hốc nguồn, bánh xe, cảm biến, tay kẹp,... Vật liệu sử dụng là nhựa PetG, được chế tạo bằng máy in 3d. Khác với phần đế phải chịu lực, trọng tải của cả robot thì phần vỏ lại nhẹ nhàng hơn khi chỉ được sử dụng để che kín cũng như bảo vệ các linh kiện bên trong khỏi va chạm từ bên ngoài, mặt khác hình dạng của phần vỏ cũng khá phức tạp và nhiều đường nét, nên việc sử dụng in 3d là phương pháp hợp lý và ít chi phí nhất. Nhựa PetG là một dòng nhựa phổ thông trong công nghệ in 3d, đảm bảo được độ cứng cần thiết mà vẫn đem lại tính thẩm mỹ cao cho sản phẩm.



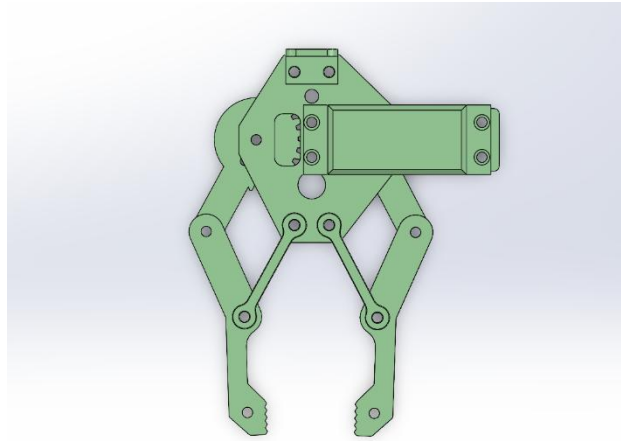
Hình 3.1: Vỏ và đế của robot

3.3.2. Cơ cấu tay gấp

Cơ cấu tay gấp được thiết kế nhằm thực hiện nhiệm vụ gấp và thả vật thể trong quá trình hoạt động của robot. Tay gấp sử dụng động cơ servo MG995 để điều khiển chuyển động đóng/mở, cho phép thao tác nhanh và chính xác.

Thiết kế tay gấp theo dạng khớp đơn giản, đảm bảo đủ lực kẹp để giữ vật thể trong quá trình di chuyển mà không gây rơi hoặc lệch vị trí. Kích thước và vật thể lớn nhất mà cơ cấu tay gấp này có thể gấp được là 50mm, với khối lượng khoảng 500 gram.

Cơ cấu tay gấp được lắp đặt ở phía trước robot, giúp thuận tiện trong việc tiếp cận và thao tác với vật thể. Đồng thời, vị trí này cũng hỗ trợ camera quan sát trực tiếp khu vực gấp, nâng cao hiệu quả phối hợp giữa nhận diện và điều khiển.



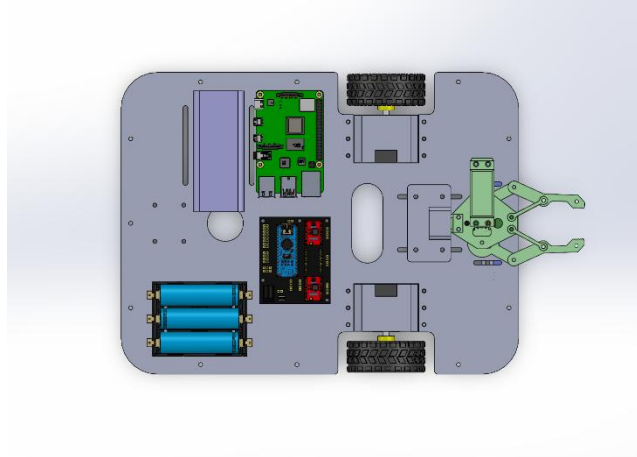
Hình 3.2: Cơ cấu tay gấp

3.3.3. Hệ thống truyền động

Hệ thống truyền động robot sử dụng là 3 bánh vi sai, với 2 bánh chủ động là 2 bánh trước và 1 bánh đa hướng ở phía sau để cân bằng và dễ dàng di chuyển. Đây là một hệ thống truyền động khá cơ bản, phổ biến, và thuận lợi trong quá trình thiết kế.

Bằng các sử dụng 2 động cơ Step 42, kích thước: 42x42x34 (mm) cho 2 bánh trước, và 2 bánh xe đường kính 60mm để tạo bộ dẫn động cầu trước cho robot giúp robot có thể di chuyển linh hoạt. Mặt đế robot được thiết kế để trục bánh xe là một đường thẳng thuộc mặt phẳng đế và mặt ngoài bánh xe đồng phẳng với viền mặt phẳng đế. Bánh dẫn động phía sau là một bánh đa hướng với đường kính bánh xe là 25mm, được đặt sao cho tạo với 2 bánh trước thành một tam giác cân, giúp ổn định di chuyển và cân bằng cho robot, đồng thời cũng linh hoạt trong các tình huống xoay chuyển, rẽ hướng robot.

Cơ cấu tay gấp được lắp đặt ở phía trước robot, giúp thuận tiện trong việc tiếp cận và thao tác với vật thể. Đồng thời, vị trí này cũng hỗ trợ camera quan sát trực tiếp khu vực gấp, nâng cao hiệu quả phối hợp giữa nhận diện và điều khiển.



Hình 3.3: Bố trí các linh kiện phần đế robot (Kích thước 330mm x 250mm)

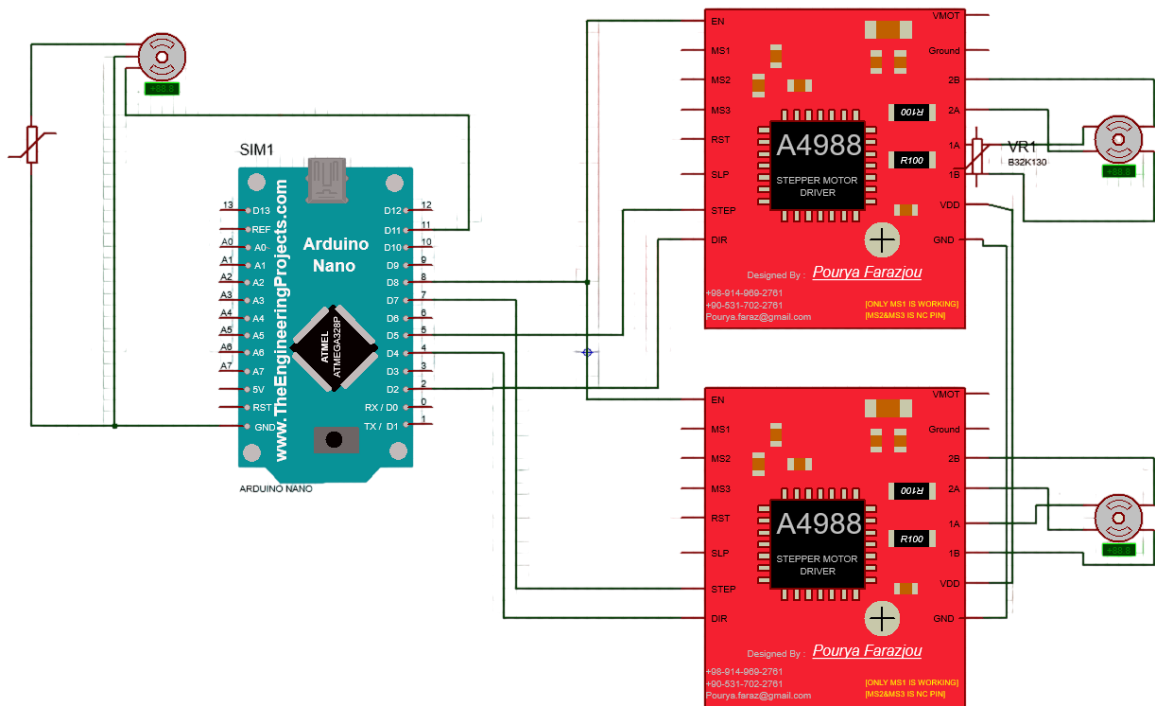
3.4. THIẾT KẾ HỆ THỐNG ĐIỆN VÀ ĐIỀU KHIỂN

3.4.1. Sơ đồ kết nối Arduino

Arduino Nano đóng vai trò là bộ điều khiển trung tâm, kết nối trực tiếp với các cơ cấu chấp hành và cảm biến bám line.

- Điều khiển động cơ: Hai driver A4988 được kết nối với các chân Digital của Arduino để phát xung (STEP) và điều khiển hướng (DIR) cho hai động cơ bước Step 42.
- Điều khiển tay gấp: Chân tín hiệu của Servo MG995 được kết nối với chân PWM trên Arduino để điều khiển chính xác góc quay từ 0° đến 180° .

- Giao tiếp: Sử dụng giao tiếp nối tiếp (Serial) thông qua các chân TX/RX hoặc cáp USB để nhận lệnh từ Raspberry Pi 4.



Hình 3.4: Sơ đồ kết nối arduino

3.4.2. Sơ đồ kết nối Raspberry Pi

Raspberry Pi 4 đóng vai trò là khối xử lý tri thức, đảm nhiệm việc phân tích dữ liệu hình ảnh.

- Kết nối Camera: Camera HD Full được kết nối qua cổng USB của Raspberry Pi để truyền tải luồng video phục vụ nhận diện vật thể.
- Nguồn cấp: Sử dụng nguồn 5VDC - 3A qua cổng USB-C để đảm bảo hiệu năng cho vi xử lý khi chạy các mô hình AI.
- Kết nối tín hiệu: Gửi các mã trạng thái hoặc kết quả nhận diện vật thể sang Arduino Nano để phối hợp thực hiện nhiệm vụ.

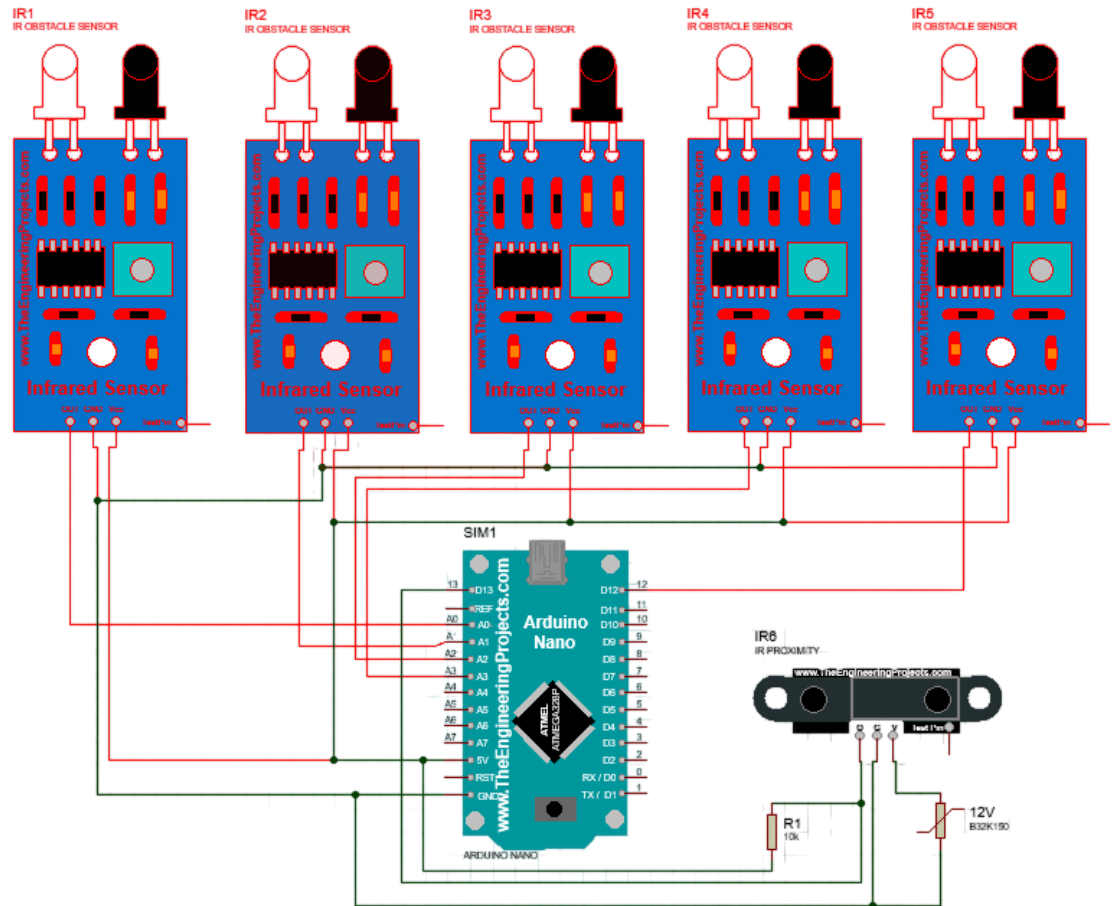


Hình 3.5: Sơ đồ kết nối Raspberry pi

3.4.3. Kết nối các cảm biến

Hệ thống cảm biến cung cấp dữ liệu đầu vào để robot nhận biết môi trường.

- Cảm biến dò line (TCRT5000): 5 cảm biến được bố trí theo hàng ngang, các chân tín hiệu Digital (D0) của cảm biến nối với các chân Digital I/O trên Arduino để xác định vị trí đường line.
- Cảm biến tiệm cận: Được lắp đặt ở vị trí phù hợp để phát hiện vật thể kim loại khi đến gần, tín hiệu đầu ra (NPN) được đưa về chân ngắt của Arduino để xử lý dừng robot kịp thời.
- Nguồn cấp cảm biến: Các cảm biến được cấp nguồn 3.3V hoặc 5V tùy loại từ board mở rộng của Arduino để đảm bảo tính ổn định và chống nhiễu.



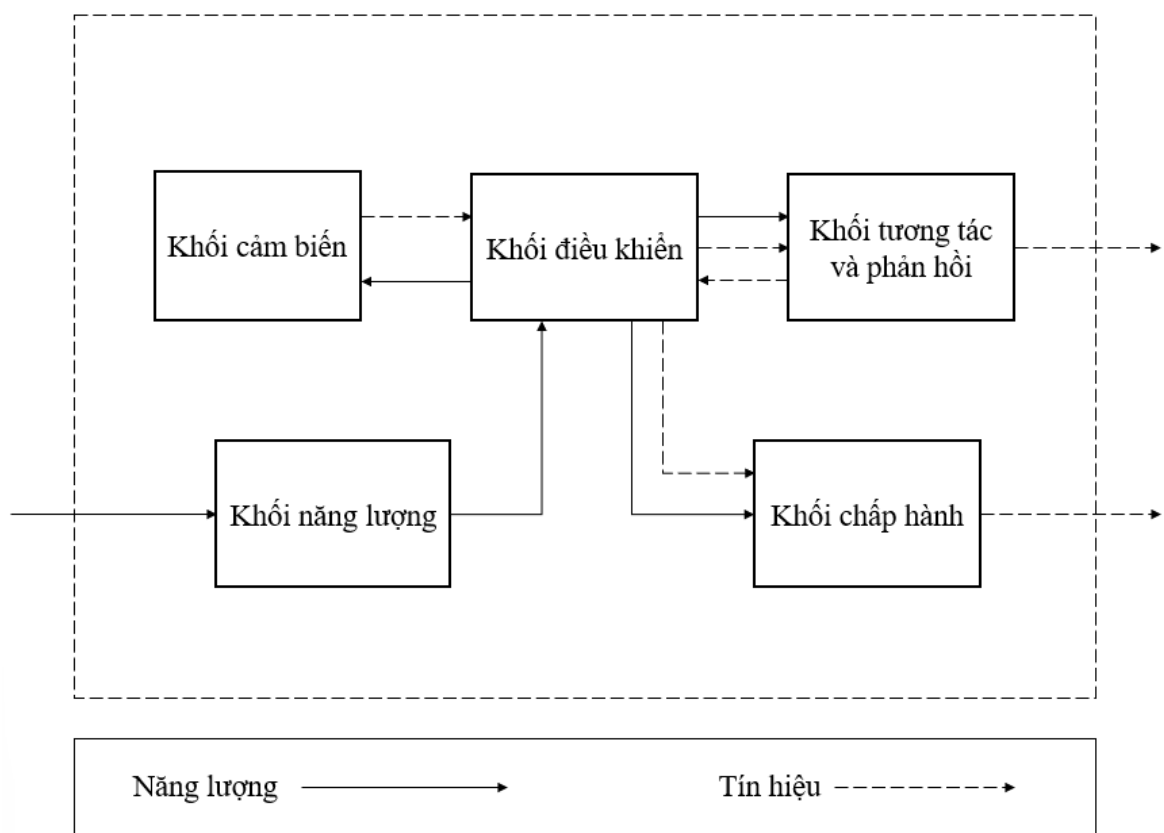
Hình 3.6: Sơ đồ kết nối các cảm biến

3.5. THIẾT KẾ SƠ ĐỒ KHỐI CỦA HỆ THỐNG ROBOT

Thiết kế tổng thể là phần quan trọng nhất trong thiết kế hệ thống, giúp hình dung tổng thể cấu trúc phần cứng, sự tương tác giữa các module và luồng thông tin từ đầu vào đến đầu ra. Hệ thống được chia thành 5 khối chức năng chính, mỗi khối đảm nhiệm một vai trò chuyên biệt nhưng có mối liên hệ chặt chẽ với nhau:

- Khối năng lượng: Sử dụng nguồn tách biệt, gồm nguồn 5V-2A cấp cho Raspberry Pi 4 và cụm pin 18650 cấp cho Arduino, cảm biến, động cơ và servo.
- Khối điều khiển: Raspberry Pi 4 xử lý hình ảnh và nhận diện vật thể; Arduino Nano nhận kết quả, đồng thời xử lý tín hiệu cảm biến để điều khiển chuyển động và cơ cấu gấp.

- Khối cảm biến: Bao gồm camera thu nhận hình ảnh, dây cảm biến TCRT5000 phục vụ bám line và cảm biến tiệm cận hỗ trợ phát hiện vị trí vật thể.
- Khối cơ cấu chấp hành: Gồm động cơ bước điều khiển qua driver A4988 và servo MG995 thực hiện thao tác gấp và di chuyển.
- Khối phản hồi: Thu thập dữ liệu từ cảm biến để điều chỉnh hoạt động của động cơ và servo, giúp hệ thống vận hành ổn định và chính xác.



Hình 3.7: Sơ đồ khối hệ thống

CHƯƠNG 4: THUẬT TOÁN ĐIỀU KHIỂN VÀ XÂY DỰNG CHƯƠNG TRÌNH

4.1. THUẬT TOÁN ĐIỀU KHIỂN ROBOT BẮM LINE

4.1.1. Cơ chế đọc cảm biến và tính toán sai số

Hệ thống sử dụng một dải gồm 5 cảm biến hồng ngoại, được bố trí và đánh dấu từ w_1 đến w_5 theo hướng từ trái sang phải. Để xác định chính xác vị trí tâm của robot so với vạch kẻ, thuật toán áp dụng phương pháp trung bình cộng có trọng số. Cụ thể, mỗi mắt cảm biến được gán một mức trọng số dựa trên khoảng cách vật lý của nó so với trục trung tâm:

- Mắt trái nhất (w_1): -10.
- Mắt trái giữa (w_2): -5.
- Mắt giữa (w_3): 0.
- Mắt phải giữa (w_4): +5.
- Mắt phải nhất (w_5): +10.

Độ lệch vị trí tại thời điểm hiện tại được tính toán thông qua công thức:

$$Error = \frac{W_1(-10) + W_2(-5) + W_3(0) + W_4(5) + W_5(10)}{\sum_{i=1}^5 W_i} \quad (4.1)$$

4.1.2. Hệ thống Máy trạng thái

Dữ liệu trạng thái của mảng cảm biến được liên tục phân tích và gán vào 5 trạng thái vận hành chính, giúp robot đưa ra các quyết định linh hoạt:

- LINE_ON: Trạng thái bám đường thẳng bình thường.
- LINE_CROSS: Nhận diện ngã tư khi có từ 4 mắt cảm biến trở lên cùng phát hiện vạch.

- LINE_SHARP_L: Cửa gắt trái (nhận diện khi ít nhất 2 mắt bên trái phát hiện vạch, đồng thời mắt phải nhất nằm ở vùng trắng).
- LINE_SHARP_R: Cửa gắt phải (nhận diện khi ít nhất 2 mắt bên phải phát hiện vạch, đồng thời mắt trái nhất nằm ở vùng trắng).
- LINE_LOST: Trạng thái mất vạch hoàn toàn (không có cảm biến nào nhận tín hiệu).

Hệ thống tích hợp một biến nhớ (*last_sharp_dir*) nhằm lưu lại hướng của khúc cua gắt gần nhất, làm cơ sở dữ liệu để phục hồi vị trí nếu xảy ra sự cố mất vạch.

4.1.3. Thuật toán điều khiển tỷ lệ - tích phân - vi phân (PID)

Các khâu của bộ điều khiển được triển khai như sau:

- Khâu Tỷ lệ (P): Bằng chính giá trị sai số hiện tại.

$$K_p = \text{current_error} \quad (4.2)$$

- Khâu Tích phân (I): Cộng dồn sai số theo thời gian để khử sai số tĩnh.

$$I = I + \text{current_error} \quad (4.3)$$

Để tránh hiện tượng bão hòa tích phân gây vọt lố hệ thống, giá trị I được giới hạn nghiêm ngặt trong khoảng $[-50, 50]$. Trong giai đoạn hiện tại, hệ số K_I được đặt bằng 0.0 để tối ưu hóa tính ổn định tĩnh.

- Khâu Vi phân (D): Tính toán độ biến thiên của sai số.

$$K_d = \text{current_error} - \text{previous_error} \quad (4.4)$$

Sau khi tính toán tổng tín hiệu điều khiển:

$$PID_value = K_p * P + K_I * I + K_d * D \quad (4.5)$$

Hệ thống tiến hành phân phối tốc độ cho hai động cơ dựa trên một tốc độ nền. Điểm đặc biệt của cơ cấu truyền động hiện tại là cấu hình timer cấp

xung nghịch đảo (số nhỏ tương ứng với tần số xung cao, tốc độ nhanh). Do đó, phương trình cập nhật tốc độ bánh xe được thiết lập:

$$Banh_phai = toc_do_nen + PID_value \quad (4.6)$$

$$Banh_trai = toc_do_nen - PID_value \quad (4.7)$$

Cả hai giá trị tốc độ đều được ràng buộc an toàn trong ngưỡng an toàn của hệ thống cơ khí từ 5 đến 200 để đảm bảo thuật toán PID luôn có tác dụng kiểm soát hợp lệ.

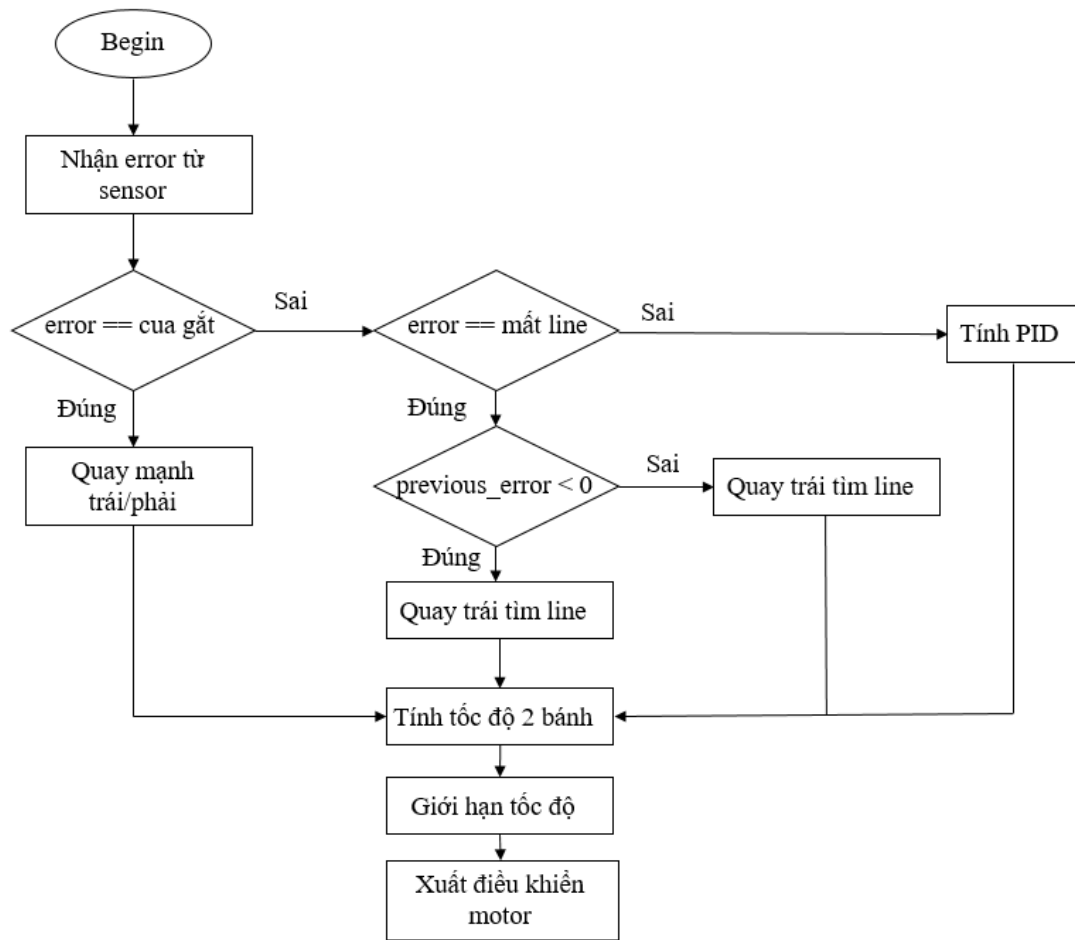
4.1.4. Phương pháp xử lý sai số

Bên cạnh bộ điều khiển PID, hệ thống được trang bị logic xử lý chuyên biệt cho các điều kiện môi trường phức tạp:

Quản lý bộ nhớ sai số và tìm vạch: Biến *previous_error* chỉ được hệ thống lưu lại khi robot đang thực sự nhìn thấy vạch line. Khi mảng cảm biến đi vào vùng mất vạch (trạng thái lỗi *current_error* = 5), bộ điều khiển sẽ phân tích *previous_error*. Nếu sai số trước đó âm (robot đang lệch phải), hệ thống điều khiển động cơ xoay trái để quét tìm lại vạch, và ngược lại.

Vào cua gắt: Khi phát hiện góc vuông (*current_error* = 6 hoặc *current_error* = 7), hệ thống tạm thời thoát khỏi khâu PID. Thay vào đó, robot áp dụng kỹ thuật Differential Steering biên độ lớn (cấp tốc độ trái dấu cho hai bánh) để bẻ lái tức thời, đảm bảo góc quay hẹp nhất có thể mà không bị lộ đà.

Chuyển hướng tại ngã tư: Khi đi vào ngã tư (*current_error* = 8), robot dừng khẩn cấp 0.5 giây để ổn định quán tính trước khi thực thi lộ trình. Dựa trên mã định danh vật phẩm (*khoi_mau*, *nuoc_ngot*, *hop_thuoc*) nhận được từ hệ thống thị giác máy tính và trạng thái hành trình (đang đi giao hay đang quay về), robot sẽ thực hiện chuỗi lệnh điều hướng tương ứng (rẽ trái, đi thẳng hoặc rẽ phải). Tốc độ vào cua ở các điểm nút này được tính toán độc lập nhằm hạn chế trượt bánh.



Hình 4.1: Lưu đồ thuật toán bám line sử dụng PID

4.2. THUẬT TOÁN NHẬN DIỆN VẬT THỂ BẰNG AI

4.2.1. So sánh các mô hình học sâu

Qua quá trình huấn luyện các mô hình học sâu như: YOLOv5, YOLOv8 và YOLOv11, nhóm đã có kết quả huấn luyện của các mô hình đó như sau:

Bảng 4.1: Kết quả huấn luyện của các mô hình YOLOv5, YOLOv8 và YOLOv11

Model	Train	Test	Valid	Epochs
YOLOv5su	1281	122	61	100 epochs
YOLOv8s	1281	122	61	100 epochs
YOLOv11s	1281	122	61	100 epochs

Do đó thông số huấn luyện các mô hình đã bằng nhau. Từ dữ liệu ảnh để huấn luyện ở trên, nhóm đã thu được kết quả sau:

Bảng 4.2: So sánh thời gian huấn luyện và độ chính xác của các mô hình YOLO (Yolov5 , Yolov8 và Yolov11)

Model	Thời gian huấn luyện	Precision (P)	Recall (R)	mAP@0.5
YOLOv5su	53 phút	0.978	0.975	0.982
YOLOv8s	42 phút	0.993	0.995	0.994
YOLOv11s	46 phút	0.996	0.998	0.997

Qua quá trình thử nghiệm, nhóm quyết định lựa chọn mô hình YOLOv8s kết hợp với máy tính nhúng Raspberry Pi 4. Sự kết hợp này không chỉ mang lại độ chính xác cao (99.4%) mà còn đảm bảo tính thời gian thực cho hệ thống. YOLOv8s đủ nhẹ để chạy ổn định trên vi xử lý ARM của Raspberry Pi 4 sau khi được tối ưu hóa, tránh hiện tượng quá nhiệt và sụt giảm FPS trong quá trình vận hành lâu dài của robot.

4.2.2. Huấn luyện mô hình nhận diện đối tượng bằng YOLOv8

Bước 1: Chuẩn bị dữ liệu huấn luyện.

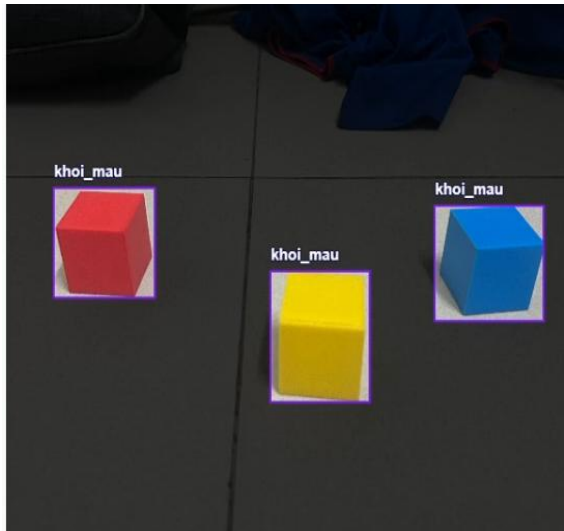
- Thu thập một tập hợp dữ liệu ảnh chứa các đối tượng cần nhận diện (khối màu, nước ngọt, hộp thuốc, viên).
- Tập dữ liệu đủ lớn và đa dạng trong các điều kiện ánh sáng khác nhau.



Hình 4.2: Tập dữ liệu ảnh chứa các đối tượng được nhận diện

Bước 2. Dán nhãn đối tượng

Phân loại và dán nhãn: Xác định và dán nhãn đúng các đối tượng trong ảnh. Nhóm đã sử dụng trang web Roboflow để thực hiện vẽ bounding box và dán nhãn cho đối tượng.



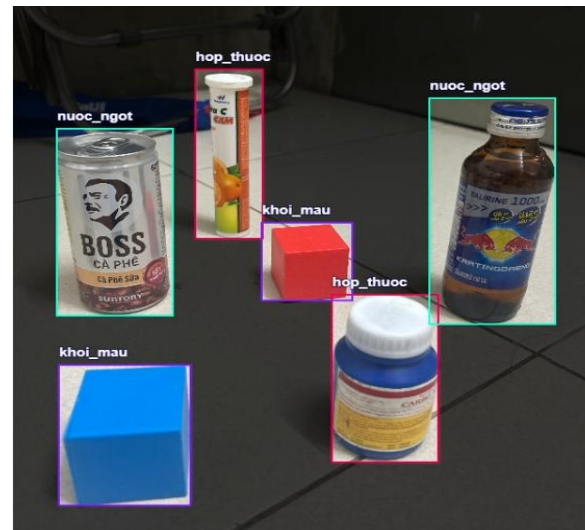
Hình 4.3: Gán nhãn các khối màu



Hình 4.4: Gán nhãn nước ngọt



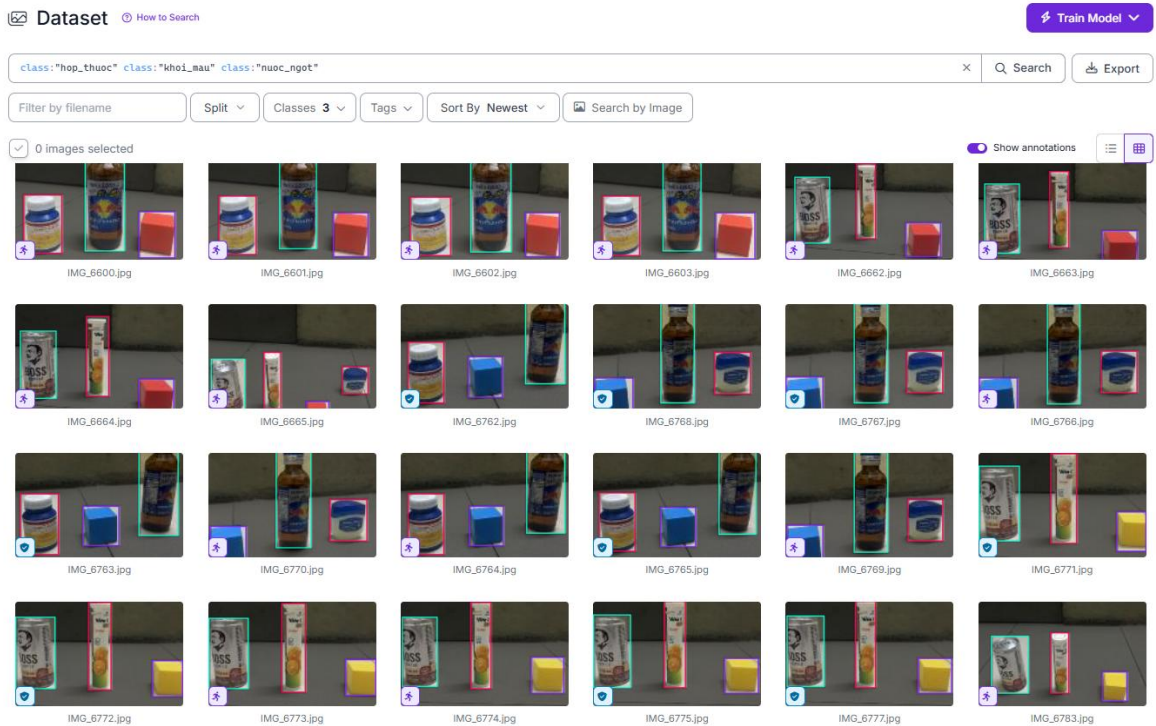
Hình 4.5: Gán nhãn hộp thuốc



Hình 4.6: Gán nhãn đa vật thể

Bước 3: Xuất bộ dữ liệu cho quá trình training model

Khi phiên bản tập dữ liệu đã được tạo ra, chúng ta có một tập dữ liệu được lưu trữ mà chúng ta có thể tải trực tiếp vào sổ tay của mình để dễ dàng huấn luyện.



Hình 4.7: Tập dữ liệu được chia thành các tập huấn luyện, đánh giá, kiểm thử

Bước 4: Tiến hành huấn luyện mô hình YOLOv8

Nhóm thực hiện huấn luyện mô hình YOLO trên Google Colab. Đầu tiên, cần cài đặt thư viện Ultralytics:

1. Cài đặt thư viện

```
[1] 15 giây
!pip install roboflow ultralytics -q

...
175.9/175.9 kB 9.1 MB/s eta 0:00:00
66.8/66.8 kB 5.5 MB/s eta 0:00:00
49.9/49.9 MB 18.7 MB/s eta 0:00:00
1.2/1.2 MB 72.7 MB/s eta 0:00:00
1.5/1.5 MB 91.9 MB/s eta 0:00:00
5.5/5.5 MB 110.7 MB/s eta 0:00:00

[2] 14 giây
from roboflow import Roboflow
from ultralytics import YOLO
import cv2
from google.colab.patches import cv2_imshow

Creating new Ultralytics Settings v0.0.6 file ✓
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
```

Hình 4.8: Cài đặt thư viện Ultralytics

- Tiếp theo, tải dữ liệu đã được phân loại và gán nhãn trước đó:

2. Nhập dữ liệu

```
[1] 11 giây
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key="Q0ZnXuecnDXH7uFz2")
project = rf.workspace("ccs-workspace-vjczv").project("object-classification-n4usc")
version = project.version(1)
dataset = version.download("yolov8")
```

Hình 4.9: Tiến hành nhập dữ liệu từ Roboflow

Bước tiếp theo ta thiết lập mô hình học máy và tiến hành huấn luyện mô hình học máy:

- 3. Thiết lập mô hình YOLOv8

```
[ ] model = YOLO('yolov8n.pt')
```

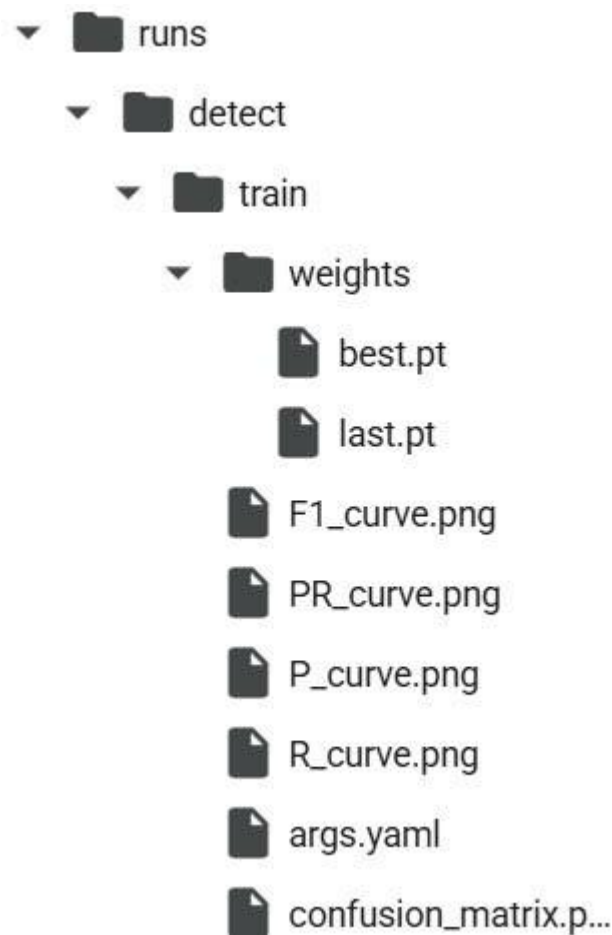
- 4. Huấn luyện mô hình YOLOv8

```
model.train(
    data="/content/Object-Classification-1/data.yaml",
    epochs=100
)
```

Ultralytics 8.4.39 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_rms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=True
Overriding model.yaml nc=80 with nc=3

Hình 4.10: Tiến hành thiết lập và huấn luyện mô hình qua 100 epochs

Kết thúc quá trình training, hệ thống tự động cho ra một số file jpg hiển thị kết quả train được.

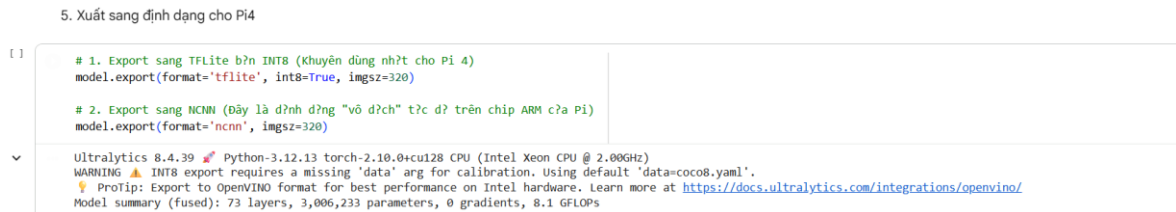


Hình 4.11: Hệ thống chia files

Kết quả cuối cùng của quá trình training là file best.py . Sử dụng file này trong code python nhận diện của dự án. Mục weights có chứa 2 file dữ liệu

last.pt và best.pt. Trong đó file last.pt có chứa kết quả train lần cuối cùng còn file Best là kết quả tốt nhất.

- Tối ưu hóa mô hình cho thiết bị nhúng (Raspberry Pi 4)



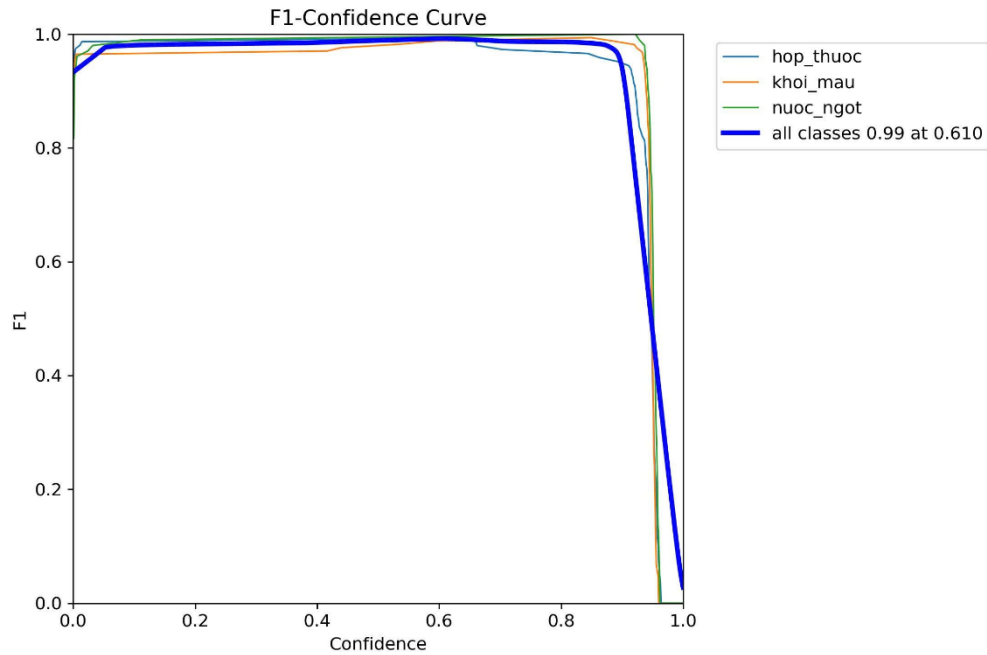
Hình 4.12: Chuyển đổi định dạng sang file TFLite chạy qua Raspberry Pi

Nhằm đảm bảo khả năng vận hành thời gian thực trên Raspberry Pi 4, mô hình YOLOv8s sau khi huấn luyện đã được tối ưu hóa và chuyển đổi sang các định dạng TFLite (INT8) và NCNN. Việc sử dụng kỹ thuật định lượng INT8 (8-bit quantization) giúp giảm đáng kể kích thước mô hình và khối lượng tính toán mà vẫn duy trì được độ chính xác cần thiết. Với cấu trúc gồm 73 lớp và khoảng 3.006.233 tham số, việc chuyển đổi này cho phép hệ thống tận dụng tối đa sức mạnh của kiến trúc chip ARM trên Pi 4. Kết quả là robot có thể xử lý hình ảnh ở độ phân giải 320x320 với tốc độ khung hình (FPS) ổn định, đảm bảo việc nhận diện màu sắc và điều khiển tay gấp diễn ra mượt mà, không bị trễ trong môi trường thực tế.

4.2.3. Phân tích kết quả train

- **Đồ thị F1 – Confidence Curve**

Đồ thị này biểu diễn mối quan hệ giữa giá trị F1 và Confidence cho các lớp và trường hợp khác nhau. Điểm số F1 (F1-score), được định nghĩa là trung bình điều hòa giữa độ chính xác và độ phủ, đóng vai trò là thang đo toàn diện để đánh giá hiệu năng của thuật toán, đặc biệt mang lại độ tin cậy cao đối với các tập dữ liệu có sự phân bố nhãn lớp không cân bằng [13]. Confidence là mức độ tự tin của mô hình vào dự đoán của nó. Đồ thị F1 – Confidence giúp xác định ngưỡng Confidence tối ưu để mô hình đạt hiệu suất cao nhất.



Hình 4.13: Đồ thị $F1$ – Confidence

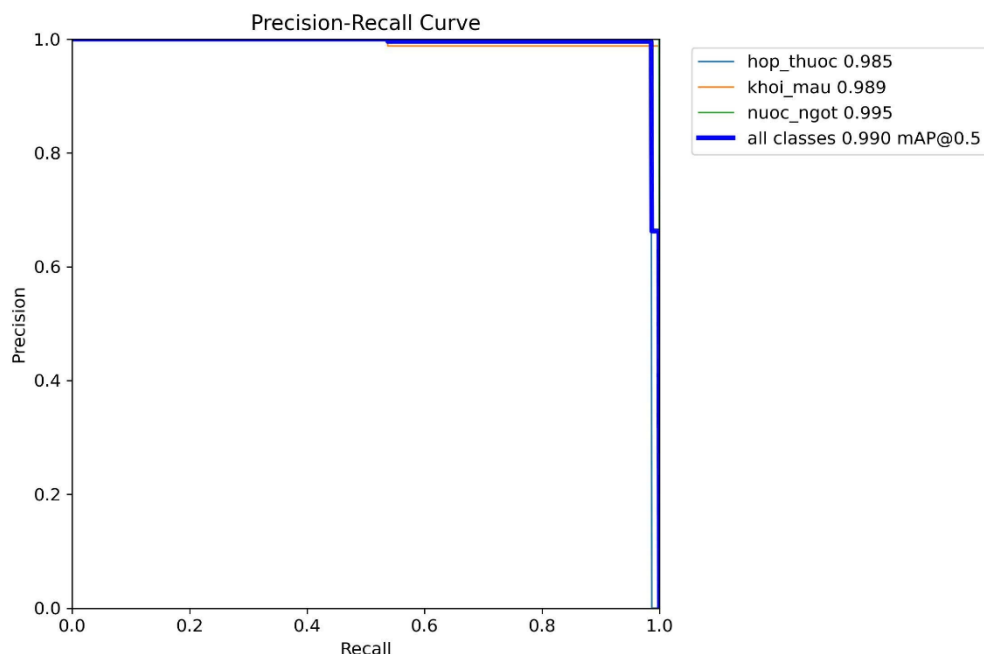
Nhận xét:

- Lớp "nuoc_ngot" (xanh lá): Đây là lớp có kết quả tốt nhất. Đường F1 duy trì ở mức tuyệt đối (~ 1.0) trên một khoảng Confidence rất rộng (từ 0.1 đến gần 0.9). Điều này chứng tỏ đặc trưng của các lon nước ngọt (màu sắc, nhãn hiệu) rất tách biệt, giúp mô hình nhận diện cực kỳ ổn định và không bỏ sót đối tượng.
- Lớp "khoi_mau" (cam): Chỉ số F1 rất cao và ổn định (đạt đỉnh ~ 0.99). Khả năng nhận diện khối màu gần như không có sai số. Việc đường biểu diễn đi ngang và phẳng cho thấy mô hình rất tự tin khi phân loại lớp này, dù ở các ngưỡng tin cậy khác nhau.
- Lớp "hop_thuoc" (xanh dương): Kết quả tương đương với lớp khối màu, đạt đỉnh F1 ~ 0.99 . Mặc dù hình dạng hộp thuốc có thể thay đổi tùy góc nhìn, nhưng mô hình vẫn học được các đặc trưng quan trọng để phân biệt chính xác với các vật thể khác trong hệ thống gắp.

Kết luận: Biểu đồ F1-Confidence đạt đỉnh **0.99 tại ngưỡng 0.610** và duy trì sự ổn định trong một khoảng giá trị rộng, chứng minh mô hình có tính bền vững (robustness) cao trước các biến động của môi trường

- **Đồ thị Precision – Recall Curve**

Đồ thị này biểu diễn mối quan hệ giữa độ chính xác và gọi lại cho các lớp khác nhau. Độ chính xác là tỷ lệ giữa số điểm true positive và tổng số điểm được dự đoán là positive. Gọi lại là tỷ lệ giữa số điểm true positive và tổng số điểm thực tế là positive. Đồ thị Precision-Recall giúp đánh giá hiệu suất mô hình trên các lớp khác nhau và xác định liệu mô hình có đủ mạnh để phân biệt giữa các lớp khác nhau hay không.



Hình 4.14: Đồ thị Precision-Recall

Nhận xét:

- Lớp “hop_thuoc” (xanh dương): mAP = 0.985. Đường cong duy trì ở mức cao và ổn định, chỉ giảm nhẹ khi Recall tiến sát giá trị 1.0. Điều này cho thấy mô hình nhận diện hộp thuốc rất tốt và cực kỳ ổn định ở nhiều góc độ.

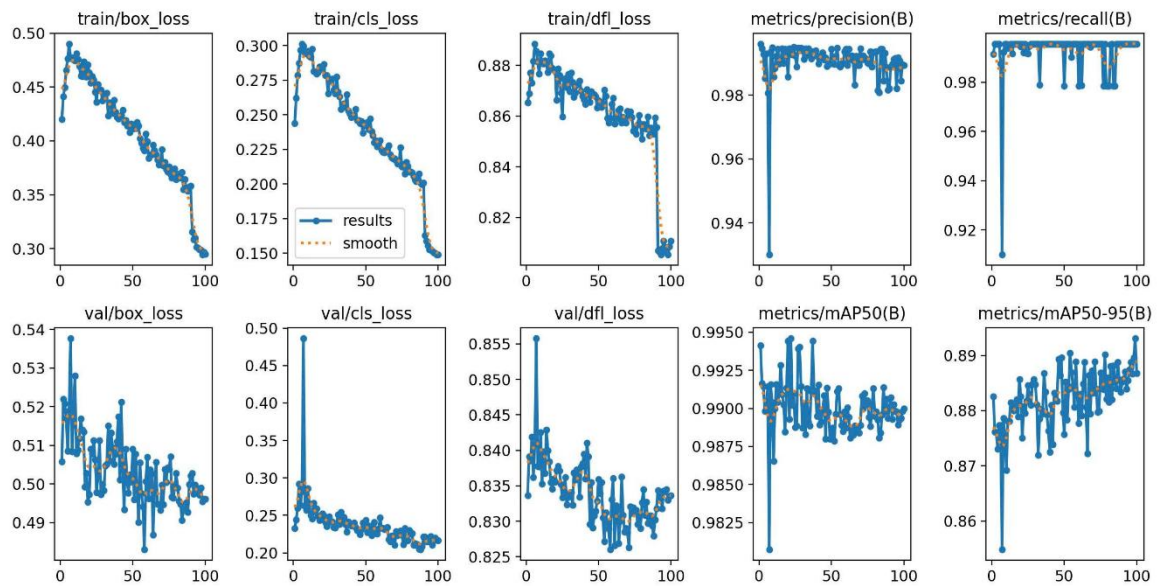
- Lớp “khoi_mau” (cam): $mAP = 0.989$. Chỉ số mAP rất cao, đường biểu diễn gần như phẳng ở mức trên 0.95, chứng tỏ mô hình có khả năng phân biệt khối màu cực kỳ chính xác và ít xảy ra sai sót.
- Lớp “nuoc_ngot” (xanh lá): $mAP = 0.995$. Đây là lớp mô hình nhận diện tốt nhất; đường biểu diễn vuông vức và bao phủ diện tích lớn nhất, cho thấy đặc trưng của vật thể này rất rõ ràng và tách biệt hoàn toàn so với nền.-
- Đường trung bình (màu xanh đậm): $mAP@0.5 = 0.990$. Kết quả tổng thể đạt mức xuất sắc. Đường cong cao và bao phủ diện tích lớn (gần như lấp đầy biểu đồ) chứng tỏ mô hình giữ được độ chính xác cực cao ngay cả khi cố gắng tìm kiếm toàn bộ vật thể (Recall tăng). Điều này khẳng định thuật toán đã học được đầy đủ các đặc trưng cần thiết và không bị nhiễu bởi môi trường xung quanh.

Kết luận: Mô hình hoạt động hoàn hảo cho cả 3 lớp, đã hội tụ và đạt độ tin cậy tối ưu. Với bộ thông số này, hệ thống đã sẵn sàng để triển khai thực tế trên robot để thực hiện các tác vụ gấp vật mà không cần cải thiện thêm về dữ liệu huấn luyện.

• **Đồ thị mất mát (Loss Graphs) và mAP**

Đồ thị mất mát biểu diễn sự thay đổi của hàm mất mát trong quá trình huấn luyện. Ta có thể vẽ riêng biệt cho mất mát hộp giới hạn (box loss), mất mát phân loại (classification loss), và mất mát dfl loss. Đồ thị mất mát giúp ta theo dõi sự tiến bộ của mô hình qua từng epoch và xác định liệu mô hình có đang học hỏi từ dữ liệu hay không, và hiện tượng quá khớp hoặc học kém

Đồ thị mAP biểu diễn giá trị mAP qua từng epoch, là chỉ số đánh giá hiệu suất tổng thể của mô hình. mAP là trung bình của Average Precision (AP) trên tất cả các lớp, nơi AP là diện tích dưới đường cong Precision-Recall. Đồ thị mAP giúp ta theo dõi hiệu suất tổng thể của mô hình qua thời gian.



Hình 4.15: Đồ thị mất mát (Loss Graphs) và mAP

Nhận xét:

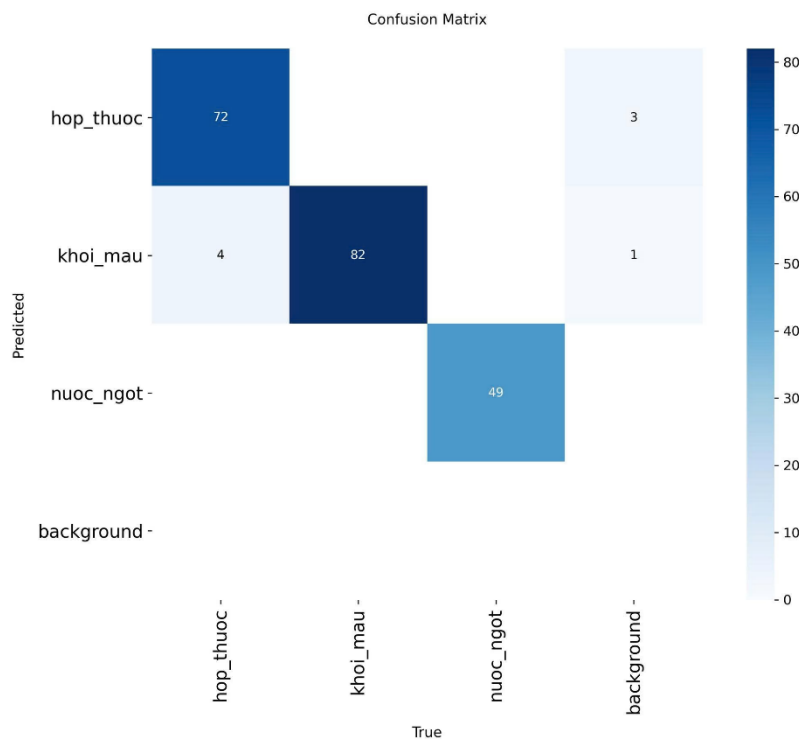
- Biểu đồ Train Loss: Box Loss, Cls Loss, và DFL Loss đều giảm ổn định và hội tụ rất tốt. Đặc biệt, tại epoch 90, các hàm mất mát giảm mạnh đột ngột do cơ chế tắt *Mosaic Augmentation* của YOLOv8 để tinh chỉnh mô hình, giúp đường "smooth" bám sát và mô hình học cực kỳ hiệu quả, không bị nhiễu.
- Biểu đồ Validation Loss: Các đường val/box_loss, val/cls_loss, và val/df_l_loss cùng giảm đều theo thời gian. Mặc dù có những dao động nhỏ do tập validation ít, nhưng xu hướng chung là giảm và đi ngang, cho thấy mô hình không có dấu hiệu overfitting.
- Precision và Recall: Cả hai chỉ số đều tăng vọt ngay trong ~10 epoch đầu tiên và duy trì mức cao kỷ lục ổn định đến cuối (~0.99). Điều này minh chứng mô hình dự đoán chính xác tuyệt đối và hầu như không bỏ sót bất kỳ đối tượng nào trong tập dữ liệu.
- mAP50: Đạt mức ~0.994, tăng nhanh và đạt trạng thái bão hòa rất sớm. Chỉ số này khẳng định khả năng nhận diện vật thể của mô hình ở ngưỡng IoU=0.5 là gần như hoàn hảo.

- mAP50-95: Tăng trưởng đều đặn và đạt mức rất cao ~ 0.895 . Điều này chứng tỏ mô hình không chỉ nhận diện đúng lớp mà còn có khả năng xác định vị trí khung bao (bounding box) cực kỳ chính xác và khít với vật thể thực tế.

Kết luận: Mô hình học rất ổn định và đạt hiệu suất xuất sắc trên cả tập train và tập val. Các loại loss giảm sâu, các chỉ số độ chính xác đều tiệm cận mức tối đa (mAP50 ~ 0.994 , mAP50-95 ~ 0.895). Đây là bộ thông số lý tưởng, khẳng định mô hình hoạt động cực kỳ hiệu quả và đã sẵn sàng để triển khai thực tế trên robot.

• *Confusion Matrix*

Ma trận nhầm lẫn cung cấp thông tin chi tiết về hiệu suất phân loại của mô hình. Mỗi hàng trong ma trận đại diện cho các lớp thực tế, và mỗi cột đại diện cho các lớp được dự đoán. Ma trận nhầm lẫn giúp ta xác định các lớp mà mô hình dễ nhầm lẫn và cần cải thiện.



Hình 4.16: Đồ thị nhầm lẫn confusion matrix

Nhận xét: Độ chính xác trên đường chéo chính (True Positives): Các ô trên đường chéo chính có màu xanh đậm và con số rất cao, cho thấy mô hình phân loại cực kỳ chính xác.

- Lớp nuoc_ngot đạt độ chính xác tuyệt đối (49/49 mẫu), không hề có sự nhầm lẫn nào với các lớp khác hay với nền.
- Lớp khoi_mau nhận diện đúng 82 mẫu, là lớp có số lượng mẫu thử nghiệm lớn nhất và đạt hiệu suất rất ổn định.
- Lớp hop_thuoc nhận diện đúng 72 mẫu.

Ngoài ra, biểu đồ xuất hiện một số sai số, được đánh giá như sau:

- Sự nhầm lẫn giữa các lớp: Có 4 mẫu khoi_mau bị nhầm thành hop_thuoc. Điều này có thể lý giải do trong một số góc chụp hoặc điều kiện ánh sáng, khối màu và hộp thuốc có thể có đặc điểm hình học (dạng khối hộp) tương đồng nhau, dẫn đến mô hình bị nhiễu nhẹ.
- Sai số với nền (Background Error): có 3 mẫu hop_thuoc và 1 mẫu khoi_mau bị mô hình dự đoán là nền (False Negative). Điều này thường xảy ra khi vật thể bị che khuất một phần hoặc nằm ở rìa khung hình khiến mô hình không đủ tự tin để gán nhãn.

Kết luận: Tỷ lệ nhầm lẫn giữa các vật thể là rất thấp (chỉ khoảng 4.8% cho lớp khối màu và gần như bằng 0 cho các lớp còn lại). Mô hình không bị hiện tượng "nhìn nhầm nền thành vật thể" từ background bằng 0. Những sai số nhỏ giữa khoi_mau và hop_thuoc hoàn toàn có thể khắc phục bằng cách thiết lập ngưỡng tin cậy phù hợp như đã phân tích ở biểu đồ F1-Curve, đảm bảo robot vận hành chính xác trong thực tế.

4.2.4. Hậu xử lý và lọc bỏ vật thể trùng lặp

Sau khi mạng nơ-ron thực hiện quá trình lan truyền thuận, dữ liệu đầu ra là một tập hợp các dự đoán thô. Để robot có thể đưa ra quyết định chính xác, thuật toán hậu xử lý được triển khai qua các giai đoạn sau:

Hậu xử lý và phân loại vật thể mục tiêu

Mỗi kết quả dự đoán từ mạng nơ-ron đều đi kèm với một xác suất định danh đối tượng. Thuật toán thực hiện hai nhiệm vụ trọng tâm:

Xác định nhãn: Sử dụng hàm tìm giá trị cực đại để xác định vật thể thuộc lớp nào trong 3 lớp: *hop_thuoc*, *khoi_mau*, *nuoc_ngot*.

$$ClassID = \operatorname{argmax}_i(P_i) \quad (4.8)$$

Trong đó P_i là xác suất thuộc về lớp đối tượng thứ i .

Lọc ngưỡng tin cậy: Hệ thống chỉ chấp nhận các dự đoán có mức độ xác nhận vượt qua một ngưỡng thực nghiệm tối ưu. Ngưỡng này được lựa chọn dựa trên thực nghiệm đồ thị F1-Curve nhằm loại bỏ các đối tượng nhiễu từ môi trường.

Lọc bỏ vật thể trùng lặp (Non-Maximum Suppression)

Trong nhận diện vật thể, một đối tượng thực tế thường bị bao phủ bởi nhiều khung bao dự đoán có vị trí tương đồng. Để tránh việc robot nhận sai số lượng hoặc nhiều tọa độ, thuật toán NMS được áp dụng để giữ lại duy nhất một khung bao có độ tin cậy cao nhất.

Tính toán IoU: Đánh giá mức độ chồng lấn giữa hai khung hình A và B.

$$IoU = \frac{Area(A \cap B)}{Area(A \cup B)} \quad (4.9)$$

Loại bỏ chồng lấn: Nhằm loại bỏ hiện tượng trùng lặp hộp bao trong khâu hậu xử lý, kỹ thuật Ức chế phi cực đại (Non-Maximum Suppression - NMS) được thực thi để giữ lại duy nhất dự đoán có độ tự tin cao nhất, dựa trên việc đánh giá chỉ số giao thoa trên liên kết (IoU) so với ngưỡng cho phép [14]. Kết quả cuối cùng là một khung bao duy nhất bao quát vật thể, giúp hệ thống xác định vị trí mục tiêu một cách dứt khoát.

4.3. THUẬT TOÁN XÁC ĐỊNH VỊ TRÍ VẬT THỂ

4.3.1. Phân tích và trích xuất tham số hình học của đối tượng

Trong quá trình di chuyển bám theo vạch, camera trên Raspberry Pi liên tục phân tích hình ảnh và nhận diện phân loại vật thể mục tiêu (hop_thuoc, khoi_mau, hoặc nuoc_ngot). Kết quả phân loại được mã hóa thành các ký tự tương ứng ('1', '2', '3') và truyền trực tiếp xuống Arduino qua chuẩn giao tiếp Serial (UART) ở tốc độ 9600 baud.

Tại bộ vi điều khiển trung tâm, thuật toán áp dụng cơ chế "lắng nghe không đồng bộ". Thông tin vật thể sẽ liên tục được cập nhật vào biến vat_nhan_dien với điều kiện cơ cấu kẹp đang ở trạng thái trống (khoa_tin_hieu == false). Để tối ưu hóa tài nguyên xử lý và tránh tràn bộ đệm, thuật toán tích hợp các bộ lọc thông minh:

- Lọc nhiễu: Bỏ qua các ký tự rác sinh ra trong quá trình truyền nhận như ký tự xuống dòng hoặc về đầu dòng (`\n`, `\r`).
- Chống lặp: Chỉ ghi nhận và xử lý tín hiệu khi dữ liệu mới nhận được khác với dữ liệu trước đó `data != last_data`.

4.3.2. Xác định vị trí tiếp cận chính xác

Một thách thức lớn trong gấp thả tự động là camera có thể nhận diện được vật thể từ khoảng cách xa, nhưng hệ thống cơ khí cần một điểm neo vật lý chính xác để tay kẹp (Servo) hoạt động hiệu quả. Để giải quyết, hệ thống sử dụng một cảm biến tiệm cận từ làm mốc kích hoạt.

Thuật toán liên tục theo dõi sự thay đổi trạng thái của cảm biến tiệm cận. Thay vì chỉ đọc mức logic đơn thuần, hệ thống bắt sừn xuống bằng cách so sánh trạng thái hiện tại (`current_sensor_state`) và trạng thái ở chu kỳ trước (`last_sensor_state`). Vị trí gấp vật thể được xác định là thành công khi cảm biến chuyển từ mức cao (chưa chạm ngưỡng) xuống mức thấp (vật thể đã nằm đúng trọng tâm của cơ cấu gấp).

4.3.3. Chuỗi logic "Xác nhận" và "Bắt giữ"

Ngay khi thuật toán xác định vật thể đã nằm đúng vị trí, hệ thống lập tức ngắt khâu điều khiển di chuyển bám line và kích hoạt chuỗi hành động bắt giữ với độ ưu tiên cao nhất:

- Dừng khẩn cấp và triệt tiêu quán tính: Vận tốc hai động cơ bước lập tức được gán bằng 0. Hệ thống áp dụng một khoảng trễ 2 giây để triệt tiêu hoàn toàn quán tính của robot, đảm bảo khung gầm đứng yên tĩnh tuyệt đối trước khi thao tác gấp.
- Vết cặn bộ đệm: Nhằm đề phòng độ trễ tín hiệu từ Raspberry Pi, ngay tại vị trí dừng, Arduino sẽ dùng vòng lặp `while` đọc toàn bộ dữ liệu còn tồn đọng trong bộ đệm Serial. Bước xác nhận này đảm bảo mã nhận diện (`vat_nhan_dien`) lưu trữ trong RAM là kết quả phân loại cuối cùng và chính xác nhất từ hệ thống AI.
- Thực thi cơ cấu chấp hành: Kích hoạt động cơ Servo quay từ góc mở (`goc_mo = 0`) đến góc đóng (`goc_dong = 70`) để giữ chặt vật thể.
- Chuyển đổi trạng thái máy: Hệ thống bật cờ `khoa_tin_hieu = true` để khóa ngõ vào Serial, ngưng nhận dữ liệu từ camera trong suốt quá trình vận chuyển nhằm tránh nhiễu tín hiệu. Thiết lập cờ `dang_ve_nguon = false` để báo hiệu robot đã có hàng và chuẩn bị bước vào phân hệ thuật toán dẫn đường ngã tư để đi giao hàng tại các ô quy định. Lùi robot lại một khoảng nhỏ để tạo không gian rẽ và tiếp tục hành trình bám line.

4.4. THUẬT TOÁN ĐIỀU KHIỂN TAY GẤP

Thuật toán điều khiển cơ cấu tay gấp là một phân hệ hoạt động với mức độ ưu tiên cao nhất trong vòng lặp hệ thống (vượt qua cả thuật toán dò đường PID). Hệ thống sử dụng một cảm biến tiệm cận từ làm tín hiệu kích hoạt và một động cơ RC Servo làm cơ cấu chấp hành.

Thuật toán được xây dựng dựa trên nguyên lý bắt sườn tín hiệu và máy trạng thái đảo, bao gồm các chu trình xử lý sau:

- Cơ chế phát hiện và ngắt khẩn cấp

Vi xử lý liên tục lấy mẫu trạng thái của cảm biến tiệm cận (`current_sensor_state`) và so sánh với trạng thái của chu kỳ ngay trước đó (`last_sensor_state`). Thuật toán không kích hoạt khi cảm biến duy trì ở mức thấp (LOW), mà chỉ kích hoạt duy nhất tại thời điểm xảy ra sườn tức là khoảnh khắc trạng thái chuyển đột ngột từ cao (chưa có vật) xuống thấp (vừa chạm vật).

Ngay khi điều kiện sườn xuống thỏa mãn, hệ thống lập tức can thiệp vào bộ điều khiển di chuyển:

- Dừng đột ngột: Ép vận tốc hai động cơ bước về 0.
- Triệt tiêu quán tính: Khởi tạo một độ trễ 2 giây để robot dừng lại hoàn toàn, đảm bảo tay gấp không bị lệch khỏi trọng tâm vật thể do trớn di chuyển.

- Máy trạng thái đảo chiều kẹp/nhả

Khác với các hệ thống sử dụng hai nút bấm hoặc hai cảm biến riêng biệt cho việc gấp và nhả, thuật toán này tối ưu hóa phần cứng bằng cách sử dụng chung một tín hiệu ngắt thông qua biến cờ trạng thái `dang_kep`. Mỗi lần cảm biến tiệm cận được kích hoạt, biến này sẽ đảo trạng thái (`dang_kep = !dang_kep`), phân nhánh luồng thực thi thành hai kịch bản đối lập:

Kịch bản 1: Thực thi gấp vật

Xảy ra khi robot đang ở trạng thái trống và tiến vào khu vực lấy hàng:

Khóa góc Servo: Động cơ Servo nhận lệnh quay từ góc mở (0) sang góc đóng (70) để ôm chặt vật thể.

Khóa nhiều truyền thông: Bật cờ `khoa_tin_hieu = true` để từ chối mọi dữ liệu nhận diện mới từ Raspberry Pi, bảo vệ mã hàng hóa hiện tại không bị ghi đè sai lệch trong suốt quãng đường di chuyển.

Cập nhật định tuyến: Đặt cờ `dang_ve_nguon = false`, báo hiệu cho hệ thống điều hướng ngã tư biết rằng robot đang mang hàng và cần di chuyển đến các ô địa chỉ (1, 2, hoặc 3).

Cơ động thoát vị trí: Hai bánh xe thực hiện lùi lại một khoảng ngắn, sau đó cấp vận tốc trái dấu để xoay chuyển hướng thân xe, chuẩn bị cho hành trình bám line tiếp theo.

Kịch bản 2: Thực thi nhả vật

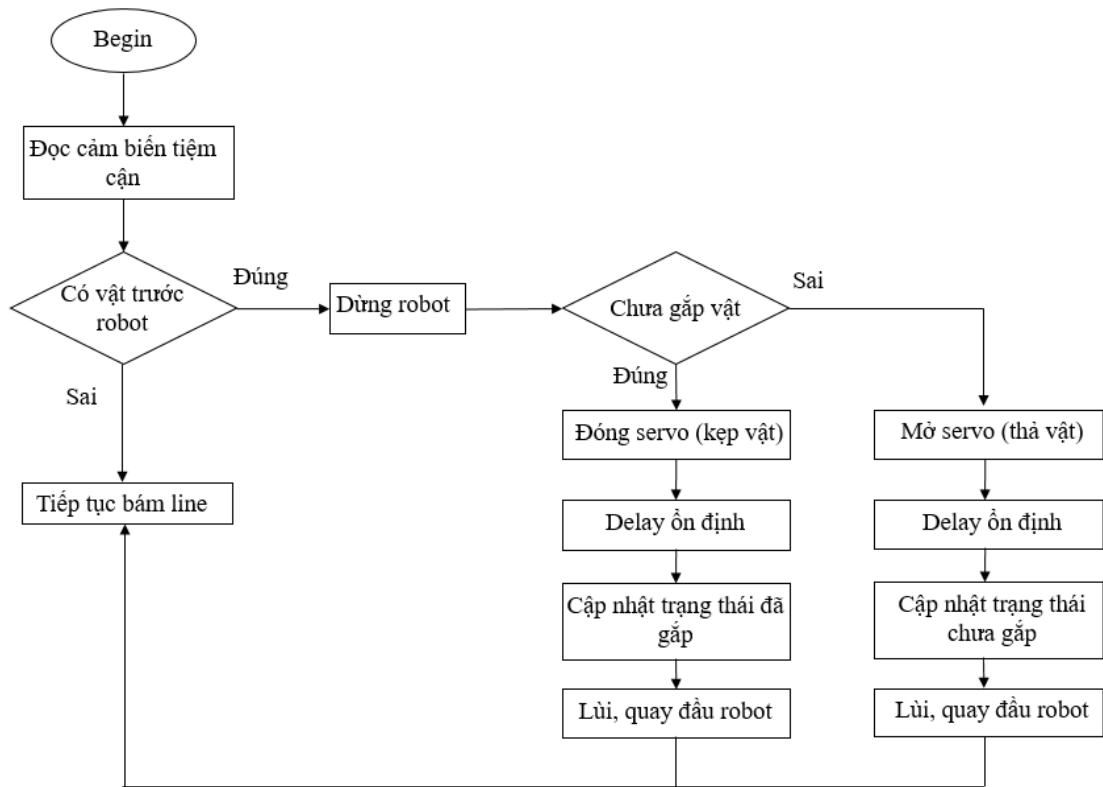
Xảy ra khi robot đã mang hàng đến đích và tiến vào sát vách tường hoặc điểm chắn của ô giao hàng:

Mở góc Servo: Động cơ Servo nhận lệnh quay về góc mở (0) để giải phóng vật thể.

Cập nhật định tuyến: Đặt cờ `dang_ve_nguon = true`, báo hiệu trạng thái trống tải. Lúc này, thuật toán xử lý ngã tư sẽ đảo ngược logic hướng đi để đưa robot quay trở về điểm xuất phát.

Cơ động quay đầu: Tương tự kịch bản gấp, robot lùi lại một khoảng đủ rộng, sau đó cấp vận tốc lớn hơn để thực hiện góc quay đầu 180, hướng mảng cảm biến dò đường về lại vạch line chính.

Kết thúc mỗi chu trình, hệ thống luôn thiết lập một khoảng thời gian chờ an toàn 3 giây để đảm bảo cơ cấu cơ khí của Servo đã hoàn tất góc quay trước khi trao lại quyền điều khiển cho thuật toán bám đường PID.



Hình 4.17: Lưu đồ thuật toán điều khiển tay gấp

4.5. LƯU ĐỒ THUẬT TOÁN HOẠT ĐỘNG CỦA ROBOT

Thuật toán hoạt động tổng thể của robot được xây dựng nhằm đảm bảo sự phối hợp nhịp nhàng giữa các chức năng: bám line, nhận diện vật thể, điều khiển tay gấp và điều hướng di chuyển. Hệ thống được tổ chức theo cơ chế vòng lặp liên tục, trong đó các khối chức năng được xử lý theo mức độ ưu tiên nhằm đảm bảo khả năng phản ứng nhanh và hoạt động ổn định trong môi trường thực tế.

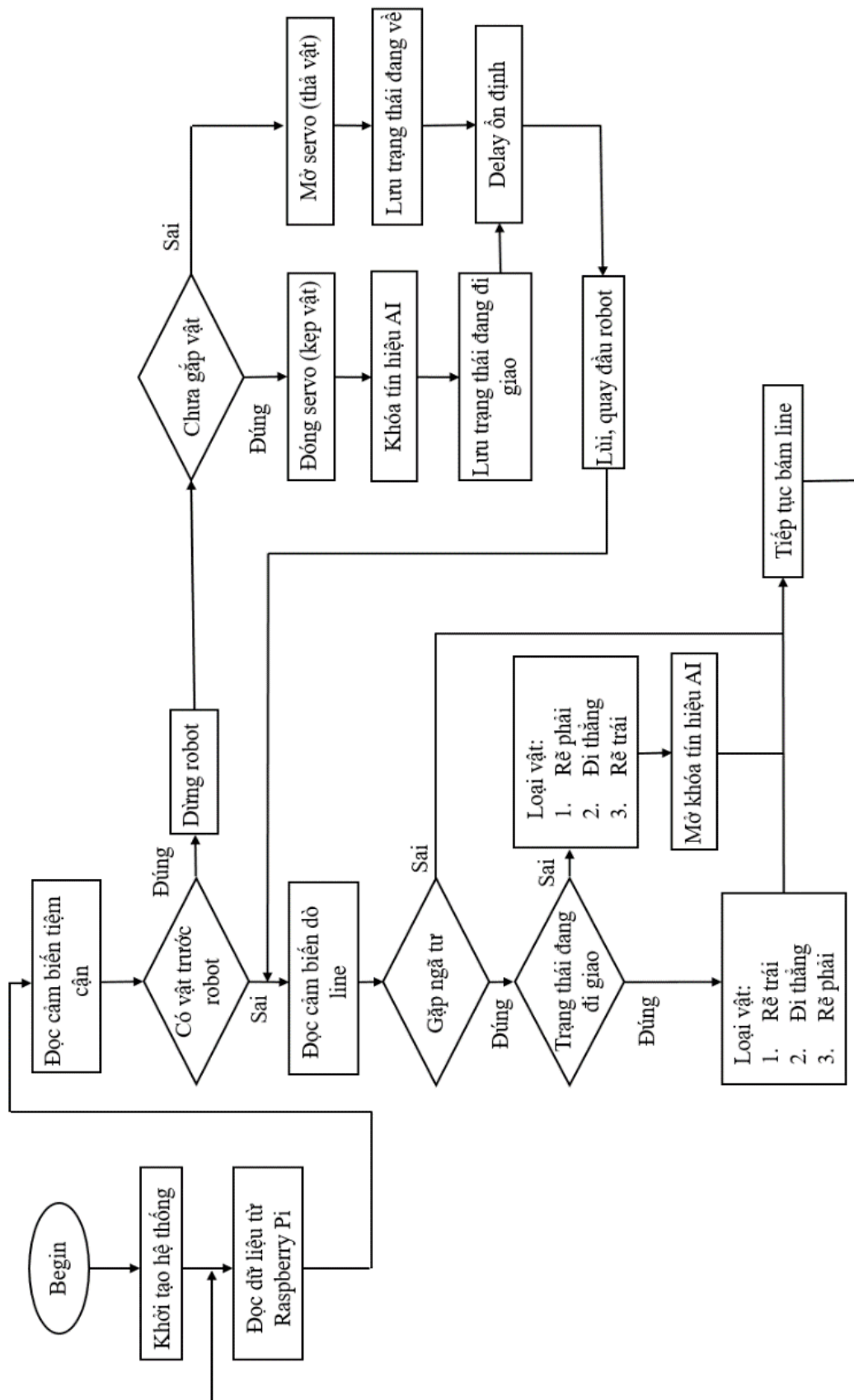
Cụ thể, trong mỗi chu kỳ lặp, robot trước tiên thực hiện việc nhận dữ liệu từ hệ thống xử lý ảnh trên Raspberry Pi thông qua giao tiếp nối tiếp. Dữ liệu này biểu thị loại vật thể đã được nhận diện và được lưu lại để phục vụ cho quá trình điều hướng sau đó. Tiếp theo, robot tiến hành đọc tín hiệu từ cảm biến tiệm cận để xác định sự xuất hiện của vật thể phía trước. Trong trường hợp phát hiện vật, hệ thống sẽ tạm dừng chuyển động và kiểm tra trạng thái của tay gấp. Nếu robot chưa thực hiện thao tác gấp, cơ cấu servo sẽ được kích hoạt để kẹp

vật, đồng thời khóa việc nhận dữ liệu mới từ Raspberry Pi nhằm tránh xung đột thông tin. Ngược lại, nếu robot đang ở trạng thái đã gặp vật và đến vị trí cần thả, servo sẽ được điều khiển mở để nhả vật, sau đó hệ thống cập nhật trạng thái và thực hiện quay đầu để quay về điểm xuất phát.

Trong trường hợp không phát hiện vật thể, robot chuyển sang xử lý tín hiệu từ dây cảm biến dò line để xác định vị trí tương đối của mình so với đường dẫn. Từ dữ liệu cảm biến, sai số điều khiển được tính toán và sử dụng làm đầu vào cho bộ điều khiển PID nhằm điều chỉnh tốc độ hai bánh xe, giúp robot di chuyển bám sát đường line một cách ổn định. Đồng thời, hệ thống cũng kiểm tra điều kiện xuất hiện ngã tư dựa trên mẫu tín hiệu từ cảm biến. Khi phát hiện ngã tư, robot sẽ tạm dừng và đưa ra quyết định hướng di chuyển dựa trên loại vật thể đã được xác định trước đó. Nếu đang trong quá trình vận chuyển vật, robot sẽ lựa chọn hướng rẽ tương ứng với từng loại vật; ngược lại, nếu đang quay về, robot sẽ điều chỉnh hướng để trở về vị trí ban đầu và đồng thời mở khóa việc nhận dữ liệu từ hệ thống nhận diện.

Ngoài ra, thuật toán cũng được xây dựng với khả năng xử lý các tình huống đặc biệt có thể xảy ra trong thực tế, chẳng hạn như mất tín hiệu line hoặc nhiễu từ cảm biến. Trong những trường hợp này, robot có thể dựa vào sai số trước đó để thực hiện các thao tác hiệu chỉnh hướng nhằm tìm lại đường line, từ đó duy trì tính liên tục của quá trình di chuyển. Điều này góp phần nâng cao độ tin cậy và tính ổn định của hệ thống trong điều kiện vận hành thực tế.

Toàn bộ quá trình trên được lặp lại liên tục trong suốt thời gian hoạt động của robot, tạo thành một hệ thống điều khiển khép kín. Cách tiếp cận này cho phép robot vừa đảm bảo khả năng bám line chính xác, vừa thực hiện hiệu quả các nhiệm vụ nhận diện, gặp và phân loại vật thể. Lưu đồ thuật toán được xây dựng nhằm biểu diễn trực quan quá trình xử lý này, qua đó làm rõ trình tự thực hiện cũng như mối quan hệ giữa các khối chức năng trong hệ thống, đồng thời hỗ trợ cho việc phân tích, đánh giá và cải tiến hệ thống trong các giai đoạn tiếp theo.



Hình 4.18: Lưu đồ thuật toán hoạt động của robot

4.6. XÂY DỰNG CHƯƠNG TRÌNH ĐIỀU KHIỂN CHO ARDUINO

Chương trình điều khiển trên Arduino được xây dựng nhằm hiện thực hóa toàn bộ thuật toán điều khiển của robot trong môi trường thực tế. Arduino đóng vai trò là bộ điều khiển trung tâm, chịu trách nhiệm tiếp nhận dữ liệu từ Raspberry Pi, xử lý tín hiệu từ các cảm biến và điều khiển các cơ cấu chấp hành như động cơ và tay gấp. Chương trình được tổ chức theo mô hình vòng lặp liên tục (loop()), đảm bảo hệ thống có thể phản hồi nhanh với các thay đổi của môi trường.

Về cấu trúc tổng thể, chương trình được chia thành các khối chức năng chính, mỗi khối đảm nhiệm một nhiệm vụ cụ thể nhưng có sự liên kết chặt chẽ trong quá trình vận hành:

Khởi tạo hệ thống : Arduino cấu hình các chân điều khiển động cơ, cảm biến dò line, cảm biến tiệm cận và servo. Đồng thời, giao tiếp Serial được thiết lập với tốc độ 9600 baud để nhận dữ liệu từ Raspberry Pi. Các biến trạng thái như trạng thái gấp (isGrabbing), trạng thái quay về và biến lưu loại vật (object_type) được khởi tạo ban đầu.

Tiếp nhận dữ liệu từ Raspberry Pi: Trong mỗi vòng lặp, Arduino kiểm tra Serial.available() để xác định có dữ liệu mới hay không. Khi có dữ liệu, chương trình sử dụng Serial.read() để đọc ký tự ('1', '2', '3') tương ứng với loại vật thể.

Điểm quan trọng trong code là:

- Chỉ cập nhật dữ liệu khi không bị khóa.
- Sử dụng biến flag (lock) để tránh thay đổi dữ liệu khi robot đang gấp hoặc vận chuyển.

→ Điều này giúp đảm bảo tính ổn định của quá trình điều khiển.

Xử lý cảm biến tiệm cận (ưu tiên cao nhất): Sau khi nhận dữ liệu, Arduino kiểm tra cảm biến tiệm cận để phát hiện vật thể.

Tùy theo trạng thái hiện tại:

- Nếu chưa gặp: dừng robot và đóng servo để kẹp vật.
- Nếu đã gặp: mở servo để thả vật tại điểm đích.

Đồng thời:

- Cập nhật trạng thái hệ thống.
- Khóa hoặc mở khóa dữ liệu từ Raspberry Pi.
- Thực hiện quay đầu robot.

→ Đây là khối có mức ưu tiên cao nhất vì liên quan trực tiếp đến nhiệm vụ chính.

Xử lý cảm biến dò line và điều hướng: Khi không có vật phía trước, robot sử dụng dãy cảm biến dò line để xác định vị trí.

Chương trình thực hiện:

Tính toán sai số (error) dựa trên tín hiệu cảm biến.

Nhận diện ngã tư thông qua mẫu tín hiệu đặc biệt. Khi gặp ngã tư:

- Nếu đang đi giao: rẽ theo object_type (1, 2, 3)
- Nếu đang quay về: rẽ ngược lại để về điểm xuất phát

Đồng thời mở khóa nhận dữ liệu để chuẩn bị cho chu kỳ mới

Điều khiển chuyển động bằng PID: Trong trường hợp di chuyển bình thường, Arduino áp dụng thuật toán PID để điều khiển tốc độ hai bánh xe.

Quá trình này bao gồm:

- Tính toán các thành phần P, I, D từ sai số.
- Kết hợp thành giá trị điều khiển.
- Điều chỉnh tốc độ động cơ trái và phải.

Ngoài ra, chương trình còn xử lý:

- Trường hợp mất line.
- Trường hợp cua gấp.

→ Giúp robot di chuyển ổn định hơn trong thực tế.

Chương trình điều khiển được xây dựng theo hướng kết hợp giữa xử lý tuần tự và quản lý trạng thái. Các khối chức năng được ưu tiên theo thứ tự rõ ràng: phát hiện vật thể, điều hướng và cuối cùng là bám line. Việc sử dụng các biến trạng thái và cơ chế khóa dữ liệu giúp hệ thống tránh được xung đột trong quá trình điều khiển, đồng thời đảm bảo tính liên tục và chính xác của hoạt động robot.

CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG

5.1. XÂY DỰNG MÔ HÌNH ROBOT THỰC TẾ

5.1.1. Lắp ráp hệ thống cơ khí

Phần đế robot: Khung đế chịu lực chính được thiết kế với kích thước 330mm x 250mm. Vật liệu được sử dụng là nhôm 6060 với độ dày 3mm, gia công bằng phương pháp cắt laser để đảm bảo tính chính xác tuyệt đối về dung sai và kích thước theo bản vẽ. Nhôm được lựa chọn thay vì inox hay nhựa là do nhôm có độ cứng cao, khối lượng nhẹ, tính thẩm mỹ tốt, dễ gia công và giá thành hợp lý. Bề mặt đế được bố trí sẵn các lỗ bắt vít để cố định board mạch, cảm biến và động cơ..

Phần vỏ robot: Vỏ có nhiệm vụ che chắn, bảo vệ các linh kiện bên trong khỏi va chạm vật lý và tạo tính thẩm mỹ cho mô hình. Vỏ được thiết kế dạng xe ô tô, bao gồm các hộc lắp nguồn, bánh xe, cảm biến và tay kẹp. Do hình dáng có nhiều đường nét phức tạp, phần vỏ được chế tạo bằng công nghệ in 3D sử dụng nhựa PetG, một loại vật liệu phổ thông giúp đảm bảo độ cứng cần thiết mà vẫn mang lại bề mặt hoàn thiện đẹp mắt.



Hình 5.1: Khung đế và vỏ của robot sau khi gia công

Lắp đặt hệ thống truyền động: Hai động cơ bước Step 42 được gắn cố định vào phần đế để dẫn động cho hai bánh xe trước (đường kính 60mm). Một bánh xe đa hướng đường kính 25mm được lắp ở phía sau để tạo thành cấu trúc tam giác cân, giúp robot cân bằng và di chuyển linh hoạt.

Lắp đặt cơ cấu tay gấp: Để thực thi nhiệm vụ thu gom sản phẩm, mặt trước robot được trang bị một tay gấp sử dụng cơ cấu khớp động khép kín. Cơ cấu này được truyền động trực tiếp từ động cơ servo MG995 có mô-men xoắn lớn. Với thiết kế nhỏ gọn, dễ gia công, tay gấp có khả năng đóng/mở linh hoạt để giữ các vật thể có kích thước lớn nhất lên đến 50mm và khối lượng chịu tải khoảng 500 gram.



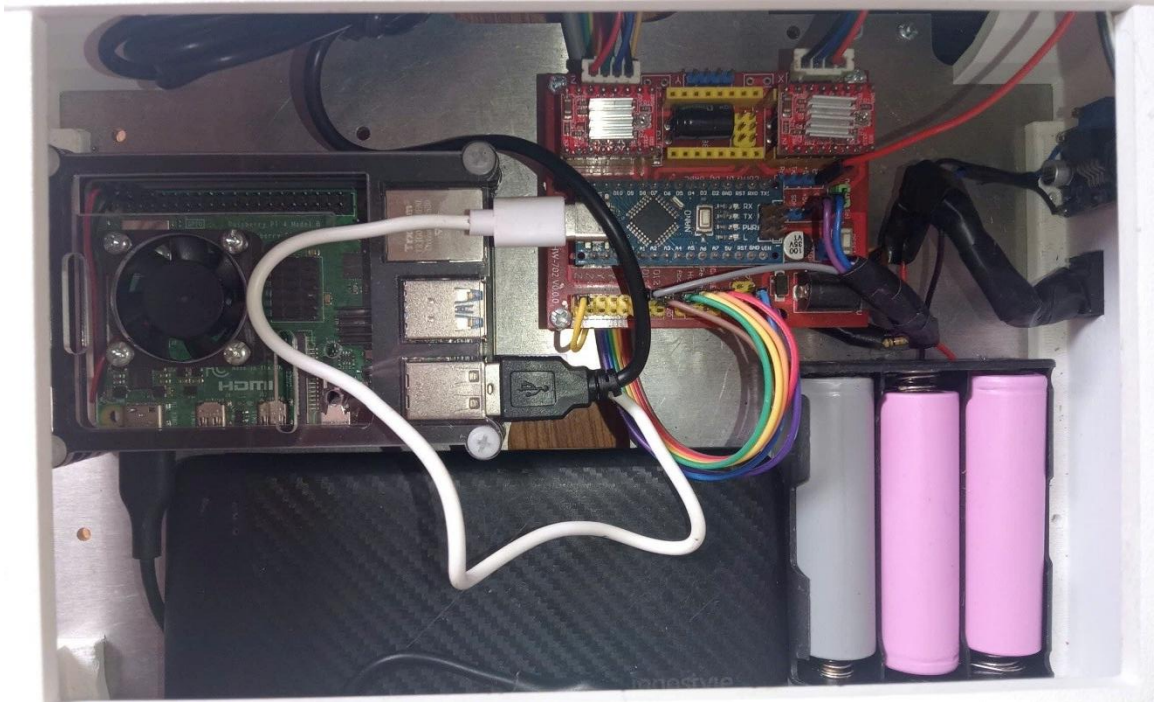
Hình 5.2: Cơ cấu chấp hành của robot sau khi gia công

5.1.2. Tích hợp hệ thống điện, cảm biến và điều khiển

Module điều khiển và xử lý: Mạch Arduino Nano được gắn cố định vào phần đế, đóng vai trò vi xử lý trung tâm điều khiển chuyển động. Ngay cạnh đó, máy tính nhúng Raspberry Pi 4 cũng được lắp đặt để chuyên trách khối lượng tính toán lớn của tác vụ trí tuệ nhân tạo và xử lý ảnh.

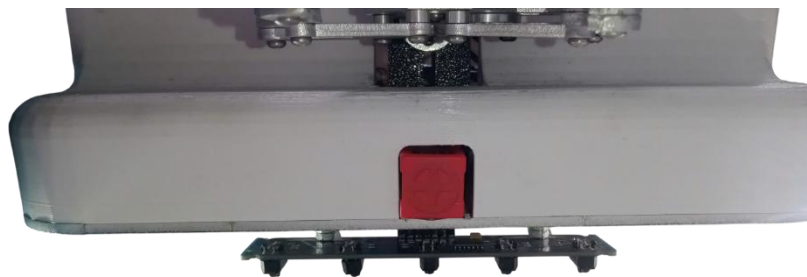
Hệ thống trí tuệ nhân tạo: Camera được kết nối với Raspberry Pi 4 để thu nhận hình ảnh môi trường. Sau khi Pi 4 xử lý, nhận dạng và phân loại sản phẩm, tín hiệu quyết định sẽ được truyền về Arduino thông qua giao thức kết nối nối

tiếp. Arduino sau đó sẽ phối hợp giữa việc điều hướng robot và kích hoạt servo MG995 thực hiện thao tác kẹp vật chính xác.



Hình 5.3: Hai bộ điều khiển chính được đi dây và lắp lên robot

Kết nối cảm biến: Thanh cảm biến dò line TCRT5000 (gồm 5 mắt đọc) được gắn dưới gầm xe, sát mặt đường để nhận diện vạch kẻ, kết hợp cùng cảm biến tiệm cận từ để phát hiện vật thể kim loại. Tín hiệu từ các cảm biến này truyền về Arduino.



Hình 5.4: Cảm biến dò line và cảm biến tiệm cận từ sau khi lắp lên robot

Quản lý năng lượng tách biệt: Nhằm đảm bảo hiệu suất hoạt động tốt nhất và tránh hiện tượng nhiễu điện từ do động cơ gây ra cho các bộ xử lý, hệ thống nguồn được thiết kế thành hai phần hoàn toàn tách biệt:

- Nguồn cung cấp cho Raspberry Pi 4: Sử dụng một nguồn cấp điện 5V - 2A chuyên dụng cho Raspberry Pi 4. Việc tách riêng nguồn này giúp Raspberry Pi 4 luôn duy trì được dòng điện ổn định để thực hiện các tác vụ tính toán nặng như thu nhận hình ảnh từ camera và nhận diện vật thể bằng AI mà không bị sụt áp khi robot khởi động hoặc gấp vật.
- Nguồn cung cấp cho khối điều khiển và chấp hành: Sử dụng hệ thống nguồn gồm 3 viên pin 18650. Nguồn năng lượng này được dùng để cấp phát cho:
 - Arduino Nano: Vi điều khiển trung tâm đảm nhiệm việc đọc tín hiệu cảm biến và xử lý logic di chuyển.
 - Hệ thống động cơ: Cung cấp năng lượng cho các động cơ bước Step 42 thông qua mạch driver A4988 và động cơ servo MG995 của tay gấp.
 - Hệ thống cảm biến: Cấp nguồn cho cụm cảm biến dò line TCRT5000 và cảm biến tiệm cận từ để nhận diện vạch kẻ cũng như vật thể kim loại

5.1.3. Tổng thể mô hình sau khi hoàn thiện

Sau khi hoàn tất quá trình chế tạo và tích hợp, mô hình robot tự hành đã đáp ứng đầy đủ các tiêu chuẩn thiết kế ban đầu. Về cấu trúc cơ khí, hệ thống được tối ưu hóa với phần đế nhôm 6060 đảm bảo khả năng chịu tải và độ cứng vững, kết hợp cùng vỏ nhựa PetG chế tạo bằng công nghệ in 3D. Khung gầm sử dụng cơ cấu truyền động vi sai ba bánh kết hợp cơ cấu tay gấp động học bốn khâu, đảm bảo tính linh hoạt trong quá trình điều hướng và độ chính xác khi thao tác gấp nhả vật thể.

Về hệ thống điện tử và điều khiển, mô hình ứng dụng kiến trúc xử lý phân tán. Máy tính nhúng Raspberry Pi 4 đảm nhiệm các tác vụ thị giác máy tính và phân loại, trong khi vi điều khiển Arduino Nano chịu trách nhiệm thu

nhận tín hiệu cảm biến và xuất xung điều khiển cơ cấu chấp hành. Đặc biệt, tính ổn định của toàn hệ thống được đảm bảo nhờ phương pháp cách ly nguồn điện: một mạch nguồn 5V-2A chuyên biệt được cấp cho Raspberry Pi nhằm chống sụt áp khi thực thi thuật toán AI, độc lập hoàn toàn với khối 3 pin 18650 cung cấp năng lượng cho Arduino, hệ thống cảm biến, động cơ bước và servo.



Hình 5.5: Mô hình robot sau khi hoàn thiện

Dựa trên quá trình hoàn thiện và đo đạc thực tế, cấu hình và hiệu năng của robot được tổng hợp theo bảng 5.1 thông số sau:

Bảng 5.1: Thông số thực tế của robot

Thông số	Kết quả
Khối lượng	4.2 kg
Tốc độ tối đa	15 cm/s
Dung lượng Pin	3000 mAh
Độ chính xác AI	> 95%
Thời gian xử lý	~50 ms/frame

5.2. THỬ NGHIỆM KHẢ NĂNG BẮM LINE CỦA ROBOT

- Thiết lập thử nghiệm

Để đánh giá hiệu năng của hệ thống, tiến hành thí nghiệm trong cùng điều kiện hoạt động với hai trường hợp:

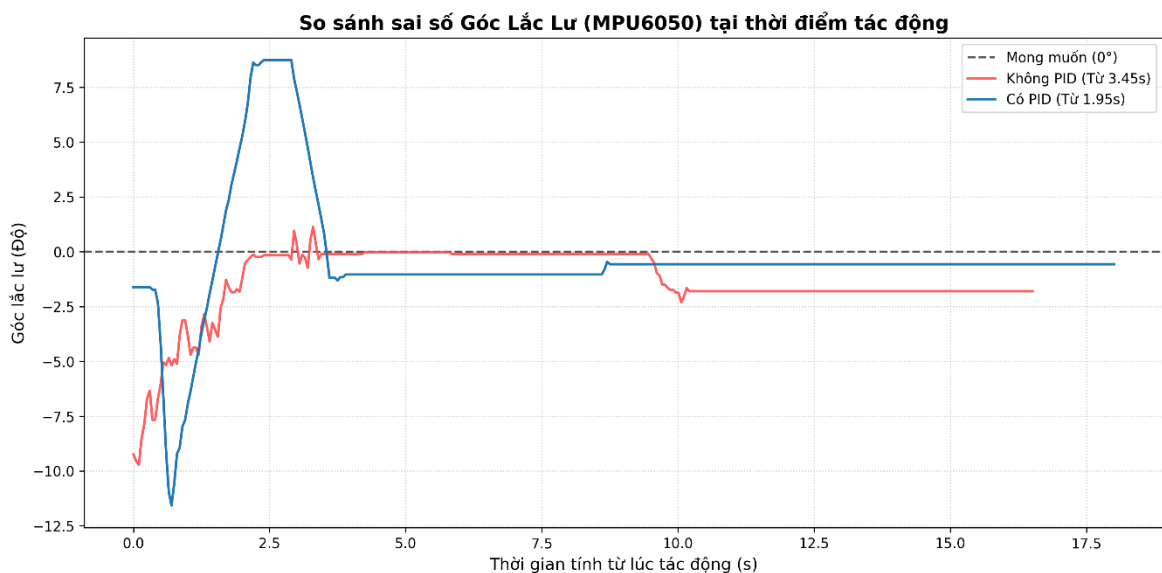
- Trường hợp 1: Robot không sử dụng bộ điều khiển PID
- Trường hợp 2: Robot sử dụng bộ điều khiển PID

Quy trình thử nghiệm được thực hiện như sau: Robot di chuyển ổn định trên đường line. Tại một thời điểm xác định, tác động một nhiễu bên ngoài làm robot lệch khỏi line

Ghi nhận:

- Góc dao động của robot (từ cảm biến MPU6050)
- Sai số vị trí so với line
- Quan sát quá trình và thời gian robot quay trở lại trạng thái bám line

- Kết quả thử nghiệm



Hình 5.6: Đồ thị so sánh sai số góc lắc lư (MPU6050) giữa hai hệ thống có PID và không có PID

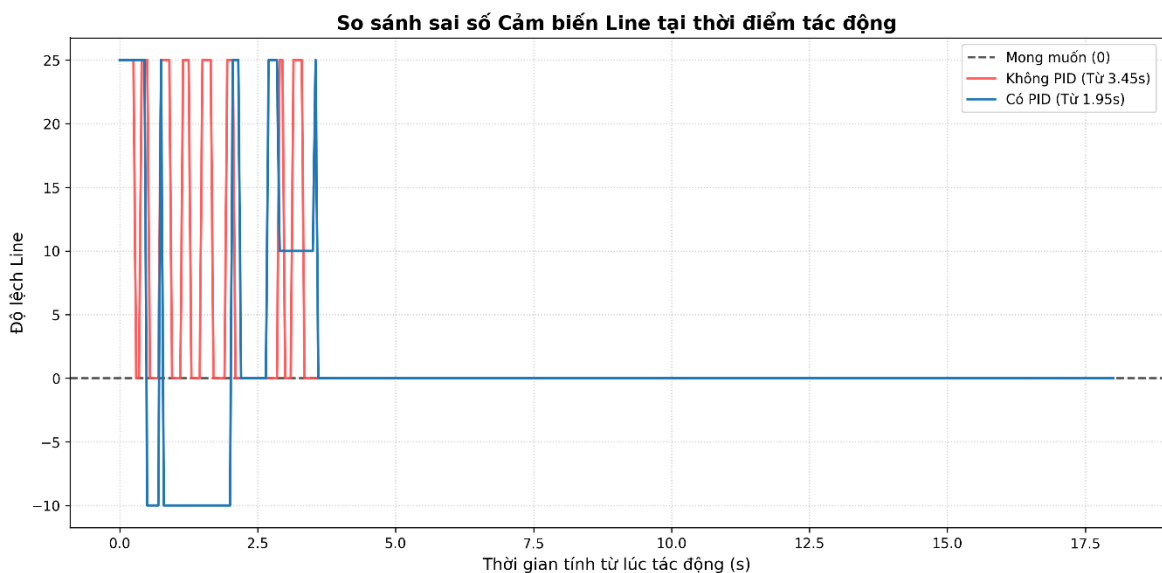
Dựa vào hình 5.6 ta có nhận xét:

Đồ thị so sánh đáp ứng góc lắc lư của hệ thống phản ánh rõ rệt vai trò của bộ điều khiển PID trong việc giữ ổn định tư thế cho robot:

Trường hợp không có PID: Ngay sau thời điểm chịu tác động góc lệch của robot tăng mạnh và liên tục dao động với biên độ lớn. Do không có lực phản hồi điều chỉnh, các dao động này không thể tự dập tắt hoặc mất rất nhiều thời gian, khiến robot rơi vào trạng thái mất kiểm soát tư thế trầm trọng.

Trường hợp có PID: Tại thời điểm tác động, mặc dù robot vẫn bị lệch góc do quán tính của ngoại lực, bộ điều khiển PID đã ngay lập tức tính toán và đưa ra tín hiệu phản hồi. Hệ thống nhanh chóng dập tắt các xung dao động đầu tiên. Góc lắc lư được kéo về sát đường mong muốn (0°) chỉ sau một khoảng thời gian ngắn (thời gian quá độ nhỏ), sai số xác lập sau đó được duy trì ổn định quanh mức triệt tiêu.

Kết luận: Bộ điều khiển PID giúp hệ thống kiểm soát tư thế cực tốt, giảm thiểu biên độ dao động đỉnh và tăng tốc độ ổn định của góc lái MPU6050 vượt trội so với hệ thống chạy vòng hở.



Hình 5.7: Đồ thị so sánh độ lệch quỹ đạo (Cảm biến Line) giữa hai hệ thống có PID và không có PID

Dựa vào *hình 5.7* ta có nhận xét:

Đồ thị bám quỹ đạo từ cảm biến mắt line minh chứng cho khả năng định hướng và chính xác khi di chuyển của robot:

Trường hợp không có PID: Khi không có thuật toán điều khiển, độ lệch line biến thiên tự do và có xu hướng tăng dần theo thời gian. Robot dễ dàng bị văng hoàn toàn ra khỏi vạch (sai số line đạt mức cực đại) và không có khả năng tự quay trở lại quỹ đạo mong muốn do các bánh xe chỉ quay theo thiết lập thô ban đầu.

Trường hợp có PID: Khi có sự tham gia của PID, ngay khi mắt line phát hiện robot bắt đầu chệch khỏi tâm vạch (sai số khác 0), thuật toán đã can thiệp để điều chỉnh tốc độ các động cơ điều hướng. Nhờ đó, đường cong bám line nhanh chóng hội tụ và dao động bám rất sát đường mong muốn (vị trí bằng 0), hiện tượng "vượt lố" (overshoot) xảy ra không đáng kể và triệt tiêu nhanh.

Kết luận: Thuật toán PID giúp robot bám quỹ đạo một cách mượt mà, chính xác, đảm bảo robot không bị mất line ngay cả khi di chuyển qua các đoạn cua hoặc chịu tác động ngoại cảnh làm lệch hướng.

5.3. THỬ NGHIỆM NHẬN DIỆN VẬT THỂ BẰNG CAMERA

- *Thiết lập thử nghiệm*

Thí nghiệm được thực hiện trên hệ thống robot di động sử dụng camera gắn trực tiếp để thu thập hình ảnh môi trường phía trước. Dữ liệu hình ảnh được truyền về bộ xử lý Raspberry Pi 4 để thực hiện nhận diện bằng mô hình YOLOv8 đã được huấn luyện trước. Đối tượng thử nghiệm gồm 3 loại:

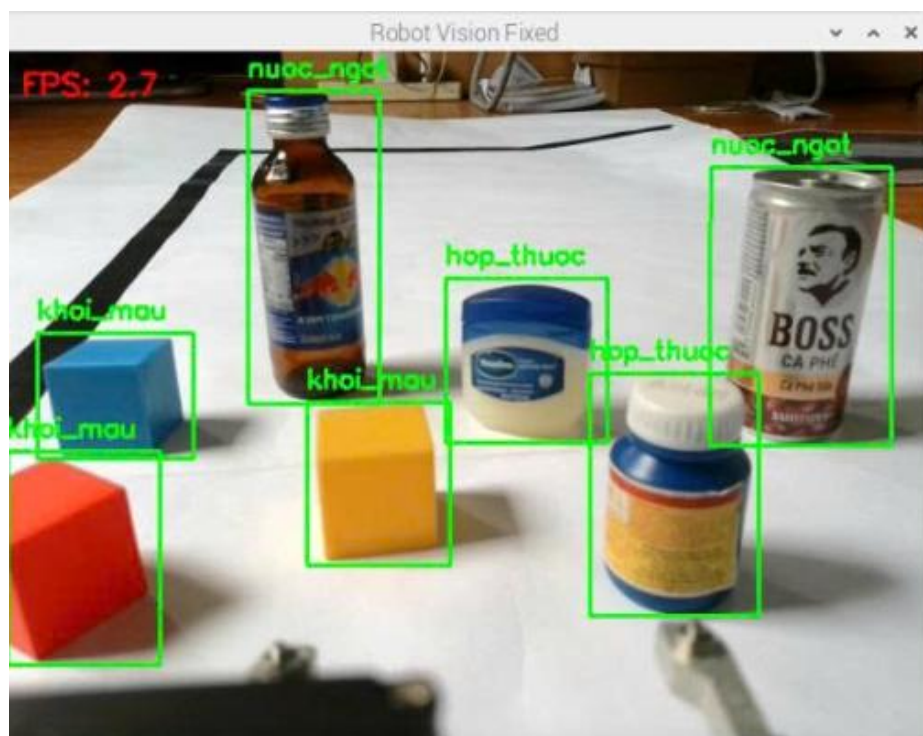
- Hộp màu (đỏ, xanh, vàng)
- Lọ thuốc (Vaseline, Carbo)
- Chai nước ngọt

Được đặt cố định trong vùng quan sát của camera với khoảng cách từ 15 đến 30 cm. Thí nghiệm được tiến hành trong điều kiện ánh sáng trong nhà tương đối ổn định nhằm hạn chế nhiễu từ môi trường. Trong quá trình thử nghiệm, robot di chuyển đến vị trí nhận diện, dừng lại, sau đó camera tiến hành thu hình và hệ thống AI thực hiện phân tích, đưa ra kết quả phân loại vật thể. Toàn bộ quá trình được lặp lại nhiều lần để đảm bảo tính khách quan và độ tin cậy của kết quả thu được.

- ***Thử nghiệm nhận diện đa vật thể***

Trong điều kiện thực nghiệm thứ nhất, hệ thống được kiểm tra khả năng xử lý khi có nhiều đối tượng xuất hiện đồng thời trong cùng một khung hình.

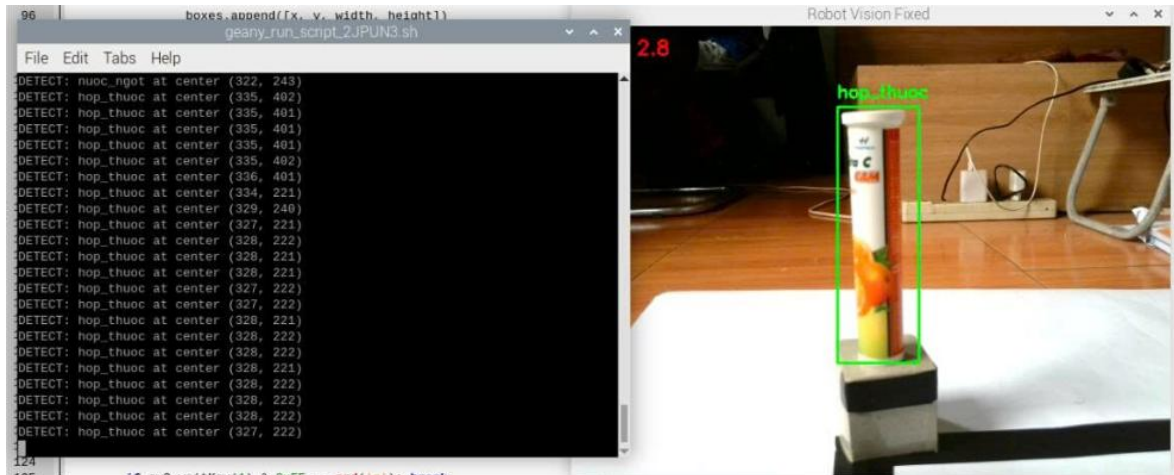
Mô hình thể hiện khả năng nhận diện chính xác các lớp đối tượng mục tiêu với các khung bao bám sát biên dạng thực của vật thể. Ngay cả khi xảy ra hiện tượng chồng lấn cục bộ hệ thống vẫn duy trì sự phân biệt rạch ròi giữa các thực thể. Chỉ số tin cậy của các dự đoán luôn duy trì ở mức cao, khẳng định mô hình đã trích xuất được các đặc trưng tối ưu từ tập dữ liệu huấn luyện, đảm bảo tính ổn định cho quá trình vận hành.



Hình 5.8: Thử nghiệm nhận diện đa vật thể trong một khung hình

- Thử nghiệm nhận diện đơn vật thể

Thử nghiệm thứ hai tập trung vào việc đánh giá độ chính xác của dữ liệu đầu ra khi robot đang di chuyển trên quỹ đạo bám line để thực hiện nhận diện từng vật thể riêng lẻ.



Hình 5.9: Nhận diện vật thể robot chạy theo đường line và trả về tọa độ tâm vật thể

Khi phát hiện đối tượng, mô hình thực hiện trích xuất thông tin vị trí của khung bao dự đoán dưới dạng tọa độ các điểm biên ($x_{min}, y_{min}, x_{max}, y_{max}$) trong không gian ảnh. Để phục vụ bài toán điều khiển cơ cấu chấp hành (tay gắp) từ tọa độ tâm của vật thể (x_c, y_c).

Đánh giá sai số: Kết quả thực nghiệm cho thấy việc xác định tọa độ có sai số nằm trong ngưỡng cho phép, đáp ứng tốt yêu cầu về độ chính xác của bài toán gắp đặt vật thể trong thực tế.

5.4. THỬ NGHIỆM GẮP VÀ VẬN CHUYỂN VẬT THỂ

- Thiết lập thử nghiệm

Thí nghiệm được thực hiện trên hệ thống robot di động hoàn chỉnh, bao gồm cơ cấu di chuyển, camera nhận diện và cơ cấu gắp vật thể. Các vật thể thử nghiệm được đặt tại vị trí cố định trên khu vực làm việc. Robot bắt đầu từ điểm xuất phát, di chuyển theo line đến khu vực chứa vật thể. Robot dừng lại tại vị trí thích hợp để thực hiện thao tác gắp bằng cơ cấu servo. Tiếp theo, robot vận

chuyển vật thể theo đường line đến vị trí đích tương ứng và tiến hành thả vật. Thử nghiệm được tiến hành nhiều lần với các vị trí vật thể khác nhau nhằm đánh giá độ chính xác trong quá trình dừng, khả năng gấp và độ ổn định khi vận chuyển. Điều kiện môi trường được giữ ổn định với ánh sáng trong nhà và mặt sàn phẳng nhằm hạn chế các yếu tố nhiễu ảnh hưởng đến kết quả.

- ***Kết quả thử nghiệm***

Kết quả thử nghiệm cho thấy hệ thống robot hoạt động ổn định và thực hiện đúng quy trình gấp và vận chuyển vật thể đã đề ra. Robot có khả năng di chuyển chính xác theo line, dừng tại vị trí phù hợp để thực hiện thao tác gấp, đồng thời cơ cấu gripper hoạt động hiệu quả, đảm bảo giữ vật thể trong suốt quá trình vận chuyển.



Hình 5.10: Thử nghiệm gấp và vận chuyển vật thể

5.5. ĐÁNH GIÁ KẾT QUẢ THỰC NGHIỆM

Độ ổn định của phần cứng: Hệ thống cơ khí (bao gồm khung nhôm, vỏ in 3D, tay kẹp và hệ truyền động) thể hiện độ bền cơ học cao, chịu tải tốt và hoạt động trơn tru. Robot vận hành ổn định sau nhiều giờ liên tục mà không gặp hiện tượng nứt vỡ, biến dạng hay hao mòn.

Hệ thống xử lý và phản hồi tín hiệu: Cảm biến dò line và tiệm cận từ có độ nhạy cao, hỗ trợ robot bám quỹ đạo và nhận diện điểm dừng gấp chính xác.

Dù cảm biến dò line thỉnh thoảng bị nhiễu nhẹ do ánh sáng gắt, vi điều khiển Arduino và mạch driver vẫn xử lý tín hiệu mượt mà, không xảy ra quá tải, tràn bộ nhớ hay mất bước động cơ.

Hệ thống xử lý ảnh: Bộ đôi Camera và Raspberry Pi 4 thực hiện tốt việc phân loại vật thể theo màu sắc, hình dạng trong điều kiện ánh sáng tiêu chuẩn. Tuy nhiên, độ chính xác có xu hướng giảm khi phong nền phức tạp và tốc độ khung hình (FPS) hiện tại chưa đủ đáp ứng nếu robot di chuyển ở vận tốc cao.

Tổng quan mô hình: Robot đã hoàn thành xuất sắc các mục tiêu thiết kế ban đầu. Các module phần cứng và phần mềm phối hợp nhịp nhàng, tạo thành một hệ thống tự hành khép kín đáng tin cậy, làm nền tảng vững chắc cho các cải tiến tiếp theo trong tương lai.

5.6. HẠN CHẾ CỦA HỆ THỐNG VÀ HƯỚNG PHÁT TRIỂN

5.6.1. Hạn chế của hệ thống

Hệ thống cảm biến và nhận diện: Khả năng dò line phụ thuộc nhiều vào điều kiện môi trường; ánh sáng gắt hoặc bề mặt phản xạ không đều dễ gây nhiễu tín hiệu, dẫn đến sai lệch quỹ đạo. Đối với xử lý ảnh, khi phong nền phức tạp hoặc màu vật thể chìm vào nền, độ chính xác nhận dạng bị suy giảm đáng kể. Hơn nữa, tốc độ xử lý khung hình (FPS) hiện tại chưa đủ lớn để robot kịp thời nhận dạng nếu di chuyển ở tốc độ cao.

Cơ khí và truyền động: Sai số nhỏ giữa hai bánh chủ động vẫn tồn tại, có thể làm robot lệch hướng dần khi vận hành trong thời gian dài. Việc thiếu vắng encoder khiến hệ thống không có luồng phản hồi tốc độ để điều khiển một cách chính xác tuyệt đối. Bên cạnh đó, tay gắp chưa điều chỉnh được lực kẹp linh hoạt, khiến việc thích ứng với các vật thể có kích thước/khối lượng khác nhau gặp khó khăn.

Mức độ tự hành: Hoạt động của robot hiện vẫn bị giới hạn ở việc bám theo vạch kẻ sẵn, chưa thể tự định vị hay xây dựng bản đồ để di chuyển linh hoạt trong các không gian có cấu trúc phức tạp.

5.6.2. Phương hướng phát triển

Cải tiến phần cứng và cơ khí: Chuyển đổi sang hệ thống truyền động đa hướng và sử dụng động cơ DC hoặc BLDC kết hợp với encoder và driver chất lượng cao để tăng hiệu suất tốc độ và độ chính xác. Nâng cấp tay gấp bằng cách bổ sung cảm biến lực và thêm các bậc tự do (như cơ cấu nâng/hạ) để tương tác tốt hơn với đa dạng vật thể. Cải thiện cụm camera và trang bị thêm hệ thống chiếu sáng phụ trợ để ổn định nguồn sáng.

Tối ưu thuật toán xử lý: Ứng dụng các thuật toán học sâu mạnh mẽ hơn kết hợp với kỹ thuật lọc tín hiệu để loại bỏ nhiễu môi trường, từ đó giảm độ trễ, tăng tốc độ xử lý và nâng cao độ tin cậy của cả AI lẫn bộ dò line.

Nâng cấp tư duy tự hành: Tích hợp các thuật toán định vị và lập bản đồ đồng thời (SLAM) giúp robot tiến tới mức độ tự hành cao hơn, thoát khỏi sự phụ thuộc vào đường line và mở rộng khả năng ứng dụng vào thực tế công nghiệp.

TÀI LIỆU THAM KHẢO

- [1] G. Fragapane, R. De Koster, F. Sgarbossa và J. C. Strandhagen, “Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda,” *European Journal of Operational Research*, tập 294, số 2, pp. 405-426, 2021.
- [2] A. e. a. Macario Barros, “A comprehensive survey of visual slam algorithms,” *IEEE Access*, tập 10, pp. 45842-45862, 2022.
- [3] A. K. M. M. Islam và e. al., “Design and Fabrication of a Line Follower Robot using PID Controller,” trong *2019 IEEE International Conference on Robotics, Automation, Artificial-intelligence and Internet-of-Things (RAAICON)*, 2019.
- [4] A. Borase, D. K. Maghade, S. Y. Sondkar và S. N. Pawar, “A review of PID control, tuning methods and applications,” *International Journal of Dynamics and Control*, tập 9, số 2, pp. 818-827, 2021.
- [5] A. Voulodimos, N. Doulamis, A. Doulamis và E. Protopapadakis, “Deep Learning for Computer Vision: A Brief Review,” *Computational Intelligence and Neuroscience*, tập 2018, 2018.
- [6] S. Alzubaidi và e. al., “Review of deep learning: concepts, CNN architectures, challenges, applications, future directions,” *Journal of Big Data*, tập 8, số 53, 2021.
- [7] J. Terven và D. Córdova-Esparza, “A Comprehensive Review of YOLO: From YOLOv1 to YOLOv8 and Beyond,” *arXiv*, 2023.
- [8] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Z. Chen, J.-W. Hsieh và I.-H. Yeh, “CSPNet: A New Backbone that can Enhance Learning Capability of CNN,” trong *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2020.

- [9] Z. Tian, C. Shen, H. Chen và T. He, “FCOS: Fully Convolutional One-Stage Object Detection,” trong *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [10] R. Padilla, S. L. Netto và E. A. B. da Silva, “A Survey on Performance Metrics for Object-Detection Algorithms,” trong *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)*, 2020.
- [11] L. Hosang, R. Benenson và B. Schiele, “Learning non-maximum suppression,” trong *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [12] S. Liu, L. Qi, H. Qin, J. Shi và J. Jia, “Path Aggregation Network for Instance Segmentation,” trong *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [13] C. Shorten và T. M. Khoshgoftaar, “A survey on Image Data Augmentation for Deep Learning,” *Journal of Big Data*, tập 6, số 60, 2019.

PHỤ LỤC

Phụ lục 1: Mã nguồn arduino

```
#include <Servo.h>

// ===== 1. KHAI BÁO CHÂN MOTOR & SENSOR DÒ LINE =====
// QUY ƯỚC: M1 = bánh PHẢI, M3 = bánh TRÁI
#define Enable 8
#define Step_3 7
#define Step_1 5
#define Dir_3 4
#define Dir_1 2

#define Pin_ss1 A0 // sensor trái nhất
#define Pin_ss2 A1
#define Pin_ss3 A2 // sensor giữa
#define Pin_ss4 A3
#define Pin_ss5 12 // sensor phải nhất

// ===== 2. KHAI BÁO SERVO & CẢM BIẾN TIỆM CẬN =====
#define SENSOR_TIEM_CAN 13
#define SERVO_PIN 11

// ===== 3. BIẾN NHẬN DIỆN & LOGIC HỆ THỐNG =====
char vat_nhan_dien = '0'; // '1': hop_thuoc, '2': khoi_mau, '3': nuoc_ngot
bool dang_ve_nguon = false; // Trạng thái để biết robot đang quay lại điểm A
char last_data = '0'; // Biến chống spam từ file test
bool khoa_tin_hieu = false; // Khóa miệng Pi lại trong suốt 1 chu kỳ

Servo myServo;
int goc_mo = 0;
int goc_dong = 70;
bool dang_kep = false;
int last_sensor_state = HIGH;

// ===== 4. BIẾN HỖ TRỢ MOTOR & TIMER =====
volatile int16_t Count_timer1 = 0, Count_timer3 = 0;
int16_t Count_TOP1, Count_BOT1, Count_TOP3, Count_BOT3;
int8_t Dir_M1 = 0, Dir_M3 = 0;

// ===== 5. THAM SỐ TỐC ĐỘ =====
const int BASE_PERIOD = 20; // Số nhỏ = nhanh
const int MIN_PERIOD = 5;
```

```

const int MAX_PERIOD = 200;
const float TURN_SLOWDOWN = 1.8;

// Tham số của găt PIVOT: 1 bánh tiến chậm, 1 bánh lùi nhanh -> xoay tại chỗ
const int PIVOT_FORWARD = 25;
const int PIVOT_REVERSE = -50;

// VEL_SLEW: gia tốc mượt cho stepper, tránh mất bước
const int VEL_SLEW = 12;

// ===== 6. THAM SỐ PID =====
float Kp = 1.8;
float Kd = 40.0;
float Ki = 0.0;
float previous_error = 0;
float P, I, D, PID_value;
float error_pid = 0;

// ===== 7. STATE MACHINE & MEMORY =====
enum LineState { LINE_ON, LINE_LOST, LINE_CROSS,
LINE_SHARP_L, LINE_SHARP_R };
LineState line_state = LINE_ON;
LineState last_sharp_dir = LINE_ON; // Nhớ hướng của găt cuối cùng

// ===== 8. BIẾN ĐIỀU KHIỂN VELOCITY =====
int target_period_M1 = MAX_PERIOD;
int target_period_M3 = MAX_PERIOD;
int v_M1_curr = 0, v_M3_curr = 0;

// ===== 9. ĐIỀU KHIỂN STEPPER =====
void Speed_M1(int16_t x) {
    if (x == 0) { Dir_M1 = 0; Count_timer1 = 0; PORTD &= 0b11011111;
return; }
    if (x > 0) { Dir_M1 = 1; PORTD |= 0b00000100; }
    else      { Dir_M1 = -1; PORTD &= 0b11111011; }
    Count_BOT1 = abs(x); Count_TOP1 = abs(x) / 2;
}

void Speed_M3(int16_t x) {
    if (x == 0) { Dir_M3 = 0; Count_timer3 = 0; PORTD &= 0b01111111;
return; }
    if (x > 0) { Dir_M3 = 1; PORTD |= 0b00010000; }
    else      { Dir_M3 = -1; PORTD &= 0b11101111; }
}

```

```

    Count_BOT3 = abs(x); Count_TOP3 = abs(x) / 2;
}

// ===== 10. CHUYỂN ĐỔI VELOCITY <-> PERIOD =====
int16_t v_to_period(int v) {
    if (v == 0) return 0;
    v = constrain(v, -100, 100);
    int p = MAX_PERIOD - (long)(MAX_PERIOD - MIN_PERIOD) * (abs(v)
- 1) / 99;
    return (v > 0) ? p : -p;
}

int16_t period_to_v(int p) {
    if (p == 0) return 0;
    int ap = constrain(abs(p), MIN_PERIOD, MAX_PERIOD);
    int v = 1 + (long)(MAX_PERIOD - ap) * 99 / (MAX_PERIOD -
MIN_PERIOD);
    return (p > 0) ? v : -v;
}

int16_t slew_lin(int curr, int target, int step) {
    int d = target - curr;
    if (abs(d) <= step) return target;
    return curr + (d > 0 ? step : -step);
}

// ===== 11. ĐỌC CẢM BIẾN DÒ LINE =====
float readLinePosition() {
    uint8_t w1 = !digitalRead(Pin_ss1);
    uint8_t w2 = !digitalRead(Pin_ss2);
    uint8_t w3 = !digitalRead(Pin_ss3);
    uint8_t w4 = !digitalRead(Pin_ss4);
    uint8_t w5 = !digitalRead(Pin_ss5);
    uint8_t sum = w1 + w2 + w3 + w4 + w5;

    // Cua gạt cực nhạy: ít nhất 2 mắt bên cua chạm vạch, mắt đối diện trắng
    bool det_sl = (w1 && w2 && !w5);
    bool det_sr = (w4 && w5 && !w1);
    bool det_lost = (sum == 0);

    if (sum >= 4) {
        line_state = LINE_CROSS;
    } else if (det_sl) {
        line_state = LINE_SHARP_L;
    }
}

```

```

    last_sharp_dir = LINE_SHARP_L;
} else if (det_sr) {
    line_state = LINE_SHARP_R;
    last_sharp_dir = LINE_SHARP_R;
} else if (det_lost) {
    line_state = LINE_LOST;
} else {
    line_state = LINE_ON;
    if (sum > 0 && w3) {
        last_sharp_dir = LINE_ON;
    }
}

if (sum == 0) return 0;
int16_t pos_sum = (int16_t)w1 * (-10) + (int16_t)w2 * (-5) + (int16_t)w3 *
0 + (int16_t)w4 * 5 + (int16_t)w5 * 10;
return (float)pos_sum / (float)sum;
}

// ===== 12. ÁP DỤNG TỐC ĐỘ MUỘT =====
void applyMotorSpeed() {
    v_M1_curr = slew_lin(v_M1_curr, period_to_v(target_period_M1),
VEL_SLEW);
    v_M3_curr = slew_lin(v_M3_curr, period_to_v(target_period_M3),
VEL_SLEW);
    Speed_M1(v_to_period(v_M1_curr));
    Speed_M3(v_to_period(v_M3_curr));
}

// ===== 13. THUẬT TOÁN PID DÒ LINE =====
void runLinePID() {
    static unsigned long last_pid_time = 0;
    unsigned long current_time = millis();

    // Chạy PID mỗi 5ms
    if (current_time - last_pid_time >= 5) {
        last_pid_time = current_time;
        error_pid = readLinePosition();

        switch (line_state) {

            case LINE_SHARP_L:
                target_period_M1 = PIVOT_FORWARD; // bánh phải tiến
                target_period_M3 = PIVOT_REVERSE; // bánh trái lùi -> xoay trái

```

```

previous_error = error_pid;
break;

case LINE_SHARP_R:
    target_period_M1 = PIVOT_REVERSE; // bánh phải lùi
    target_period_M3 = PIVOT_FORWARD; // bánh trái tiến -> xoay phải
    previous_error = error_pid;
    break;

case LINE_CROSS:
    // Luợt qua ngã tư (ngã tư sẽ được phát hiện riêng ở loop để xử lý theo
Pi)
    target_period_M1 = BASE_PERIOD;
    target_period_M3 = BASE_PERIOD;
    previous_error = error_pid;
    break;

case LINE_LOST:
    // Chiến thuật nhớ: ưu tiên hướng của gắt gần nhất
    if (last_sharp_dir == LINE_SHARP_L) {
        target_period_M1 = PIVOT_FORWARD;
        target_period_M3 = PIVOT_REVERSE;
    } else if (last_sharp_dir == LINE_SHARP_R) {
        target_period_M1 = PIVOT_REVERSE;
        target_period_M3 = PIVOT_FORWARD;
    } else {
        if (previous_error < 0) {
            target_period_M1 = PIVOT_FORWARD;
            target_period_M3 = PIVOT_REVERSE;
        } else {
            target_period_M1 = PIVOT_REVERSE;
            target_period_M3 = PIVOT_FORWARD;
        }
    }
    break;

case LINE_ON:
default:
    P = error_pid;
    D = error_pid - previous_error;
    PID_value = Kp * P + Kd * D;
    previous_error = error_pid;

```

```

    int adaptive_base = BASE_PERIOD + (int)(fabs(error_pid) *
TURN_SLOWDOWN);

    target_period_M1 = adaptive_base + (int)PID_value;
    target_period_M3 = adaptive_base - (int)PID_value;

    if (target_period_M1 > 0) target_period_M1 =
constrain(target_period_M1, MIN_PERIOD, MAX_PERIOD);
    else target_period_M1 = constrain(target_period_M1, -MAX_PERIOD,
-MIN_PERIOD);

    if (target_period_M3 > 0) target_period_M3 =
constrain(target_period_M3, MIN_PERIOD, MAX_PERIOD);
    else target_period_M3 = constrain(target_period_M3, -MAX_PERIOD,
-MIN_PERIOD);
    break;
}
}

    applyMotorSpeed();
}

// ===== 14. CUA GẮT TRÁI/PHẢI THEO KIỂU PIVOT (ĐÃ FIX LỖI
XOAY MÙ) =====
void pivotTurn(bool turn_left, unsigned long duration_ms) {
    previous_error = 0;
    last_sharp_dir = LINE_ON;

    if (turn_left) {
        target_period_M1 = PIVOT_FORWARD;
        target_period_M3 = PIVOT_REVERSE;
    } else {
        target_period_M1 = PIVOT_REVERSE;
        target_period_M3 = PIVOT_FORWARD;
    }

    unsigned long start = millis();
    bool left_cross_zone = false;

    while (millis() - start < duration_ms) {
        applyMotorSpeed();

        // BẮT BUỘC "XOAY MÙ" TRONG 250ms ĐẦU TIÊN
        // Để cày văng bánh xe ra khỏi vạch thẳng, không bị báo nhầm w3

```



```

if (millis() - start > 150) {

    uint8_t w1 = !digitalRead(Pin_ss1);
    uint8_t w2 = !digitalRead(Pin_ss2);
    uint8_t w3 = !digitalRead(Pin_ss3);
    uint8_t w4 = !digitalRead(Pin_ss4);
    uint8_t w5 = !digitalRead(Pin_ss5);
    uint8_t sum = w1 + w2 + w3 + w4 + w5;

    if (!left_cross_zone) {
        // ÉP BUỘC: Phải ra vùng trắng hoàn toàn (sum == 0) mới tính là thoát
        ngã tư cũ
        if (sum == 0) left_cross_zone = true;
        } else {
            // Đã thoát ngã tư, nếu quét trúng line mới thì break
            if (w3 && sum <= 3) {
                break;
            }
        }
        delay(1);
    }

    if (turn_left) last_sharp_dir = LINE_SHARP_L;
    else last_sharp_dir = LINE_SHARP_R;

    previous_error = 0;
}

// ===== 15. DỪNG ĐỘNG CƠ AN TOÀN =====
void stopMotors() {
    target_period_M1 = 0;
    target_period_M3 = 0;
    unsigned long t0 = millis();
    while (millis() - t0 < 100) {
        applyMotorSpeed();
        delay(1);
    }
    Speed_M1(0); Speed_M3(0);
    v_M1_curr = 0; v_M3_curr = 0;
}

// ===== 16. CHẠY MOTOR THÔ =====
void rawDrive(int s1, int s3, unsigned long ms) {

```

```

v_M1_curr = period_to_v(s1);
v_M3_curr = period_to_v(s3);
target_period_M1 = s1;
target_period_M3 = s3;
Speed_M1(s1);
Speed_M3(s3);
delay(ms);
}

// ===== 17. ĐỌC CẢM BIẾN KIỂU CŨ =====
int readSensorLegacy() {
    int ss1 = digitalRead(Pin_ss1);
    int ss2 = digitalRead(Pin_ss2);
    int ss3 = digitalRead(Pin_ss3);
    int ss4 = digitalRead(Pin_ss4);
    int ss5 = digitalRead(Pin_ss5);

    if (ss1==0 && ss2==0 && ss3==0 && ss4==0 && ss5==0) return 8;
    return 0;
}

void setup() {
    // Motor & Dò line
    pinMode(Enable, OUTPUT);
    pinMode(Step_1, OUTPUT); pinMode(Dir_1, OUTPUT);
    pinMode(Step_3, OUTPUT); pinMode(Dir_3, OUTPUT);
    pinMode(Pin_ss1, INPUT); pinMode(Pin_ss2, INPUT);
    pinMode(Pin_ss3, INPUT); pinMode(Pin_ss4, INPUT);
    pinMode(Pin_ss5, INPUT);
    digitalWrite(Enable, LOW);

    // Servo & Tiệm cận
    pinMode(SENSOR_TIEM_CAN, INPUT_PULLUP);
    myServo.attach(SERVO_PIN);
    myServo.write(goc_mo);

    // Timer2 cho Step Motor
    TCCR2A = 0; TCCR2B = 0;
    TCCR2B |= (1 << CS21); OCR2A = 39;
    TCCR2A |= (1 << WGM21); TIMSK2 |= (1 << OCIE2A);

    last_sensor_state = digitalRead(SENSOR_TIEM_CAN);

    Serial.begin(9600);

```

```

}

void loop() {
  // ===== LẮNG NGHE DỮ LIỆU TỪ RASPBERRY PI =====
  if (Serial.available() > 0) {
    char data = Serial.read();
    if (data == '\n' || data == '\r') return;

    if (khoa_tin_hieu == false) {
      if (data != last_data) {
        last_data = data;
        if (data == '1' || data == '2' || data == '3') {
          vat_nhan_dien = data;
          Serial.print("Da nhan loai vat: ");
          Serial.println(vat_nhan_dien);
        }
      }
    }
  }
  else {
    Serial.println("Dang kep vat - Bo qua tin hieu moi tu Pi");
  }
}

// ===== UՍ TIỀN 1: Cảm biến tiệm cận để gắp/thả =====
int current_sensor_state = digitalRead(SENSOR_TIEM_CAN);
if (current_sensor_state == LOW && last_sensor_state == HIGH) {
  stopMotors();
  Serial.println("DUNG ROBOT DE GAP/NHA");
  delay(2000);
  dang_kep = !dang_kep;

  if (dang_kep) {
    Serial.println("-> KEP VAT");
    myServo.write(goc_dong);
    delay(1000);
    khoa_tin_hieu = true;
    dang_ve_nguon = false;

    rawDrive(-30, -30, 1500);
    rawDrive(35, -35, 1400);
  } else {
    Serial.println("-> NHA VAT");
    myServo.write(goc_mo);
    delay(1000);
  }
}

```

```

    dang_ve_nguon = true;

    rawDrive(-50, -50, 2500);
    rawDrive(45, -45, 2300);
}

// Clear buffer Serial để tránh bị kẹt data cũ từ Pi sau 5s delay
while(Serial.available() > 0) Serial.read();

// Reset trạng thái PID sau khi xong pha hậu kẹp/nhả
previous_error = 0;
last_sharp_dir = LINE_ON;
v_M1_curr = 0;
v_M3_curr = 0;

delay(3000);
}
else {
    // ===== UU TIÊN 2: Dò line + Xử lý ngã tư =====
    int error_status = readSensorLegacy();
    if (error_status == 8) { // GẶP NGÃ TƯ (00000)

        if (dang_ve_nguon == false) { // ĐANG ĐI GIAO HÀNG
            if (vat_nhan_dien == '1') {
                Serial.println("RE TRAI VAO 1 (PIVOT)");
                rawDrive(BASE_PERIOD, BASE_PERIOD, 250);
                pivotTurn(true, 2500);
            }
            else if (vat_nhan_dien == '2') {
                Serial.println("DI THANG VAO 2");
                rawDrive(BASE_PERIOD, BASE_PERIOD, 1000);
            }
            else if (vat_nhan_dien == '3') {
                Serial.println("RE PHAI VAO 3 (PIVOT)");
                rawDrive(BASE_PERIOD, BASE_PERIOD, 250);
                pivotTurn(false, 2500);
            }
        }
        else { // ĐANG QUAY VỀ NGUỒN
            if (vat_nhan_dien == '1') {
                Serial.println("VE NGUON: RE PHAI (PIVOT)");
                rawDrive(BASE_PERIOD, BASE_PERIOD, 200);
                pivotTurn(false, 2200);
            }

```

```

else if (vat_nhan_dien == '3') {
    Serial.println("VE NGUON: RE TRAI (PIVOT)");
    rawDrive(BASE_PERIOD, BASE_PERIOD, 250);
    pivotTurn(true, 2200);
}
else {
    Serial.println("VE NGUON: DI THANG");
    rawDrive(BASE_PERIOD, BASE_PERIOD, 1000);
}
khoa_tin_hieu = false;
}

// Reset PID state sau khi xử lý ngã tư
previous_error = 0;
v_M1_curr = 0;
v_M3_curr = 0;
last_sharp_dir = LINE_ON;
}
else {
    // Dò line bình thường qua thuật toán PID nâng cấp
    runLinePID();
}
}

last_sensor_state = current_sensor_state;
}

// ===== 18. NGẮT TIMER PHÁT XUNG MOTOR =====
ISR(TIMER2_COMPA_vect) {
    if (Dir_M1 != 0) {
        Count_timer1++;
        if (Count_timer1 <= Count_TOP1) PORTD |= 0b00100000;
        else PORTD &= 0b11011111;
        if (Count_timer1 > Count_BOT1) Count_timer1 = 0;
    }
    if (Dir_M3 != 0) {
        Count_timer3++;
        if (Count_timer3 <= Count_TOP3) PORTD |= 0b10000000;
        else PORTD &= 0b01111111;
        if (Count_timer3 > Count_BOT3) Count_timer3 = 0;
    }
}
}

```

Phụ lục 2: Mã nguồn trí tuệ nhân tạo xác định vật thể

```
# -*- coding: utf-8 -*-
import cv2
import time
import numpy as np
import tflite_runtime.interpreter as tflite
import serial
import threading

# =====
# 1. CẤU HÌNH SERIAL (KHỚP VỚI ARDUINO)
# =====
try:
    arduino = serial.Serial(port='/dev/ttyUSB0', baudrate=9600, timeout=0.1)
    print("[INFO] Ket noi Arduino thanh cong!")
except:
    print("[WARNING] Khong tim thay Arduino tai /dev/ttyUSB0.")
    arduino = None

# =====
# 2. CẤU HÌNH MODEL & CLASSES
# =====
MODEL_PATH = 'best_int8.tflite'
CONF_THRESHOLD = 0.61
NMS_THRESHOLD = 0.45
CLASSES = ['hop_thuoc', 'khoi_mau', 'nuoc_ngot']

# ----- TỐI ƯU 1: Dùng 2 thread cho TFLite -----
interpreter = tflite.Interpreter(model_path=MODEL_PATH, num_threads=2)
interpreter.allocate_tensors()

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
input_size = input_details[0]['shape'][1]
input_dtype = input_details[0]['dtype']
is_int8 = input_dtype in (np.int8, np.uint8)
input_index = input_details[0]['index']
output_index = output_details[0]['index']

# Tính sẵn ngưỡng confidence (tránh nhân mỗi frame)
effective_conf = CONF_THRESHOLD * 255 if is_int8 else
CONF_THRESHOLD
```

```
# =====
# 3. TỐI ƯU 2: CAMERA THREAD (Đọc ảnh song song)
# =====
# Tách camera ra thread riêng -> main thread không bao giờ phải chờ frame
class CameraThread:
    def __init__(self, src=0, width=320, height=240):
        self.cap = cv2.VideoCapture(src)
        # Giảm resolution camera -> ít pixel cần xử lý hơn
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
        # Giảm buffer camera xuống 1 -> luôn lấy frame mới nhất
        self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)

        self.ret = False
        self.frame = None
        self.lock = threading.Lock()
        self.running = True

        # Đọc frame đầu tiên
        self.ret, self.frame = self.cap.read()

        # Khởi chạy thread đọc camera
        self.thread = threading.Thread(target=self._update, daemon=True)
        self.thread.start()

    def _update(self):
        while self.running:
            ret, frame = self.cap.read()
            with self.lock:
                self.ret = ret
                self.frame = frame

    def read(self):
        with self.lock:
            return self.ret, self.frame.copy() if self.frame is not None else None

    def release(self):
        self.running = False
        self.thread.join(timeout=2)
        self.cap.release()

# =====
# 4. CẤU HÌNH HIỂN THỊ & CHỐNG SPAM
# =====
```

```
# Bật/tắt hiển thị (TẮT khi chạy thật để tăng FPS đáng kể)
```

```
SHOW_DISPLAY = True
```

```
# TỐI ƯU 3: Chống spam Serial - chỉ gửi khi đổi kết quả hoặc sau cooldown
```

```
last_sent_char = '0'
```

```
last_sent_time = 0
```

```
SEND_COOLDOWN = 0.5 # Gửi tối đa 2 lần/giây (giây)
```

```
# TỐI ƯU 4: Đếm FPS
```

```
fps_counter = 0
```

```
fps_time_start = time.time()
```

```
current_fps = 0
```

```
# =====
```

```
# 5. TỐI ƯU 5: PRE-ALLOCATE BUFFER
```

```
# =====
```

```
# Tạo sẵn mảng input để tránh np.expand_dims + astype mỗi frame
```

```
input_buffer = np.empty((1, input_size, input_size, 3), dtype=input_dtype)
```

```
# =====
```

```
# 6. LUỒNG XỬ LÝ CHÍNH (ĐÃ TỐI ƯU)
```

```
# =====
```

```
cam = CameraThread(src=0, width=320, height=240)
```

```
print(f"[INFO] Camera: 320x240 | Model input: {input_size}x{input_size} |  
INT8: {is_int8}")
```

```
print(f"[INFO] Hiện thị: {'BAT' if SHOW_DISPLAY else 'TAT'} | Nhận 'q'  
để thoát")
```

```
try:
```

```
    while True:
```

```
        ret, frame = cam.read()
```

```
        if not ret or frame is None:
```

```
            continue
```

```
        h_orig, w_orig = frame.shape[:2]
```

```
        # --- TIỀN XỬ LÝ ẢNH (Tối ưu: ghi thẳng vào buffer) ---
```

```
        img_resized = cv2.resize(frame, (input_size, input_size))
```

```
        img_rgb = cv2.cvtColor(img_resized, cv2.COLOR_BGR2RGB)
```

```
        if is_int8:
```

```
            input_buffer[0] = img_rgb
```

```
        else:
```

```
            np.divide(img_rgb, 255.0, out=input_buffer[0], casting='unsafe')
```



```

# --- INFERENCE ---
interpreter.set_tensor(input_index, input_buffer)
interpreter.invoke()
output = interpreter.get_tensor(output_index)[0].transpose()

# --- HẬU XỬ LÝ (Tối ưu: lọc bằng numpy thay vì vòng for) ---
class_scores = output[:, 4:]
class_ids_all = np.argmax(class_scores, axis=1)
confs_all = class_scores[np.arange(len(class_scores)), class_ids_all]

# Lọc theo ngưỡng confidence (vectorized, nhanh hơn for loop)
mask = confs_all > effective_conf

if np.any(mask):
    filtered_output = output[mask]
    filtered_confs = confs_all[mask].astype(float)
    filtered_ids = class_ids_all[mask]

    # Tính bounding boxes (vectorized)
    cx = filtered_output[:, 0]
    cy = filtered_output[:, 1]
    bw = filtered_output[:, 2]
    bh = filtered_output[:, 3]

    x_arr = ((cx - bw / 2) * w_orig / input_size).astype(int)
    y_arr = ((cy - bh / 2) * h_orig / input_size).astype(int)
    w_arr = (bw * w_orig / input_size).astype(int)
    h_arr = (bh * h_orig / input_size).astype(int)

    boxes = np.stack([x_arr, y_arr, w_arr, h_arr], axis=1).tolist()
    confidences = filtered_confs.tolist()

    # NMS
    indices = cv2.dnn.NMSBoxes(boxes, confidences,
CONF_THRESHOLD, NMS_THRESHOLD)

    if len(indices) > 0:
        now = time.time()
        for i in indices.flatten():
            c_id = int(filtered_ids[i])
            send_char = str(c_id + 1)

```

```

        # TỐI ƯU 3: Chỉ gửi Serial khi khác kết quả cũ HOẶC đã hết
cooldown
        if arduino and (send_char != last_sent_char or (now -
last_sent_time) > SEND_COOLDOWN):
            arduino.write(send_char.encode())
            last_sent_char = send_char
            last_sent_time = now
            print(f"GUI -> {CLASSES[c_id]}: Ma {send_char} | FPS:
{current_fps:.1f}")

        # Vẽ khung (chỉ khi bật hiển thị)
        if SHOW_DISPLAY:
            bx, by, bw_d, bh_d = boxes[i]
            cv2.rectangle(frame, (bx, by), (bx + bw_d, by + bh_d), (0, 255,
0), 2)

            cv2.putText(frame, f"{CLASSES[c_id]} {send_char}", (bx, by
- 10),

                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

    # --- HIỂN THỊ (tốn ~5-15ms trên Pi, tắt khi chạy thật) ---
    if SHOW_DISPLAY:
        cv2.putText(frame, f"FPS: {current_fps:.1f}", (10, 25),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
        cv2.imshow("He thong Giao hang Tu dong", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    # --- ĐẾM FPS ---
    fps_counter += 1
    elapsed = time.time() - fps_time_start
    if elapsed >= 1.0:
        current_fps = fps_counter / elapsed
        fps_counter = 0
        fps_time_start = time.time()

finally:
    cam.release()
    if arduino:
        arduino.close()
    cv2.destroyAllWindows()
    print("[INFO] Da dong tat ca ket noi.")

```